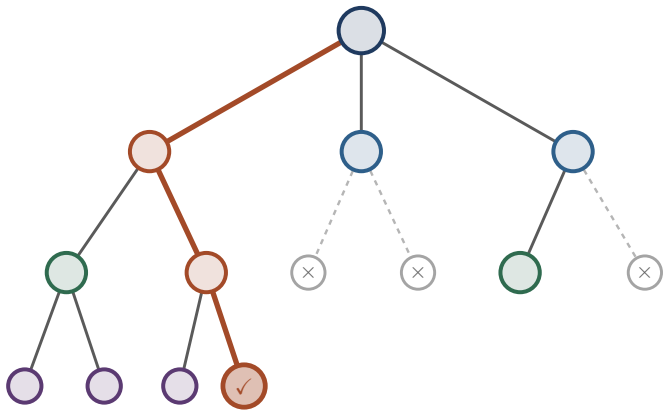
Informatik von Grund auf — Band 12

Der Raum der Möglichkeiten

Eine vollständige Einführung in die symbolische künstliche Intelligenz



Von der Suche bis zum Schluss

Jeder Schritt erklärt. Mit vollständig durchgerechneten Übungsaufgaben.

Informatik von Grund auf — Band 12
Der Raum der Möglichkeiten · Symbolische künstliche Intelligenz

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Informatik von Grund auf, Band 12: Der Raum der Möglichkeiten*. Fassung 1.0.0.
`https://github.com/juergenmaier2026/informatik-von-grund-auf`

Satz: LaTeX mit Palatino. Fassung 1.0.0, Oktober 2026. 164 Seiten.

Stand der Forschung: August 2026

Inhaltsverzeichnis

Vorwort 1

I Suchen 9

1 Zeichen und Zustandsräume 11

2 Suche ohne Vorwissen 19

3 Heuristische Suche 27

4 Suche gegen einen Gegner 37

5 Lokale Suche 47

II Nebenbedingungen und Logik 55

6 Nebenbedingungen 57

7 Aussagenlogik und SAT 65

8 Prädikatenlogik 73

9 Logisches Programmieren 81

III Wissen und Handeln 91

10 Wissen darstellen 93

11 Planen 101

12 Unsicherheit 109

13 Nichtmonotones Schließen 117

IV Bilanz 125

14 Was schiefging und was blieb 127

15 Der Stand im Jahr 2026 **137**

V Anhang **145**

A Formeln und Zahlen **147**

B Landkarte des Bandes **151**

C Glossar **153**

D Zeittafel **155**

E Kommentierte Quellen **157**

Vorwort: Denken als Suchen

Dieser Band handelt von dem Versuch, Denken als Umgang mit Zeichen zu erklären und nachzubauen. *Er behandelt eine Forschungsrichtung, die große Erwartungen weckte, sie nicht erfüllte, und deren Verfahren heute trotzdem überall im Einsatz sind.*

Der Grundgedanke in einem Satz

ALLEN NEWELL und HERBERT SIMON formulierten 1976 eine Behauptung, die dem ganzen Gebiet zugrunde liegt.

> *Ein System, das Zeichen bearbeitet, verfügt über die notwendigen und hinreichenden Mittel für allgemeines intelligentes Handeln.*

Notwendig	Wer intelligent handelt, muss Zeichen bearbeiten
Hinreichend	Wer Zeichen richtig bearbeitet, handelt intelligent

> *Beide Hälften sind bestritten worden, die zweite heftiger als die erste. Sie sind auch keine Sätze im mathematischen Sinn, sondern eine Arbeitshypothese, die man durch Bauen prüfen wollte.*

Die zweite Behauptung derselben Arbeit.

> *Ein solches System löst Aufgaben, indem es Lösungsvorschläge erzeugt und schrittweise abändert, also indem es einen Raum von Möglichkeiten durchsucht.*

Das ist die Namensgeberin dieses Bandes. Fast jedes Verfahren in Teil I und II lässt sich als eine Antwort auf die Frage lesen, wie man einen unvorstellbar großen Raum durchsucht, ohne ihn zu durchlaufen.

Wie groß diese Räume sind

Aufgabe	Zustände	Bemerkung
Acht-Puzzle	$1{,}8 \cdot 10^5$	vollständig durchsuchbar
Fünfzehn-Puzzle	10^{13}	Heuristik nötig
Rubik-Würfel	$4{,}3 \cdot 10^{19}$	
Dame	$5 \cdot 10^{20}$	2007 vollständig gelöst
Schach	$\approx 10^{47}$	Stellungen
Go	$\approx 10^{170}$	Stellungen

Zum Vergleich: Das beobachtbare Weltall enthält etwa 10^{80} Atome. Beim Schach übersteigt die Zahl der Stellungen diese Zahl nicht, die Zahl der möglichen Partien allerdings bei weitem.

Was daraus folgt.

Vollständig durchsuchen	Nur bei kleinen Aufgaben; die Grenze liegt bei etwa 10^{10} Zuständen
Abschneiden	Zweige verwerfen, von denen man zeigen kann, dass sie nichts bringen
Schätzen	Eine Bewertung benutzen, die nicht immer stimmt
Umformulieren	Die Aufgabe so beschreiben, dass der Raum kleiner wird

Die letzte Zeile wird am häufigsten unterschätzt und ist die wirksamste. Eine geschickte Darstellung kann einen Raum von 10^{20} auf 10^6 Zustände zusammenschrumpfen lassen, und keine noch so gute Suche holt das wieder ein.

Was dieses Buch will

Erstens: Die Verfahren durchrechnen, nicht beschreiben.

A*, Alpha-Beta, Resolution, Rückverfolgung und DPLL werden an kleinen Beispielen von Hand ausgeführt, bis das Ergebnis dasteht. Bei jedem wird auch gezeigt, wann es versagt.

Zweitens: Ehrlich sein über das, was nicht funktioniert hat.

> *Dieses Gebiet hat zwei Zusammenbrüche erlebt, in denen die Fördermittel versiegten und die Erwartungen sich als unhaltbar erwiesen. Kapitel 14 geht den Gründen nach, denn sie sind lehrreicher als die Erfolge.*

Drittens: Zeigen, was davon geblieben ist.

Verfahren	Herkunft	heute im Einsatz
A*	1968	Navigation, Robotik
Alpha-Beta	1958	Spielprogramme
SAT-Löser	1962/1999	**Schaltungsprüfung, Planung**
Constraint-Löser	1970er	Dienstpläne, Fertigung
Bayes-Netze	1988	Diagnose, Risikoanalyse
Beschreibungslogik	1990er	Ontologien, Terminologien
Planer	1971	Logistik, Raumfahrt

Kein einziges dieser Verfahren wird heute „künstliche Intelligenz“ genannt. Sobald etwas zuverlässig funktioniert, heißt es Optimierung, Prüfung oder Planung. Das ist eine Eigenart dieses Gebiets, auf die Kapitel 14 zurückkommt.

Drei Warnungen

Erstens: „Symbolisch“ ist kein Gegensatz zu „lernend“.
Die Gegenüberstellung von symbolischer und lernender KI hat historische Gründe und ist sachlich irreführend. Es gibt lernende symbolische Verfahren, und moderne Systeme verbinden beides.

Zweitens: Die Verfahren sind gut, die Wissensbeschaffung ist das Problem.

> *Ein Schlussfolgerungssystem arbeitet zuverlässig auf dem, was man ihm gegeben hat. Die Schwierigkeit lag nie beim Schließen, sondern immer dabei, das Wissen überhaupt in eine Form zu bringen. Kapitel 10 und 14 behandeln das.*

Drittens: Die Größenordnungen täuschen.
Ein Verfahren, das bei zwanzig Variablen in Millisekunden fertig ist, kann bei fünfzig Jahre brauchen. Bei fast allen Verfahren dieses Bandes wächst der Aufwand im schlechtesten Fall exponentiell (Band 3), und die Frage ist stets, wie selten der schlechteste Fall ist.

Zum Aufbau

- **Teil I** (Kap. 1–5): Suchen. Der Grundgedanke, uninformierte und heuristische Suche, Suche gegen einen Gegner, lokale Suche.
- **Teil II** (Kap. 6–9): Nebenbedingungen und Logik. Constraint-Probleme, SAT, Prädikatenlogik, logisches Programmieren.
- **Teil III** (Kap. 10–13): Wissen und Handeln. Wissensrepräsentation, Planen, Unsicherheit, nichtmonotones Schließen.
- **Teil IV** (Kap. 14–15): Bilanz. Was schiefging und was blieb, und der Stand im Jahr 2026.

Ein Sommer, der zehn Jahre dauern sollte

Im Sommer 1956 versammelte sich am Dartmouth College eine kleine Gruppe zu einem Forschungsvorhaben. Der Förderantrag, verfasst von John McCarthy, Marvin Minsky, Nathaniel Rochester und Claude Shannon, enthielt einen bemerkenswerten Satz.

> *Man werde versuchen herauszufinden, wie Maschinen Sprache benutzen, Abstraktionen bilden, Probleme lösen und sich selbst verbessern könnten. Ein erheblicher Fortschritt sei zu erwarten, wenn eine sorgfältig ausgewählte Gruppe einen Sommer lang gemeinsam daran arbeite.*

In diesem Antrag taucht der Ausdruck „artificial intelligence“ zum ersten Mal auf. McCarthy hat später gesagt, er habe einen Namen gebraucht, der das Vorhaben von der damals dominierenden Kybernetik abgrenzte.

Was tatsächlich geschah.

Geplant	Zehn Personen, zwei Monate, entscheidender Durchbruch
Tatsächlich	Wechselnde Teilnehmer, kein gemeinsames Ergebnis, kein Abschlussbericht
Geblieben	Der Name, ein Netzwerk von Personen, und eine Forschungsrichtung

Das einzige lauffähige Programm brachten NEWELL *und* SIMON *mit: den* Logic Theorist*, der Sätze aus den* Principia Mathematica *bewies. Für einen Satz fand er einen Beweis, den die Autoren selbst nicht gefunden hatten. Die eingereichte Notiz darüber wurde von der Fachzeitschrift abgelehnt, unter anderem weil ein Programm als Mitautor genannt war.*

Die Vorhersagen.

Jahr	Vorhersage	Eingetroffen
1957	Schachweltmeister in zehn Jahren	1997, nach vierzig
1965	In zwanzig Jahren jede Arbeit	nein
1967	KI im Wesentlichen gelöst in einer Generation	nein
1970	Maschine mit menschlicher Allgemeinbildung in drei bis acht Jahren	nein

Die erste Zeile ist die interessanteste, weil sie eintrat, nur eben vier Jahrzehnte später und auf eine Weise, die niemand erwartet hatte: nicht durch Nachbau menschlichen Denkens, sondern durch Alpha-Beta-Suche über Millionen Stellungen je Sekunde.

Warum diese Vorhersagen so falsch waren, ist die eigentliche Frage dieses Bandes. Die kurze Antwort steht in Kapitel 14: Man hatte die Verfahren, und man unterschätzte, wie viel Wissen ein Mensch benutzt, ohne es zu bemerken.

Siebzig Jahre symbolische KI

1950	Turing	„Computing Machinery and Intelligence“
1955	Newell, Simon	Logic Theorist
1956	**Dartmouth**	**der Name entsteht**
1958	McCarthy	LISP; „Programs with Common Sense“
1958	Newell, Simon	Alpha-Beta-Idee
1959	Samuel	lernendes Dameprogramm
1960	Davis, Putnam	Verfahren für die Aussagenlogik
1962	Davis, Logemann, Loveland	DPLL
1965	**Robinson**	**Resolution und Unifikation**
1965	Weizenbaum	ELIZA
1968	**Hart, Nilsson, Raphael**	**A***
1969	McCarthy, Hayes	das Rahmenproblem
1971	**Fikes, Nilsson**	**STRIPS**
1972	Colmerauer, Roussel	Prolog
1972	Dreyfus	„What Computers Can’t Do“
1974	Minsky	Rahmen als Wissensform
1974	Kowalski	Logik als Programmiersprache
1974–80	**Erster KI-Winter**	**Lighthill-Bericht 1973**
1976	Newell, Simon	Symbolsystemhypothese
1976	Shortliffe	MYCIN

1980	Reiter	Default-Logik
1980	Searle	Das chinesische Zimmer
1980–87	**Expertensysteme**	**XCON spart Millionen**
1983	Kirkpatrick u. a.	simuliertes Abkühlen
1984	Lenat	Cyc beginnt
1985	Korf	iterative Vertiefung mit A*
1987–93	**Zweiter KI-Winter**	**LISP-Maschinen brechen ein**
1988	**Pearl**	**Bayes-Netze**
1991	Brooks	„Intelligence Without Representation“
1997	**Deep Blue**	**schlägt Kasparow**
1999	Marques-Silva, Sakallah	konfliktgetriebenes Lernen in SAT
2001	Chaff	SAT-Löser werden praktisch brauchbar
2004	OWL	Beschreibungslogik im Netz
2006	Monte-Carlo-Baumsuche	Durchbruch bei Go
2016	**AlphaGo**	**Suche und Lernen verbunden**
2020er	Neurosymbolik	Löser als Werkzeug von Sprachmodellen
2026	—	Prüfen statt Raten

Zwei Zeilen verdienen eine Anmerkung.
1965, Robinson. Ein einziges Schlussprinzip, das für die gesamte Prädikatenlogik ausreicht, zusammen mit einem Verfahren, das zwei Ausdrücke möglichst allgemein gleichmacht. Aus dieser Arbeit gingen Prolog, die automatischen Beweiser und die Typprüfung aus Band 6 hervor.
1999 und 2001. SAT-Löser galten als aussichtslos, weil das Problem NP-vollständig ist (Band 3). Dann kamen konfliktgetriebenes Lernen und geschickte Datenstrukturen, und die Leistung stieg innerhalb weniger Jahre um mehrere Größenordnungen. Es ist das eindrucksvollste Gegenbeispiel zu der Annahme, ein schweres Problem sei praktisch unlösbar.

Sobald es funktioniert, nennt es niemand mehr künstliche Intelligenz.

— eine Beobachtung, die JOHN MCCARTHY zugeschrieben wird
und als KI-Effekt *bekannt ist. Sie erklärt, warum das Gebiet*
als gescheitert gilt, obwohl seine Verfahren
in jedem Navigationsgerät stecken.

Part I

Suchen

1 Zeichen und Zustandsräume

Was ein Zeichensystem ist

Newell und Simon meinten mit einem Zeichensystem etwas Bestimmtes und Bescheidenes.

Zeichen	Physische Muster, die sich unterscheiden lassen
Ausdrücke	Zeichen, zu Strukturen zusammengesetzt
Vorgänge	Erzeugen, kopieren, verändern, vergleichen, zerstören
Bezeichnung	Ein Ausdruck kann auf etwas verweisen
Deutung	Ein Ausdruck kann einen Vorgang bezeichnen und ihn auslösen

Die letzten beiden Punkte sind die interessanten. Ein System, dessen Ausdrücke Vorgänge bezeichnen und die es ausführen kann, kann sich selbst verändern. Es ist derselbe Selbstbezug, der in Band 2 eine Grenze markierte und in Band 6 ein Werkzeug war.

Warum LISP.
McCarthy entwarf 1958 eine Sprache, in der Programme dieselbe Gestalt haben wie Daten: Listen (Band 6, Quelle 18).

	übliche Sprache	**LISP**
Programm ist	Text	**eine Datenstruktur**
Programm erzeugen	Text zusammensetzen	Liste bauen
Programm prüfen	Zerleger nötig	durchlaufen

Für ein Gebiet, dessen Grundthese lautet, Denken sei Zeichenverarbeitung, war eine Sprache naheliegend, in der ein Programm ein Zeichenausdruck wie jeder andere ist. Dass LISP dreißig Jahre lang die Sprache der KI blieb, ist keine Zufälligkeit.

Die Hypothese ist nicht beweisbar

Weder die notwendige noch die hinreichende Hälfte lässt sich mathematisch prüfen. Sie sind empirische Behauptungen über die Welt.

Einwand von Dreyfus	Menschliches Können beruht auf Vertrautheit mit Situationen, nicht auf Regeln über Zeichen
Einwand von Searle	Zeichen zu verarbeiten heißt nicht, sie zu verstehen
Einwand von Brooks	Intelligenz braucht einen Körper und eine Umwelt, keine innere Weltbeschreibung

Alle drei Einwände sind ernst zu nehmen und keiner ist entschieden. Für die Verfahren dieses Bandes ist das ohne Belang: Ein Planer funktioniert oder nicht, unabhängig davon, ob er versteht, was er plant.

Der Begleitband dieser Reihe kommt auf die Frage zurück. Hier genügt die Feststellung, dass die Hypothese als Forschungsprogramm außerordentlich fruchtbar war, gleichgültig ob sie zutrifft.

Der Zustandsraum

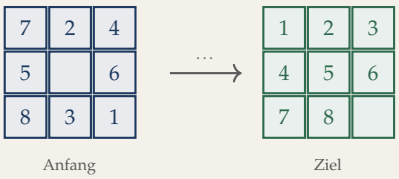
Fast jede Aufgabe dieses Bandes lässt sich in dieselbe Form bringen.

Zustände	Die Menge der möglichen Lagen
Anfangszustand	Wo man beginnt
Handlungen	Was in einem Zustand möglich ist
Übergänge	Welcher Zustand daraus folgt
Zieltest	Woran man erkennt, dass man fertig ist
Kosten	Was ein Schritt kostet

Damit ist die Aufgabe ein Graph im Sinne von Band 11, und die Suche ist eine Durchmusterung. Der Unterschied zu dort: Der Graph liegt nicht vor, sondern entsteht während der Suche, und er ist meist zu groß, um je vollständig zu existieren.

	Band 11	**hier**
Graph	liegt vor	**wird erzeugt**
Größe	bekannt	meist unbekannt
Speicherbar	ja	nein
Gefragt	Struktur	**ein Weg**

Ein Beispiel: das Acht-Puzzle.



Zustände: 9!/2 = 181 440
Verzweigung: 2–4
Tiefe: bis 31

Die Teilung durch zwei ist bemerkenswert: Von den $9! = 362\,880$ *Anordnungen ist nur die Hälfte vom Ziel aus erreichbar. Die Parität der Vertauschung ist eine Erhaltungsgröße, und wer das übersieht, sucht bei der Hälfte aller Aufgaben unendlich lange.*

Warum die Darstellung alles entscheidet

Dieselbe Aufgabe lässt sich sehr verschieden beschreiben, und der Suchraum ändert sich dabei um Größenordnungen.

Das Beispiel der verstümmelten Schachbrettes.
Von einem Schachbrett werden zwei gegenüberliegende Eckfelder entfernt. Lässt sich der Rest mit 31 Dominosteinen zu je zwei Feldern lückenlos bedecken?

Als Suche	Alle Belegungen durchprobieren; astronomisch viele Möglichkeiten
Mit Umdeutung	Jeder Stein bedeckt ein weißes und ein schwarzes Feld. Die entfernten Ecken haben **dieselbe** Farbe. Also bleiben 32 der einen und 30 der anderen Farbe. Unmöglich.

Ein Satz genügt statt einer erschöpfenden Suche. Die Umdeutung besteht darin, eine Erhaltungsgröße zu erkennen, und genau darin liegt die Schwierigkeit: Kein Suchverfahren findet sie, weil sie nicht im Zustandsraum liegt, sondern in dessen Beschreibung.

Verfahren	**Aufwand**	
Erschöpfende Suche	$> 10^{15}$ Schritte	aussichtslos
Mit Symmetrieabgleich	$\approx 10^{9}$	lange
Färbungsargument	**1 Schritt**	sofort

Das ist die Kehrseite der Symbolsystemhypothese. Sie sagt, dass Zeichenverarbeitung genügt, und sagt nichts darüber, welche *Zeichen man wählen soll. Die Wahl der Darstellung war und ist der Engpass des ganzen Gebiets.*

Aufgabe 1.1.

(a) Modellieren Sie eine Aufgabe als Zustandsraum.
(b) Rechnen Sie aus, wie schnell ein Suchraum wächst.
(c) Zeigen Sie, wie eine bessere Darstellung den Raum verkleinert.

Lösung Schritt für Schritt.

(a) Ein Zustandsraum.

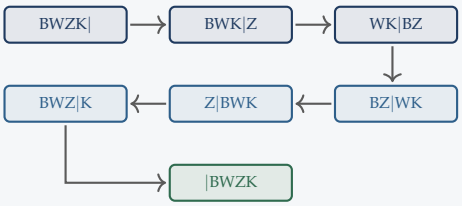
Die Aufgabe. Ein Bauer will Wolf, Ziege und Kohl über einen Fluss bringen. Das Boot fasst ihn und höchstens einen Mitreisenden. Ohne Aufsicht frisst der Wolf die Ziege und die Ziege den Kohl.
Die Zustände. Wer ist auf welcher Seite? Vier Beteiligte, zwei Seiten:

$$2^4 = 16 \text{ mögliche Zustände.}$$

Die gültigen Zustände.

Bedingung	**scheidet aus**
Alle Kombinationen	16
Wolf und Ziege ohne Bauer	-2
Ziege und Kohl ohne Bauer	-2
Gültig	**10**

Der Suchgraph.



Der kürzeste Weg hat sieben Überfahrten. Bemerkenswert ist, dass er einen Schritt enthält, der rückwärts führt: Der Bauer bringt die Ziege wieder zurück. Ein gieriges Verfahren, das nie einen Rückschritt zulässt, findet die Lösung nicht.

Das ist die kleinste Aufgabe, an der sich zeigt, warum lokale Suche (Kapitel 5) versagen kann und systematische Suche nicht.

(b) Wie schnell ein Suchraum wächst.
Die Grundformel. Bei Verzweigungsgrad b und Tiefe d:

$$N = \frac{b^{d+1} - 1}{b - 1} \approx b^d \quad \text{für } b > 1.$$

d	$b = 2$	$b = 10$	$b = 35$
2	7	111	1261
4	31	11 111	$1{,}5 \cdot 10^{6}$
8	511	$1{,}1 \cdot 10^{8}$	$2{,}3 \cdot 10^{12}$
12	8191	$1{,}1 \cdot 10^{12}$	$3{,}4 \cdot 10^{18}$
16	131 071	$1{,}1 \cdot 10^{16}$	$5{,}2 \cdot 10^{24}$

Was das an Zeit und Speicher bedeutet.
Bei 10^{7} erzeugten Knoten je Sekunde und 100 Byte je Knoten:

d $(b = 10)$	**Knoten**	**Zeit**	**Speicher**
6	$1{,}1 \cdot 10^{6}$	0,1 s	110 MB
8	$1{,}1 \cdot 10^{8}$	11 s	11 GB
10	$1{,}1 \cdot 10^{10}$	18 min	1,1 TB
12	$1{,}1 \cdot 10^{12}$	31 h	110 TB

Der Speicher wird vor der Zeit zum Hindernis. Bei Tiefe acht könnte man elf Sekunden warten, und man kann keine elf Gigabyte Knoten aufbewahren. Genau darum geht es bei der iterativen Vertiefung in Kapitel 2: Zeit gegen Speicher zu tauschen.

Wie viel eine Halbierung des Verzweigungsgrads bringt.

b	b^{12}	**Verhältnis**
10	10^{12}	1
5	$2{,}4 \cdot 10^{8}$	$1/4100$
3	$5{,}3 \cdot 10^{5}$	$1/1{,}9 \cdot 10^{6}$
2	4096	$1/2{,}4 \cdot 10^{8}$

Ein Verfahren, das den Verzweigungsgrad von zehn auf fünf senkt, ist bei Tiefe zwölf viertausendmal schneller. Das ist die Wirkung von Alpha-Beta in Kapitel 4 und von Konsistenzprüfungen in Kapitel 6.

(c) Eine bessere Darstellung.

Die Aufgabe. Acht Damen auf ein Schachbrett stellen, so dass keine eine andere schlägt.
Darstellung 1 – Naiv. Acht Damen auf 64 Felder:

$$\binom{64}{8} = 4\,426\,165\,368 \approx 4{,}4 \cdot 10^9.$$

Darstellung 2 – Eine Dame je Spalte.
Da in jeder Spalte genau eine Dame stehen muss, genügt für jede Spalte die Zeilennummer:

$$8^8 = 16\,777\,216 \approx 1{,}7 \cdot 10^7.$$

Darstellung 3 – Eine Permutation.
Da auch in jeder Zeile genau eine Dame steht, ist die Lösung eine Permutation:

$$8! = 40\,320.$$

Darstellung 4 – Mit Symmetrie.
Das Brett hat acht Symmetrien (Drehungen und Spiegelungen):

$$\frac{40\,320}{8} = 5040.$$

Darstellung	Raum	Faktor zur ersten
Naiv	$4{,}4 \cdot 10^9$	1
Je Spalte eine	$1{,}7 \cdot 10^7$	264
Permutation	40 320	109 777
Mit Symmetrie	**5040**	**878 207**

Faktor achthunderttausend, und zwar allein durch Nachdenken über die Beschreibung. Kein Suchverfahren der Welt macht diesen Unterschied wett. Bemerkenswert ist auch, dass jeder Schritt eine Einsicht über die Aufgabe verwendet, keine Programmiertechnik.

Die Einsichten sind nicht umsonst

Schritt	Voraussetzung	Gefahr
Je Spalte eine	folgt aus der Aufgabe	keine
Permutation	folgt ebenso	keine
Symmetrie	**gilt nur beim Zählen**	falsch beim Aufzählen

Wenn man alle Lösungen aufzählen will, darf man die Symmetrie nicht herausrechnen, sondern muss sie am Ende wieder anwenden. Und die acht Symmetrien bilden nicht immer acht verschiedene Lösungen; symmetrische Lösungen fallen mit sich selbst zusammen.

Beim Acht-Damen-Problem gibt es 92 Lösungen, die in 12 Klassen zerfallen. Aus zwölf mal acht ergäben sich 96, und die Abweichung stammt von einer Lösung, die eine Drehsymmetrie besitzt. Solche Sonderfälle sind eine verbreitete Fehlerquelle beim Ausnutzen von Symmetrien.

Newell und Simon[23] formulierten die Hypothese; McCarthy[20] stellte das Programm der logikbasierten KI auf; Russell und Norvig[28] sind das maßgebliche Lehrbuch und behandeln die Zustandsraumdarstellung ausführlich.

2 Suche ohne Vorwissen

Der gemeinsame Rahmen

Alle Suchverfahren dieses Kapitels haben denselben Aufbau und unterscheiden sich in einer einzigen Zeile.

```
rand = {Anfangszustand}
while rand nicht leer:
  n = entnimm(rand)      hier entscheidet sich alles
  if ziel(n):  return weg(n)
  for k in nachfolger(n):
rand.füge_ein(k)
```

Verfahren	Rand ist eine	entnimmt
Breitensuche	Warteschlange	den ältesten
Tiefensuche	Stapel	den jüngsten
Kostensuche	Vorrangwarte-schlange	den billigsten
A* (Kap. 3)	Vorrangwarte-schlange	den vielversprechendsten

Vier grundverschiedene Verfahren, ein Programm, eine geänderte Datenstruktur. Das ist der Grund, warum sich diese Verfahren gemeinsam behandeln lassen, und zugleich der Grund, warum man sie leicht verwechselt.

Die vier Bewertungsfragen

Jedes Suchverfahren wird an denselben vier Fragen gemessen.

Vollständig	Findet es eine Lösung, wenn es eine gibt?
Optimal	Findet es die beste?
Zeitaufwand	Wie viele Knoten werden erzeugt?
Speicheraufwand	Wie viele müssen gleichzeitig aufbewahrt werden?

Verfahren	vollst.	opt.	Zeit	Speicher
Breitensuche	ja	ja*	$\mathcal{O}(b^d)$	$\mathcal{O}(b^d)$
Tiefensuche	nein	nein	$\mathcal{O}(b^m)$	$\mathcal{O}(bm)$
Tiefenbegrenzt	nein	nein	$\mathcal{O}(b^\ell)$	$\mathcal{O}(b\ell)$
Iterative Vertiefung	**ja**	**ja***	$\mathcal{O}(b^d)$	$\mathcal{O}(\mathbf{bd})$
Kostensuche	ja	ja	$\mathcal{O}(b^{1+C/\varepsilon})$	hoch
Zweiseitig	ja	ja*	$\mathcal{O}(b^{d/2})$	$\mathcal{O}(b^{d/2})$

* *nur bei gleichen Schrittkosten*

Die hervorgehobene Zeile ist das bemerkenswerteste Ergebnis des Kapitels. Die iterative Vertiefung hat die Vollständigkeit der Breitensuche und den Speicherbedarf der Tiefensuche, und ihr Zeitaufwand liegt in derselben Größenordnung wie der der Breitensuche.

Warum das Wiederholen fast nichts kostet.
Bei der iterativen Vertiefung wird die Suche bis Tiefe 1, dann bis 2, dann bis 3 wiederholt. Man rechnet also mehrfach dasselbe.

$$N_{\mathrm{ID}} = \sum_{i=0}^{d} (d+1-i)\, b^i \quad \text{gegenüber} \quad N_{\mathrm{BFS}} = \sum_{i=0}^{d} b^i.$$

b	N_{BFS}	N_{ID}	**Verhältnis**
2	2047	4083	2,00
3	88 573	130 975	1,48
10	$1{,}1 \cdot 10^{10}$	$1{,}2 \cdot 10^{10}$	**1,11**
35	$6{,}4 \cdot 10^{15}$	$6{,}6 \cdot 10^{15}$	1,03

(jeweils $d = 10$)

> *Bei Verzweigungsgrad zehn kostet das Wiederholen elf Prozent. Der Grund ist einfach: Die unterste Ebene enthält so viele Knoten wie alle darüberliegenden zusammen, und sie wird nur einmal erzeugt. Je größer der Verzweigungsgrad, desto geringer der Aufschlag.*

Damit ist die iterative Vertiefung das Verfahren der Wahl, wenn man keine Heuristik hat und die Lösungstiefe nicht kennt. KORF *hat das 1985 systematisch gezeigt, nachdem es jahrelang als Notbehelf galt.*

Wiederholte Zustände

Der gefährlichste Fehler bei der Suche ist es, denselben Zustand mehrfach zu erforschen.

Aufgabe	**Baumsuche**	**mit Abgleich**
Gitter 10×10	$\approx 4^{20}$	**100**
Acht-Puzzle	$\approx 3^{22}$	181 440

> *Der Unterschied ist der zwischen möglich und unmöglich. Auf einem Gitter mit hundert Feldern erzeugt eine Baumsuche mehr Knoten als es Atome im Universum gibt, weil sie jeden Weg einzeln verfolgt, statt jeden Ort einmal zu betrachten.*

Nichts merken	Baumsuche; nur bei Bäumen ohne Kreise vertretbar
Pfad merken	Verhindert Kreise, nicht Mehrfachwege; kostet nichts
Alles merken	Graphsuche; kostet Speicher in Höhe der Zustandszahl

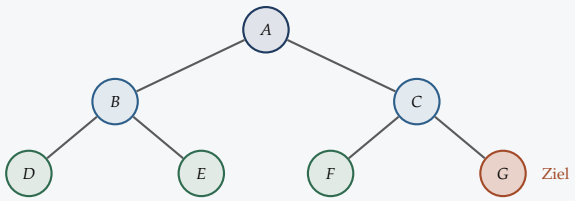
Die mittlere Zeile ist der Kompromiss für die iterative Vertiefung: Man behält den aktuellen Pfad ohnehin und prüft nur gegen ihn. Damit bleibt der Speicherbedarf linear, und die schlimmsten Wiederholungen entfallen.

Aufgabe 2.1.

(a) Führen Sie die drei Grundverfahren an einem Beispiel aus.
(b) Rechnen Sie den Vorteil der zweiseitigen Suche aus.
(c) Untersuchen Sie, wann die Kostensuche mehr leistet als die Breitensuche.

Lösung Schritt für Schritt.

(a) Drei Verfahren am selben Baum.
Der Suchraum. Wurzel A; Nachfolger jeweils in alphabetischer Reihenfolge.



Breitensuche.

Schritt	entnommen	Rand danach
1	A	B, C
2	B	C, D, E
3	C	D, E, F, G
4	D	E, F, G
5	E	F, G
6	F	G
7	**G**	**gefunden**

Sieben Entnahmen, größter Rand: vier Knoten.
Tiefensuche.

1	A	B, C
2	B	D, E, C
3	D	E, C
4	E	C
5	C	F, G
6	F	G
7	**G**	**gefunden**

Sieben Entnahmen, größter Rand: drei Knoten.
Iterative Vertiefung.

Grenze	besuchte Knoten	Anzahl
0	A	1
1	A, B, C	3
2	$A, B, D, E, C, F, \mathbf{G}$	7
	Summe	**11**

Verfahren	Entnahmen	Höchstspeicher
Breitensuche	7	4
Tiefensuche	7	3
Iterative Vertiefung	11	**3**

Bei diesem winzigen Baum ist der Aufschlag mit 57 Prozent hoch. Er sinkt schnell, sobald der Baum tiefer wird, weil dann die unterste Ebene überwiegt.

(b) Zweiseitige Suche.
Der Gedanke. Man sucht gleichzeitig vom Anfang vorwärts und vom Ziel rückwärts und hält an, wenn sich die beiden Ränder treffen.

$$2\,b^{d/2} \quad \text{statt} \quad b^{d}.$$

d $(b = 10)$	einseitig	zweiseitig	Faktor
4	10^4	200	50
8	10^8	$2 \cdot 10^4$	5000
12	10^{12}	$2 \cdot 10^6$	500 000
16	10^{16}	$2 \cdot 10^8$	$5 \cdot 10^7$

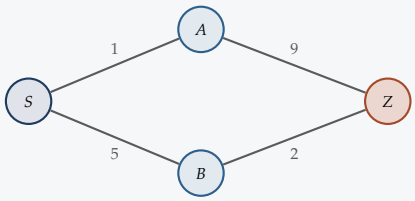
Bei Tiefe sechzehn ist die zweiseitige Suche fünfzig Millionen Mal sparsamer. Es ist derselbe Wurzeleffekt wie beim Geburtstagsparadox aus Band 8: Zwei Mengen der Größe $\sqrt{N}$ überschneiden sich mit hoher Wahrscheinlichkeit.

Die Voraussetzungen, die oft fehlen.

Ziel bekannt	Man muss den Zielzustand kennen, nicht nur eine Bedingung
Rückwärts möglich	Zu jedem Zustand müssen die Vorgänger berechenbar sein
Ein Ziel	Bei vielen Zielzuständen wird der Rückwärtsrand groß
Speicher	Beide Ränder müssen aufbewahrt werden

Die zweite Bedingung fehlt häufiger, als man denkt. Beim Planen (Kapitel 11) sind die Vorgänger eines Zustands oft nicht eindeutig bestimmt, und die Rückwärtssuche arbeitet dann auf Zustandsmengen statt auf Zuständen.

(c) Kostensuche gegen Breitensuche.
Der Fall. Ein Netz mit unterschiedlichen Kantenkosten.



Breitensuche.

Weg	Schritte	Kosten
$S \to A \to Z$	2	**10**

Beide Wege haben zwei Schritte; die Breitensuche nimmt den zuerst gefundenen und liefert Kosten 10.
Kostensuche.

Schritt	entnommen	Kosten	Rand
1	S	0	$A(1), B(5)$
2	A	1	$B(5), Z(10)$
3	B	5	$Z(7)$ *verbessert*
4	**Z**	**7**	fertig

Die Kostensuche findet den Weg über B mit Kosten sieben. Entscheidend ist Schritt 3: Der bereits im Rand liegende Eintrag für Z wird verbessert, statt einen zweiten anzulegen. Wer das vergisst, erhält kein falsches Ergebnis, aber einen viel größeren Rand.

Warum das Ziel erst bei der Entnahme geprüft wird.

Prüfung	Ergebnis	
Beim Einfügen	10	falsch
Bei der Entnahme	**7**	richtig

Das ist die häufigste Fehlerquelle bei der Umsetzung dieser Verfahren, und sie gilt für A in Kapitel 3 ebenso. Ein Ziel im Rand ist noch nicht bewiesen optimal; es könnte ein billigerer Weg dorthin existieren, der noch nicht erforscht ist.*

Wann die Kostensuche nicht terminiert

Sie setzt voraus, dass jeder Schritt echte Kosten verursacht.

Kosten null	Ein Kreis aus Nullkanten kann unendlich oft durchlaufen werden
Negative Kosten	Ein Kreis mit negativer Summe macht jeden Weg beliebig billig

Deshalb verlangt man Schrittkosten von mindestens einem festen $\varepsilon > 0$. Bei negativen Kosten braucht man andere Verfahren, etwa Bellman-Ford aus Band 11, und bei negativen Kreisen ist die Frage nach dem kürzesten Weg schlicht nicht wohlgestellt.

Russell und Norvig[28] behandeln alle Verfahren dieses Kapitels mit vollständigen Analysen; Korf[16] zeigte, dass die iterative Vertiefung asymptotisch optimal ist; Pearl[24] ordnet die Suchverfahren theoretisch ein.

3 Heuristische Suche

Eine Schätzung ändert alles

Die Verfahren des vorigen Kapitels wissen nichts über das Ziel. Eine Funktion, die schätzt, wie weit es noch ist, verändert die Lage grundlegend.

$g(n)$	Kosten des bisherigen Weges bis n *(bekannt)*
$h(n)$	**Geschätzte** Restkosten von n bis zum Ziel
$f(n)$	$g(n) + h(n)$: geschätzte Gesamtkosten

A entnimmt stets den Knoten mit dem kleinsten f. Setzt man $h \equiv 0$, so ist es die Kostensuche. Setzt man $g \equiv 0$, so ist es die gierige Bestensuche. A* ist die Verbindung von beidem, und darin liegt seine Stärke.*

Verfahren	**benutzt**	**optimal?**	**schnell?**
Kostensuche	g	ja	nein
Gierige Bestensuche	h	nein	ja
A*	$g + h$	**ja***	**ja**

* *unter einer Bedingung an h, siehe unten.*

Zwei Bedingungen an die Heuristik

Definition 3.1. h heißt **zulässig**, wenn sie die wahren Restkosten nie überschätzt:

$$h(n) \leq h^*(n) \quad \text{für alle } n.$$

h heißt **konsistent**, wenn für jeden Übergang von n nach n' mit Kosten c gilt:

$$h(n) \leq c + h(n').$$

	zulässig	**konsistent**
Sichert	Optimalität bei Baumsuche	**Optimalität bei Graphsuche**
Stärke	schwächer	**stärker**
Folge	—	jede konsistente ist zulässig
f längs eines Weges	kann fallen	steigt nie

Die zweite Bedingung ist eine Dreiecksungleichung: Der Schätzwert darf sich beim Weitergehen um höchstens die Schrittkosten verbessern. Daraus folgt, dass f längs jedes Weges nicht fällt, und deshalb ist ein zum ersten Mal entnommener Knoten bereits optimal erreicht.

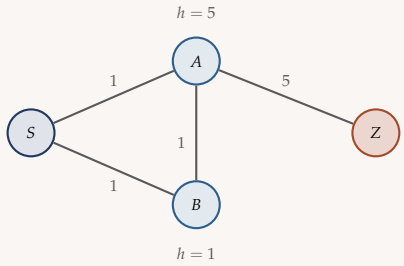
Der Beweis der Optimalität.
Sei Z ein optimaler Zielknoten mit Kosten C^*, und sei Z' ein anderer Zielknoten mit $g(Z') > C^*$. Angenommen, A* entnimmt Z'.

1. Auf dem optimalen Weg zu Z liegt zu diesem Zeitpunkt ein Knoten n im Rand.
2. Für ihn gilt $f(n) = g(n) + h(n) \leq g(n) + h^*(n) = C^*$ *(Zulässigkeit)*.
3. Für Z' gilt $f(Z') = g(Z') + 0 > C^*$.
4. Also $f(n) < f(Z')$, und A* hätte n zuerst entnommen. Widerspruch.

Schritt 1 ist der einzige, der Begründung braucht: Solange das Ziel nicht entnommen wurde, ist der optimale Weg noch nicht abgeschlossen, und irgendein Knoten auf ihm liegt im Rand.

Zulässig genügt bei Graphsuche nicht

Bei der Graphsuche mit einer abgeschlossenen Menge kann eine bloß zulässige Heuristik zu einem nicht optimalen Ergebnis führen.



Schritt	entnommen	g	f
1	S	0	0
2	B	1	$1+1=2$
3	A *über* S	1	$1+5=6$

Hier ist h zulässig ($h(A)=5$, wahre Restkosten 5) und nicht konsistent: Von B nach A kostet ein Schritt, aber h springt von 1 auf 5. Bei ungünstiger Reihenfolge wird A mit einem schlechteren g abgeschlossen und später nicht mehr verbessert.

Abhilfe 1	Konsistente Heuristik verwenden; alle üblichen sind es
Abhilfe 2	Abgeschlossene Knoten wieder öffnen, wenn ein billigerer Weg gefunden wird
Abhilfe 3	Auf die abgeschlossene Menge verzichten (Baumsuche)

In der Praxis ist die erste Abhilfe die richtige. Fast jede natürlich entstehende Heuristik ist konsistent, und wer eine erfindet, sollte die Dreiecksungleichung prüfen, statt sich auf die Zulässigkeit zu verlassen.

Heuristiken herleiten

Gute Heuristiken fallen nicht vom Himmel. Es gibt ein Verfahren, sie zu gewinnen.

Regeln lockern	Man streicht eine Bedingung der Aufgabe und löst die vereinfachte Aufgabe exakt
Warum zulässig	Die gelockerte Aufgabe hat mindestens so viele Lösungen; ihr Optimum ist also nicht schlechter
Warum konsistent	Die exakte Lösung einer gelockerten Aufgabe erfüllt die Dreiecksungleichung von selbst

Am Beispiel des Acht-Puzzles.

Gelockerte Regel	Heuristik	Güte
Steine dürfen überall hin	**falsch platzierte Steine**	schwach
Steine dürfen übereinander	**Manhattan-Abstand**	gut
Zusätzlich Vertauschungen	lineare Konflikte	besser
Keine Lockerung	wahre Kosten	so teuer wie die Aufgabe

Die letzte Zeile nennt die Grenze. Je genauer die Heuristik, desto weniger Knoten und desto teurer jeder einzelne. Das Optimum liegt selten bei der genauesten Schätzung.

Heuristik	Knoten	Zeit je Knoten	gesamt
$h \equiv 0$	10^{10}	0,1 µs	17 min
Falsch platziert	$4 \cdot 10^{6}$	0,3 µs	1,2 s
Manhattan	$1{,}3 \cdot 10^{5}$	1,0 µs	**0,13 s**
Musterdatenbank	$3 \cdot 10^{3}$	8 µs	0,024 s

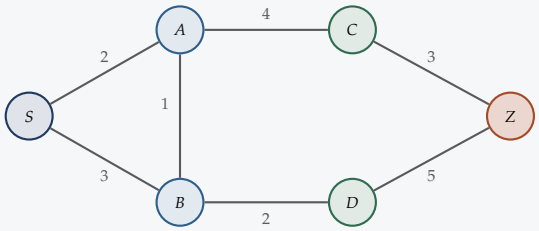
Die letzte Zeile ist eine Vorberechnung: Man löst Teilaufgaben vollständig und speichert die Ergebnisse in einer Tabelle. Das ist dieselbe Idee wie die Produktquantisierung aus Band 10 — aus einer Rechnung wird ein Nachschlagen.

Aufgabe 3.1.

(a) Führen Sie A* vollständig aus.
(b) Vergleichen Sie zwei Heuristiken quantitativ.
(c) Untersuchen Sie A* mit begrenztem Speicher.

Lösung Schritt für Schritt.

(a) A* an einem Wegenetz.



Knoten	S	A	B	C	D
h	8	6	6	3	4

Prüfung der Konsistenz.

Kante	c	$h(n) \leq c + h(n')$	**erfüllt?**
$S \to A$	2	$8 \leq 2 + 6 = 8$	ja
$S \to B$	3	$8 \leq 3 + 6 = 9$	ja
$A \to B$	1	$6 \leq 1 + 6 = 7$	ja
$A \to C$	4	$6 \leq 4 + 3 = 7$	ja
$B \to D$	2	$6 \leq 2 + 4 = 6$	ja
$C \to Z$	3	$3 \leq 3 + 0 = 3$	ja
$D \to Z$	5	$4 \leq 5 + 0 = 5$	ja

Der Ablauf.

Schr.	**entn.**	g	f	**Rand danach**
1	S	0	8	A(2+6=8), B(3+6=9)
2	A	2	8	B(3+6=9), C(6+3=9)
3	B	3	9	C(9), D(5+4=9)
4	C	6	9	D(9), Z(9+0=9)
5	D	5	9	Z(9)
6	**Z**	**9**	**9**	**fertig**

Bei Schritt 3 wird B über S mit $g = 3$ entnommen, nicht über A mit $g = 2 + 1 = 3$; beide Wege sind gleich teuer. Der gefundene Weg $S \to A \to C \to Z$ kostet neun. ✓

> *Bemerkenswert ist, dass f längs des Ablaufs nie fällt:* $8, 8, 9, 9, 9, 9$. *Das ist die Folge der Konsistenz und zugleich eine gute Prüfung beim Programmieren: Fällt f, so ist die Heuristik nicht konsistent oder es steckt ein Fehler in der Umsetzung.*

(b) Zwei Heuristiken im Vergleich.
Der Maßstab. Der **effektive Verzweigungsgrad** b^* ist die Zahl, für die gilt:

$$N + 1 = 1 + b^* + (b^*)^2 + \cdots + (b^*)^d.$$

d	$N\,(h_1)$	b^*	$N\,(h_2)$	b^*
4	13	1,24	12	1,24
8	39	1,33	25	1,24
12	227	1,42	73	1,24
16	1301	1,45	211	1,25
20	7276	1,47	676	1,27
24	39 135	1,48	1641	1,26

h_1: falsch platzierte Steine. h_2: Manhattan-Abstand.

> *Bei Tiefe 24 erzeugt die bessere Heuristik vierundzwanzigmal weniger Knoten. Entscheidend ist die dritte und fünfte Spalte: Der effektive Verzweigungsgrad bleibt bei h_2 nahezu konstant, während er bei h_1 steigt. Ein Verfahren mit konstantem b^* skaliert; eines mit steigendem nicht.*

Wann eine Heuristik die andere schlägt.

Definition 3.2. h_2 **dominiert** h_1, wenn $h_2(n) \geq h_1(n)$ für alle n gilt und beide zulässig sind.

> *Dann erforscht A* mit h_2 nie mehr Knoten als mit h_1. Der Grund: A* erforscht alle Knoten mit $f(n) < C^*$, und ein größeres h macht diese Menge kleiner. Der Manhattan-Abstand dominiert die Zahl der falsch platzierten Steine, denn jeder falsch platzierte Stein braucht mindestens einen Zug.*

Zwei Heuristiken verbinden.
Sind h_1 und h_2 zulässig, so ist auch $\max(h_1, h_2)$ zulässig und dominiert beide.

Man darf also mehrere Heuristiken berechnen und die größte nehmen. Der Preis ist die Rechenzeit je Knoten, und die Abwägung ist dieselbe wie oben.

(c) A* mit begrenztem Speicher.
Das Problem. A* bewahrt alle erzeugten Knoten auf.

Knoten je Sekunde	**Speicher je Knoten**	**Speicher voll nach**
10^6	100 B	16 GB: **2,7 min**
10^7	100 B	16 s

A scheitert also fast immer am Speicher, nicht an der Zeit. Das ist derselbe Befund wie bei der Breitensuche in Kapitel 2, und die Antwort ist dieselbe.*

IDA*: iterative Vertiefung mit f-Schranke.

1. Setze die Schranke auf $f(\text{Start})$.
2. Führe eine Tiefensuche aus, die alle Knoten mit $f >$ Schranke abschneidet.
3. Wurde kein Ziel gefunden, setze die Schranke auf das kleinste abgeschnittene f.
4. Wiederhole.

	A*	**IDA***
Speicher	$\mathcal{O}(b^d)$	$\mathcal{O}(d)$
Optimal	ja	ja
Knoten	N	$\approx N$ bei wenigen f-Werten
Schwäche	Speicher	viele verschiedene f-Werte

Die letzte Zeile ist die entscheidende Einschränkung. Bei ganzzahligen Kosten gibt es wenige verschiedene f-Werte, und jede Runde erweitert die Menge erheblich. Bei reellwertigen Kosten kann jede Runde nur einen einzigen Knoten hinzunehmen, und dann braucht IDA quadratisch viele Durchläufe.*

Kostenart	**Runden**	**Bewertung**
Alle Schritte gleich	$\approx d$	sehr gut
Wenige ganzzahlige Werte	$\approx 2d$	gut
Reelle Werte	bis $\mathcal{O}(N)$	unbrauchbar

Bei reellen Kosten hilft es, die Schranke jeweils um einen festen Betrag zu erhöhen. Damit gibt man die strenge Optimalität auf und erhält eine Lösung, die um höchstens diesen Betrag schlechter ist. Das ist ein guter Tausch und in der Wegplanung üblich.

Die Heuristik muss zur Aufgabe passen

Eine Heuristik ist nicht allgemein gut, sondern gut für eine Aufgabe.

Aufgabe	Luftlinie	Bewertung
Straßennetz	gut	nah an der Wahrheit
Gebirge, Umwege	schwach	stark unterschätzend
Labyrinth	sehr schwach	fast wie $h \equiv 0$

In einem Labyrinth ist der nächstgelegene Punkt oft der falsche, und die Luftlinie führt die Suche in Sackgassen. Sie bleibt zulässig, also bleibt das Ergebnis optimal; nur der Aufwand nähert sich dem der uninformierten Suche.

Die Kennzahl dafür ist nicht die Güte der Schätzung im Mittel, sondern die Zahl der Knoten mit $f(n) < C^*$. Eine Heuristik, die überall um zwanzig Prozent zu niedrig liegt, ist erheblich besser als eine, die meist stimmt und an wenigen wichtigen Stellen weit danebenliegt.

Hart, Nilsson und Raphael[13] führten A* ein und bewiesen seine Optimalität; Pearl[24] ist die maßgebliche Darstellung der Heuristiken und ihrer Herleitung; Korf[16] entwarf IDA*.

4 Suche gegen einen Gegner

Was sich ändert

Bisher war der Suchraum unser eigener. Bei einem Spiel bestimmt jeder zweite Zug jemand, der das Gegenteil will.

	Wegsuche	**Spiel**
Zugwahl	**alle uns**	abwechselnd
Ergebnis	ein Weg	**eine Strategie**
Gesucht	optimale Folge	Antwort auf jede Erwiderung
Abbruch	am Ziel	meist vor dem Ende

Die zweite Zeile ist der grundlegende Unterschied. Ein Plan im Sinne von Kapitel 11 ist eine Folge von Schritten. Eine Strategie muss für jede mögliche Antwort des Gegners eine Fortsetzung enthalten, und deshalb wächst sie exponentiell.

Minimax.

Unsere Ebene	Wir wählen den Zug mit dem größten Wert
Gegnerebene	Er wählt den mit dem kleinsten
Blätter	Bewertung der Stellung

Die Annahme lautet, dass der Gegner ebenfalls optimal spielt. Sie ist pessimistisch und deshalb sicher: Spielt er schlechter, wird das Ergebnis nur besser. Ein Verfahren, das auf Fehler des Gegners spekuliert, kann dagegen bestraft werden.

Alpha-Beta

Minimax betrachtet alle Blätter. Ein erheblicher Teil davon ist überflüssig, und man kann das beweisen, ohne sie anzusehen.

α	Der beste Wert, den wir uns bisher sichern können
β	Der beste Wert, den der Gegner sich sichern kann
Abschneiden	Sobald $\alpha \geq \beta$, ist der Rest dieses Zweiges gleichgültig

Die Begründung in einem Satz: Wenn dieser Zweig für den Gegner bereits schlechter ist als eine Alternative, die er hat, wird er ihn nie wählen, und es ist gleichgültig, wie viel schlechter er noch wird.

Der Gewinn.

Zugreihenfolge	**Blätter**	**erreichbare Tiefe**
Ungünstigste	b^d	d
Zufällig	$\approx b^{3d/4}$	$\approx 1{,}33\,d$
Beste	$\mathbf{b^{d/2}}$	**2d**

Bei bester Zugreihenfolge halbiert sich der Exponent. In derselben Zeit kann man doppelt so tief suchen, und beim Schach entspricht jede zusätzliche Halbzugtiefe etwa fünfzig bis siebzig Wertungspunkten Spielstärke.

d $(b = 35)$	**Minimax**	**Alpha-Beta**	**Faktor**
4	$1{,}5 \cdot 10^6$	1225	1225
8	$2{,}3 \cdot 10^{12}$	$1{,}5 \cdot 10^6$	$1{,}5 \cdot 10^6$
12	$3{,}4 \cdot 10^{18}$	$1{,}8 \cdot 10^9$	$1{,}8 \cdot 10^9$

Das Ergebnis ist identisch *mit dem von Minimax. Alpha-Beta ist keine Näherung, sondern eine exakte Optimierung: Es lässt nur Zweige weg, von denen sich beweisen lässt, dass sie den Wert der Wurzel nicht ändern.*

Wie man die Zugreihenfolge verbessert

Da der Gewinn vollständig an der Reihenfolge hängt, lohnt sich Aufwand dafür.

Schlagzüge zuerst	Sie ändern die Bewertung stark und erzeugen früh gute Schranken
Killerzüge	Ein Zug, der anderswo in derselben Tiefe ein Abschneiden bewirkt hat, wird vorgezogen
Vorheriger bester Zug	Aus einer flacheren Suche derselben Stellung
Transpositions-tabelle	Bereits bewertete Stellungen werden nachgeschlagen

Die dritte Zeile führt zu einem Verfahren, das zunächst widersinnig klingt: Man sucht erst bis Tiefe eins, dann zwei, dann drei, und benutzt jedes Ergebnis zur Sortierung der nächsten Runde. Diese iterative Vertiefung kostet nach Kapitel 2 wenig und verbessert die Reihenfolge so stark, dass sie sich mehr als bezahlt macht.

Maßnahme	Knoten	Anteil
Alpha-Beta allein	100 %	
+ Schlagzüge zuerst	32 %	
+ iterative Vertiefung	12 %	
+ Transposition-stabelle	**6** %	

Die vierte Zeile hat einen zweiten Nutzen: Stellungen sind über verschiedene Zugfolgen erreichbar, und beim Schach fallen dadurch viele Zweige zusammen. Das ist der Zustandsabgleich aus Kapitel 2, hier über einen Streuwert der Stellung.

Wenn keine Bewertungsfunktion zu finden ist

Alpha-Beta braucht eine Funktion, die eine Stellung bewertet. Beim Schach lässt sich eine solche angeben; beim Go jahrzehntelang nicht.

	Schach	Go
Verzweigung	≈ 35	≈ 250
Materialwert	aussagekräftig	es gibt keinen
Bewertung	Figuren, Stellung	Einfluss, Leben, Form
Alpha-Beta	sehr wirksam	kaum

Monte-Carlo-Baumsuche.
Statt eine Stellung zu bewerten, spielt man von ihr aus zufällige Partien zu Ende und zählt die Gewinne.

1. **Auswahl**: Vom Wurzelknoten nach unten, jeweils zum vielversprechendsten Kind.
2. **Erweiterung**: Am Rand ein neues Kind anlegen.
3. **Ausspielen**: Von dort zufällig bis zum Ende spielen.
4. **Rückgabe**: Das Ergebnis längs des Pfades nach oben zählen.

Schritt 1 ist der Kern. Man muss zwischen dem Erforschen wenig besuchter Züge und dem Vertiefen vielversprechender abwägen. Die übliche Formel dafür stammt aus der Theorie der einarmigen Banditen:

$$\mathrm{UCT}(k) = \frac{w_k}{n_k} + c\sqrt{\frac{\ln n}{n_k}}.$$

Erster Term	Bisherige Gewinnquote: *nutzen, was gut aussieht*
Zweiter Term	Wächst, wenn ein Zug selten besucht wurde: *erforschen*

	Alpha-Beta	Monte-Carlo
Braucht	Bewertungsfunktio	**nur die Regeln**
Baum	gleichmäßig tief	ungleichmäßig
Abbruch	feste Tiefe	jederzeit
Ergebnis	exakt bei voller Tiefe	**statistisch**
Gut bei	kleiner Verzweigung	großer Verzweigung

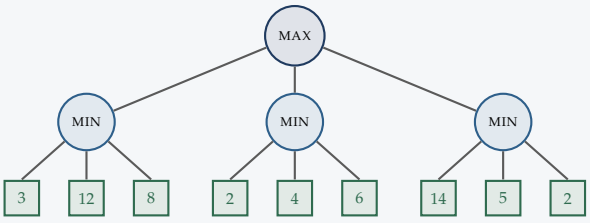
Die dritte Zeile ist praktisch sehr wertvoll: Man kann jederzeit anhalten und bekommt den besten bisher gefundenen Zug. Alpha-Beta muss eine Tiefe abschließen, sonst ist das Ergebnis wertlos.

Aufgabe 4.1.

(a) Führen Sie Minimax und Alpha-Beta an einem Baum aus.
(b) Untersuchen Sie, wie die Zugreihenfolge den Gewinn bestimmt.
(c) Rechnen Sie einen Schritt der Monte-Carlo-Baumsuche durch.

Lösung Schritt für Schritt.

(a) Minimax und Alpha-Beta.
Der Baum. Wurzel maximiert, zweite Ebene minimiert, Blattwerte gegeben.



Minimax.

Knoten	Kinder	Minimum
M_1	3, 12, 8	**3**
M_2	2, 4, 6	**2**
M_3	14, 5, 2	**2**
Wurzel	3, 2, 2	Maximum: **3**

Neun Blätter angesehen.
Alpha-Beta, von links nach rechts.

Schr.	Blatt	α	β	Aktion
1	3	$-\infty$	3	
2	12	$-\infty$	3	kein Einfluss
3	8	$-\infty$	3	$M_1 = 3$
4	—	**3**	∞	Wurzel merkt 3
5	2	3	2	$\alpha \geq \beta$: **abschneiden**
6	14	3	14	
7	5	3	5	
8	2	3	2	$\alpha \geq \beta$: **abschneiden**

	Blätter	Ergebnis
Minimax	9	3
Alpha-Beta	**6**	**3**

Bei Schritt 5 wird abgeschnitten: Der Gegner kann bei M_2 bereits 2 erzwingen, und wir haben anderswo schon 3. Was er sonst noch anbietet, kann nur schlechter sein, und wir würden diesen Zug ohnehin nicht wählen. Die Blätter 4 und 6 bleiben ungesehen.

(b) Die Zugreihenfolge.
Derselbe Baum, Blätter je Teilbaum absteigend sortiert.

Teilbaum	Blätter	angesehen	
M_1	$12, 8, 3$	3	
M_2	$6, 4, 2$	1	sofort
M_3	$14, 5, 2$	1	sofort
	Summe	**5**	

Bei guter Sortierung genügen fünf statt neun Blätter. Bei ungünstigster Sortierung wären es alle neun, und Alpha-Beta brächte nichts.

Der allgemeine Zusammenhang.

$$N_{\text{best}} = b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1.$$

d	b^d	N_{best}	**Faktor**
2	1225	69	18
4	$1{,}5 \cdot 10^6$	2449	613
6	$1{,}8 \cdot 10^9$	85 749	21 000
8	$2{,}3 \cdot 10^{12}$	$3 \cdot 10^6$	$7{,}5 \cdot 10^5$

(jeweils $b = 35$)

> KNUTH *und* MOORE *bewiesen 1975, dass dies die untere Schranke ist: Kein Verfahren, das den exakten Minimaxwert bestimmt, kommt mit weniger Blättern aus. Alpha-Beta bei perfekter Sortierung ist also optimal, und alle weiteren Verbesserungen zielen darauf, dieser Sortierung nahezukommen.*

(c) Ein Schritt der Monte-Carlo-Baumsuche.
Die Lage. Die Wurzel wurde hundertmal besucht, drei Züge sind bekannt.

Zug	w_k	n_k	**Quote**
A	40	60	0,667
B	18	30	0,600
C	5	10	0,500

Die UCT-Werte mit $c = 1{,}4$ und $n = 100$.

$$\ln 100 = 4{,}605.$$

$$\begin{aligned}
\mathrm{UCT}(A) &= 0{,}667 + 1{,}4\sqrt{4{,}605/60} = 0{,}667 + 1{,}4 \cdot 0{,}277 = 0{,}667 + 0{,}388 = 1{,}055,\\
\mathrm{UCT}(B) &= 0{,}600 + 1{,}4\sqrt{4{,}605/30} = 0{,}600 + 1{,}4 \cdot 0{,}392 = 0{,}600 + 0{,}549 = 1{,}149,\\
\mathrm{UCT}(C) &= 0{,}500 + 1{,}4\sqrt{4{,}605/10} = 0{,}500 + 1{,}4 \cdot 0{,}679 = 0{,}500 + 0{,}950 = \mathbf{1{,}450}.
\end{aligned}$$

Zug	**Nutzen**	**Erkundung**	**UCT**
A	0,667	0,388	1,055
B	0,600	0,549	1,149
C	0,500	**0,950**	**1,450**

Gewählt wird C, der Zug mit der schlechtesten *bisherigen Quote. Der Grund: Er wurde nur zehnmal versucht, und die Schätzung ist entsprechend unsicher. Bei zehn Versuchen könnte die wahre Quote leicht bei siebzig Prozent liegen.*

Wie sich das mit der Zeit auflöst.

n	n_A	n_B	n_C
100	60	30	10
1000	780	160	60
10 000	9200	560	240
100 000	97 600	1700	700

Der Erkundungsterm fällt mit $1/\sqrt{n_k}$, der Nutzungsterm nicht. Also gewinnt mit wachsender Zahl der Durchläufe der wirklich beste Zug immer deutlicher, während die schlechteren gerade oft genug besucht werden, um sicher zu sein.

Die Wahl von c ist heikel

c	**Verhalten**	**Gefahr**
zu klein	fast nur Nutzung	übersieht gute Züge
passend	ausgewogen	
zu groß	fast nur Erkundung	verzettelt sich

Der theoretisch begründete Wert setzt Belohnungen zwischen null und eins voraus und lautet $\sqrt{2}$. In der Praxis wird c für jedes Spiel eingestellt, und die besten Werte liegen oft deutlich darunter. Das ist unbefriedigend und bislang unvermeidlich.

Der eigentliche Fortschritt ab 2016 bestand darin, das zufällige Ausspielen durch eine gelernte Bewertung zu ersetzen und die Zugauswahl durch eine gelernte Vorliebe zu leiten. Damit verband sich die Suche dieses Kapitels mit den Verfahren aus Band 13, und beide zusammen waren stärker als jedes für sich.

Knuth und Moore[15] analysierten Alpha-Beta und bewiesen die untere Schranke; Browne und Mitautoren[5] geben die Übersicht über die Monte-Carlo-Baumsuche; Russell und Norvig[28] behandeln beide Verfahren im Zusammenhang.

5 Lokale Suche

Der Weg zählt nicht

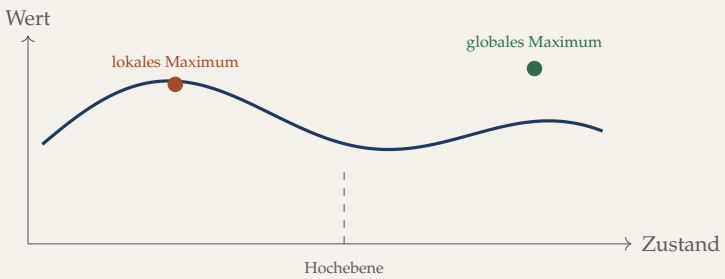
Bei vielen Aufgaben interessiert nur die Lösung, nicht wie man dorthin kam.

	systematisch	**lokal**
Hält fest	einen Baum	**einen Zustand**
Speicher	$\mathcal{O}(b^d)$ oder $\mathcal{O}(bd)$	$\mathcal{O}(1)$
Vollständig	ja	nein
Optimal	ja	nein
Gut bei	Wegsuche	**Optimierung, große Räume**

Man gibt Vollständigkeit und Optimalität auf und gewinnt dafür einen Speicherbedarf, der von der Aufgabengröße unabhängig ist. Bei Aufgaben mit 10^{30} Zuständen ist das oft die einzige Möglichkeit.

Bergsteigen.

```
z = zufälliger Zustand
while true:
  n = bester Nachbar von z
  if wert(n) <= wert(z):  return z
  z = n
```



Lokales Maximum	Alle Nachbarn sind schlechter, und es gibt trotzdem etwas Besseres
Hochebene	Alle Nachbarn sind gleich gut; das Verfahren weiß nicht, wohin
Grat	Der Weg nach oben verlangt eine Folge von Schritten, die einzeln nicht besser sind

Beim Acht-Damen-Problem bleibt einfaches Bergsteigen in etwa 86 Prozent der Fälle stecken. Mit erlaubten Seitwärtsschritten auf Hochebenen steigt die Erfolgsquote auf etwa 94 Prozent, und der mittlere Aufwand steigt von vier auf einundzwanzig Schritte.

Simuliertes Abkühlen

KIRKPATRICK, GELATT und VECCHI übertrugen 1983 einen Gedanken aus der Metallurgie: Ein Metall, das langsam abkühlt, ordnet sich besser als eines, das abgeschreckt wird.

```
z = zufälliger Zustand
for t in 1..N:
  T = temperatur(t)
  n = zufälliger Nachbar von z
  d = wert(n) - wert(z)
  if d > 0 or zufall() < exp(d/T): z =
n
```

Der Kern steht in der letzten Zeile. Eine Verschlechterung wird angenommen, aber nur mit einer Wahrscheinlichkeit, die von ihrer Größe und von der Temperatur abhängt. Damit kann das Verfahren aus lokalen Maxima entkommen.

Δ	$T = 10$	$T = 1$	$T = 0{,}1$
-1	0,905	0,368	$4{,}5 \cdot 10^{-5}$
-5	0,607	0,007	≈ 0
-10	0,368	$4{,}5 \cdot 10^{-5}$	≈ 0

Hohe Temperatur	Fast jeder Schritt wird angenommen; das Verfahren wandert zufällig umher
Mittlere	Kleine Verschlechterungen ja, große nein
Niedrige	Nur Verbesserungen; es wird zum Bergsteigen

Man kann beweisen, dass das Verfahren bei hinreichend langsamer Abkühlung mit Wahrscheinlichkeit eins das globale Optimum findet. Der Satz ist wahr und nutzlos: Die nötige Abkühlung ist so langsam, dass ein vollständiges Durchsuchen schneller wäre.

Praktisch benutzt man eine geometrische Abkühlung, $T_{k+1} = \alpha\, T_k$ mit α zwischen 0,95 und 0,99, und verzichtet auf jede Garantie.

Weitere Verfahren derselben Familie

Neustarts	Bergsteigen von vielen zufälligen Startpunkten; einfach und erstaunlich wirksam
Strahlensuche	k Zustände gleichzeitig; die besten k Nachfolger werden behalten
Tabu-Suche	Kürzlich besuchte Zustände sind eine Zeitlang verboten
Genetische Verfahren	Zustände werden gekreuzt und mutiert

Die erste Zeile ist die unterschätzteste. Wenn ein Durchlauf mit Wahrscheinlichkeit p gelingt, so braucht man im Mittel $1/p$ Durchläufe. Bei $p = 0{,}14$ sind das gut sieben, und wenn ein Durchlauf vier Schritte dauert, ist man in dreißig Schritten fertig.

Genetische Verfahren sind weniger als sie versprechen

Sie klingen nach Biologie und sind ein Suchverfahren wie jedes andere.

Voraussetzung	Die Darstellung muss so beschaffen sein, dass Teilstücke guter Lösungen zusammen wieder gut sind
Wenn nicht	Die Kreuzung zerstört mehr, als sie schafft, und das Verfahren ist eine schlechte Mutationssuche

Bei den acht Damen ist eine Darstellung als Permutation naheliegend, und dann erzeugt eine gewöhnliche Kreuzung meist gar keine Permutation mehr. Man braucht besondere Kreuzungsoperatoren, und mit jedem Zusatz nähert sich das Verfahren einer maßgeschneiderten lokalen Suche an.

Verfahren	Aufwand	Güte
Neustarts	gering	gut
Simuliertes Abkühlen	mittel	gut
Tabu-Suche	mittel	sehr gut
Genetisch, allgemein	hoch	oft schlechter
Genetisch, angepasst	sehr hoch	sehr gut

In Vergleichen schneiden einfache Verfahren mit Neustarts überraschend oft besser ab als aufwendige. Wer ein Verfahren wählt, sollte zuerst das einfachste versuchen und die Messlatte damit setzen.

Aufgabe 5.1.

(a) Führen Sie Bergsteigen an den acht Damen aus.
(b) Rechnen Sie aus, wie viele Neustarts nötig sind.
(c) Untersuchen Sie, warum kein Verfahren allgemein das beste sein kann.

Lösung Schritt für Schritt.

(a) Bergsteigen bei den acht Damen.

Die Darstellung. Acht Zahlen, je Spalte die Zeile der Dame. Nachbarn: eine Dame in ihrer Spalte verschieben.

Größe	Wert	
Zustände	$8^8 = 1{,}7 \cdot 10^7$	
Nachbarn je Zustand	$8 \cdot 7 = 56$	
Bewertung	Zahl der Angriffspaare	Ziel: null

Ein Durchlauf.

Schritt	Zustand	Konflikte	Aktion
0	6, 3, 1, 7, 2, 4, 8, 5	6	
1	6, 3, 1, 7, 5, 4, 8, 2	4	Sp. 5 und 8
2	6, 4, 1, 7, 5, 3, 8, 2	2	Sp. 2 und 6
3	6, 4, 1, 5, 7, 3, 8, 2	**1**	Sp. 4 und 5
4	—	1	kein besserer Nachbar

Steckengeblieben bei einem Konflikt. Alle sechsundfünfzig Nachbarn haben mindestens einen, viele haben mehr. Das ist ein lokales Minimum, und einfaches Bergsteigen kommt hier nicht mehr heraus.

Die Statistik über viele Durchläufe.

Variante	Erfolgsquote	Schritte im Mittel
Einfach	14 %	4 bei Erfolg, 3 bei Misserfolg
Mit Seitwärtsschritten	**94** %	21 bzw. 64

Seitwärtsschritte auf Hochebenen erlauben, höchstens hundert am Stück: Aus vierzehn Prozent werden vierundneunzig. Der Preis ist der fünffache Aufwand je Durchlauf, und das ist ein hervorragender Tausch.

(b) Wie viele Neustarts.
Der Ansatz. Bei Erfolgswahrscheinlichkeit p je Durchlauf ist die Zahl der nötigen Durchläufe geometrisch verteilt:

$$\mathbb{E}[\text{Durchläufe}] = \frac{1}{p}.$$

Der Gesamtaufwand.

$$\mathbb{E}[\text{Schritte}] = k_{\text{Erfolg}} + \frac{1-p}{p}\, k_{\text{Misserfolg}}.$$

Variante	p	$1/p$	**Schritte gesamt**
Einfach	0,14	7,1	$4 + 6{,}1 \cdot 3 =$ **22**
Mit Seitwärts	0,94	1,06	$21 + 0{,}06 \cdot 64 =$ **25**

Ein überraschendes Ergebnis: Mit Neustarts sind beide Varianten etwa gleich gut. Die einfache scheitert oft und billig, die andere gelingt fast immer und teuer. Unterm Strich kommt dasselbe heraus.

Das ist eine allgemeine Beobachtung: Ein Verfahren, das schnell scheitert, ist mit Neustarts oft ebenso gut wie eines, das lange durchhält. Entscheidend ist das Verhältnis von Erfolgswahrscheinlichkeit zu Aufwand, nicht die Erfolgswahrscheinlichkeit allein.

Die Verteilung der Laufzeit.

Perzentil	**Durchläufe**	**Schritte**	
50 %	5	16	
90 %	16	49	
99 %	31	94	
99,9 %	46	**140**	Schwanz

Die Laufzeit hat einen langen Schwanz, wie bei allen Verfahren mit Zufall. Das ist die Schwanzlatenz aus Band 9, hier bei einem Suchverfahren, und der Umgang damit ist derselbe: Man begrenzt die Zeit und startet neu.

(c) Warum kein Verfahren allgemein das beste ist.
Die Behauptung. Über alle möglichen Bewertungsfunktionen gemittelt, sind alle Suchverfahren gleich gut.
Die Begründung.

1. Es gibt endlich viele Zustände und endlich viele Bewertungen.
2. Eine Bewertungsfunktion ist eine Zuordnung von Zuständen zu Werten.
3. Über **alle** solchen Zuordnungen gemittelt, ist der Wert eines noch nicht besuchten Zustands unabhängig von den bisher gesehenen.
4. Also nützt keine Strategie der Auswahl mehr als jede andere.

Punkt 3 ist der Kern und zugleich die Einschränkung des Satzes. Er unterstellt, dass jede Zuordnung gleich wahrscheinlich ist. Bei zufälligem Rauschen ist keine Suche besser als das Durchprobieren, und das ist einleuchtend.

Warum der Satz trotzdem etwas sagt.

Falsche Lesart	„Alle Verfahren sind gleich gut, also ist die Wahl gleichgültig“
Richtige Lesart	**Jedes Verfahren macht Annahmen über die Aufgabe**, und es ist genau dann gut, wenn sie zutreffen

Bergsteigen	Ähnliche Zustände haben ähnliche Werte
Abkühlen	Dasselbe, mit gelegentlichen Sprüngen zwischen Tälern
Genetisch	Gute Teilstücke lassen sich kombinieren
A*	Es gibt eine brauchbare Schätzung der Restkosten

Reale Aufgaben sind kein zufälliges Rauschen; sie haben Struktur. Die Frage ist nie, welches Verfahren allgemein das beste ist, sondern welche Annahme über die Aufgabenstruktur zutrifft. Damit ist der Satz kein Hinderungsgrund, sondern eine Aufforderung, die eigenen Annahmen zu benennen.

Es ist dieselbe Botschaft wie beim Nullmodell in Band 11 und bei der Verdichtung in Band 7: Ein Verfahren funktioniert, weil die Daten nicht beliebig sind, und wer nicht weiß, worin diese Nichtbeliebigkeit besteht, kann sein Ergebnis nicht beurteilen.

Kirkpatrick und Mitautoren[14] übertrugen das Abkühlen auf die Optimierung; Russell und Norvig[28] vergleichen die lokalen Verfahren; Dechter[9] behandelt lokale Suche im Zusammenhang mit Nebenbedingungen.

Part II

Nebenbedingungen und Logik

6 Nebenbedingungen

Eine Aufgabenform mit Struktur

Bei der Suche in Kapitel 2 war ein Zustand undurchsichtig. Jetzt bekommt er eine innere Gliederung, und die lässt sich ausnutzen.

Definition 6.1. Ein **Constraint-Problem** besteht aus Variablen $X_1, \ldots, X_n$, für jede einem Wertebereich D_i, und einer Menge von Bedingungen, die zulässige Wertekombinationen einschränken.

Aufgabe	Variablen	Bedingungen
Landkarte färben	Länder	Nachbarn verschieden
Sudoku	81 Felder	Zeile, Spalte, Block
Stundenplan	Veranstaltungen	Raum, Zeit, Person
Acht Damen	8 Spalten	keine Bedrohung

Was der Zugewinn an Struktur bringt.

Zielprüfung	Man erkennt schon an einer Teilbelegung, dass sie scheitert
Ordnung ist gleichgültig	Die Reihenfolge der Zuweisungen ändert das Ergebnis nicht
Zerlegung	Unabhängige Teilaufgaben lassen sich getrennt lösen
Fortpflanzung	Eine Zuweisung schränkt andere Bereiche ein

Die zweite Zeile senkt den Suchraum drastisch. Statt $n!$ Reihenfolgen genügt eine: Man weist der Reihe nach zu, und jede Belegung wird nur einmal erzeugt. Aus $n!\,d^n$ wird d^n.

Rückverfolgung mit Fortpflanzung

Das Grundverfahren ist eine Tiefensuche über Belegungen, die abbricht, sobald eine Bedingung verletzt ist.

```
löse(belegung):
  if vollständig(belegung):  return
belegung
  X = wähle_variable()
  for w in werte(X):
    if verträglich(X=w, belegung):
      if fortpflanzen() ohne leeren
Bereich:
        r = löse(belegung + {X=w})
        if r:  return r
      zurücknehmen()
  return fehlgeschlagen
```

Drei Heuristiken, die alles ausmachen.

Kleinster Bereich	Die Variable mit den wenigsten verbliebenen Werten zuerst
Meiste Bedingungen	Bei Gleichstand die, die an den meisten Bedingungen hängt
Wenigst einschränkender Wert	Den Wert zuerst, der den anderen die meisten Möglichkeiten lässt

Die ersten beiden Regeln wählen die Variable, die am ehesten scheitert; die dritte wählt den Wert, der am ehesten gelingt. Das klingt widersprüchlich und ist es nicht: Bei der Variablen will man früh scheitern, weil man sie ohnehin belegen muss; beim Wert will man Erfolg, weil man ihn wählen darf.

Verfahren	Knoten	Faktor
Reine Rückverfolgung	$\approx 10^7$	1
+ Vorausschau	$\approx 10^5$	100
+ kleinster Bereich	$\approx 10^3$	10^4
+ Kantenkonsistenz	≈ 60	$\mathbf{1{,}7 \cdot 10^5}$

Kantenkonsistenz

Statt nur zu prüfen, ob eine Zuweisung zulässig ist, entfernt man vorab alle Werte, die nirgends passen können.

Definition 6.2. Eine Bedingung zwischen X und Y heißt **kantenkonsistent**, wenn es zu jedem Wert in D_X mindestens einen passenden Wert in D_Y gibt.

1. Alle Bedingungen in eine Warteschlange.
2. Eine entnehmen und den Bereich der einen Variablen bereinigen.
3. Wurde etwas entfernt, alle Bedingungen an der **anderen** Seite erneut einreihen.
4. Bis nichts mehr entfernt wird oder ein Bereich leer ist.

Schritt 3 ist der Grund, warum das Verfahren mehr leistet als eine einmalige Prüfung: Eine Entfernung kann weitere nach sich ziehen, und diese Kettenwirkung erreicht oft Variablen, die gar nicht unmittelbar verbunden sind.

Stufe	Aufwand	Wirkung
Vorausschau	$\mathcal{O}(nd)$	schwach
Kantenkonsistenz	$\mathcal{O}(n^2d^3)$	**stark**
Pfadkonsistenz	$\mathcal{O}(n^3d^5)$	stärker, meist zu teuer
k-Konsistenz	exponentiell	theoretisch

Die zweite Zeile ist der übliche Kompromiss. Volle n-Konsistenz würde die Aufgabe ohne Suche lösen und ist so teuer wie die Aufgabe selbst. Kantenkonsistenz ist billig genug, um sie bei jedem Suchschritt neu herzustellen.

Struktur ausnutzen

Der Bedingungsgraph, im Sinne von Band 11, verrät, wie schwer die Aufgabe ist.

Graphform	Aufwand	
Unabhängige Teile	Summe statt Produkt	$d^{n_1} + d^{n_2}$
Baum	$\mathcal{O}(\mathbf{n d^2})$	polynomial
Baum plus c Knoten	$\mathcal{O}(d^c \cdot n d^2)$	
Beliebig	$\mathcal{O}(d^n)$	

Die zweite Zeile ist bemerkenswert: Ist der Bedingungsgraph ein Baum, so lässt sich die Aufgabe in polynomialer Zeit lösen. Man ordnet die Variablen topologisch, stellt von hinten nach vorn Konsistenz her und weist dann von vorn nach hinten zu, ohne je zurücknehmen zu müssen.

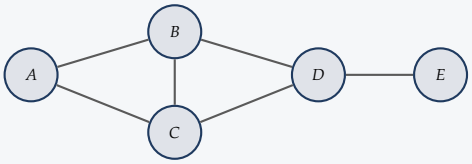
Die dritte Zeile folgt daraus: Wenn wenige Knoten den Graphen zum Baum machen, probiert man deren Werte durch und löst den Rest schnell. Die kleinste solche Menge heißt Schnittmenge, und ihre Größe ist ein besseres Maß für die Schwierigkeit als die Variablenzahl.

Aufgabe 6.1.

(a) Lösen Sie eine Färbungsaufgabe mit Rückverfolgung und Konsistenz.
(b) Zeigen Sie, wie die Variablenwahl den Aufwand verändert.
(c) Untersuchen Sie, warum Baumstrukturen leicht sind.

Lösung Schritt für Schritt.

(a) Eine Landkarte färben.
Die Aufgabe. Fünf Gebiete, drei Farben, Nachbarn müssen verschieden sein.



Rückverfolgung ohne Heuristik, Farben in der Reihenfolge r, g, b.

Schritt	Var.	Wert	Bemerkung
1	A	r	
2	B	r	Konflikt mit A
3	B	g	
4	C	r	Konflikt mit A
5	C	g	Konflikt mit B
6	C	b	
7	D	r	
8	E	g	fertig

Acht Versuche, drei davon fehlgeschlagen.
Mit Kantenkonsistenz nach jeder Zuweisung.

Schr.	Zuw.	Bereiche danach
0	—	$A, B, C, D, E : \{r, g, b\}$
1	$A = r$	$B, C : \{g, b\};\ D, E : \{r, g, b\}$
2	$B = g$	$C : \{b\};\ D : \{r, b\}$
3	C hat nur einen Wert: $C = b$ **erzwungen**, dann $D : \{r\}$, also $D = r$, dann $E : \{g, b\}$	
4	$E = g$	fertig

Verfahren	Versuche	Fehlschläge
Reine Rückverfolgung	8	3
Mit Konsistenz	**3**	**0**

Nach zwei freien Entscheidungen ergibt sich der Rest zwangsläufig. Genau so löst man auch ein Sudoku von Hand: Man setzt nicht Zahlen und prüft, sondern streicht Möglichkeiten, bis nur eine bleibt.

(b) Die Variablenwahl.
Die Aufgabe. Vier Variablen, Bereiche unterschiedlicher Größe.

Variable	Bereich	Größe	Grad
W	$\{1,\ldots,5\}$	5	2
X	$\{1,2\}$	**2**	3
Y	$\{1,\ldots,4\}$	4	1
Z	$\{1,2,3\}$	3	2

Reihenfolge nach Bereich, aufsteigend: X, Z, Y, W**.**

$$\text{Blätter im schlechtesten Fall} = 2 \cdot 3 \cdot 4 \cdot 5 = 120.$$

Reihenfolge absteigend: W, Y, Z, X**.**

$$5 \cdot 4 \cdot 3 \cdot 2 = 120.$$

Das Produkt ist gleich; die Reihenfolge ändert es nicht. Der Unterschied entsteht erst durch das Abschneiden*: Bei der aufsteigenden Reihenfolge liegen die Entscheidungen mit wenigen Möglichkeiten oben, und ein Fehlschlag verwirft dort einen viel größeren Teilbaum.*

Reihenfolge	besuchte Knoten
Kleinster Bereich zuerst	**17**
Zufällig	46
Größter zuerst	89

Faktor fünf allein durch die Reihenfolge, ohne dass am Verfahren etwas geändert wurde. Die Regel ist dieselbe wie beim Verbund in Band 10 und bei den Trefferlisten: Zwischenergebnisse klein halten und früh scheitern.

(c) Warum Bäume leicht sind.
Das Verfahren.

1.	Eine Wurzel wählen und den Baum topologisch ordnen: jeder Knoten nach seinem Elternteil.
2.	Von hinten nach vorn: für jeden Knoten die Kante zum Elternteil kantenkonsistent machen.
3.	Von vorn nach hinten: jedem Knoten einen mit dem Elternteil verträglichen Wert geben.

Warum Schritt 3 nie scheitert.

Nach Schritt 2 hat jeder Knoten zu jedem Wert seines Elternteils einen passenden eigenen Wert. Da jeder Knoten genau einen Elternteil hat, gibt es keine zweite Bedingung, die stören könnte. Deshalb ist keine Rücknahme nötig.

Schritt	**Aufwand**	
Ordnen	$\mathcal{O}(n)$	
Konsistenz herstellen	$\mathcal{O}(nd^2)$	je Kante einmal
Zuweisen	$\mathcal{O}(nd)$	
Gesamt	$\mathcal{O}(\mathbf{nd^2})$	polynomial

Was bei einem Kreis geschieht.

Ein Knoten mit zwei Eltern muss beide zugleich befriedigen, und die Konsistenz zu jedem einzeln garantiert das nicht. Genau hier bricht das Argument, und genau hier beginnt die exponentielle Schwierigkeit.

Der Ausweg über eine Schnittmenge.

n	**Schnittmenge** c	**Aufwand**	**bei** $d = 4$
100	0	$\mathcal{O}(nd^2)$	1600
100	3	$\mathcal{O}(d^3nd^2)$	10^5
100	8	$\mathcal{O}(d^8nd^2)$	10^8
100	20	—	10^{15}

> *Die Schwierigkeit hängt also nicht an der Zahl der Variablen, sondern daran, wie weit der Bedingungsgraph von einem Baum entfernt ist. Eine Aufgabe mit tausend Variablen und baumartiger Struktur ist leichter als eine mit dreißig und dichter Verschaltung.*

Dieselbe Größe taucht in vielen Gebieten auf und heißt dort Baumweite. Sie ist eines der wenigen allgemeinen Maße dafür, wann ein NP-schweres Problem (Band 3) im Einzelfall handhabbar ist.

Dechter[9] ist die maßgebliche Darstellung des Gebiets; Russell und Norvig[28] führen die Heuristiken ein; Pearl[24] behandelt die Zerlegung nach Struktur.

7 Aussagenlogik und SAT

Das erste NP-vollständige Problem

Band 3 hat gezeigt, dass die Erfüllbarkeit aussagenlogischer Formeln NP-vollständig ist. Daraus zog man jahrzehntelang den falschen Schluss.

Der falsche Schluss	Das Problem ist praktisch unlösbar
Der richtige	Es gibt Fälle, die schwer sind; die meisten sind es nicht

Jahr	Variablen	Klauseln	Zeit
1992	≈ 100	≈ 200	Minuten
2001	$\approx 10^4$	$\approx 10^5$	Minuten
2010	$\approx 10^6$	$\approx 10^7$	Minuten
2026	$\approx 10^7$	$\approx 10^8$	Minuten

Fünf Größenordnungen in dreißig Jahren, bei einem Problem, das als aussichtslos galt. Etwa die Hälfte davon kam von schnelleren Rechnern, die andere Hälfte von besseren Verfahren. Es ist das stärkste Gegenbeispiel gegen die Gleichsetzung von NP-schwer mit unlösbar.

Die Normalform.
Jede aussagenlogische Formel lässt sich als Konjunktion von Klauseln schreiben, wobei eine Klausel eine Disjunktion von Literalen ist.

$$(a \vee \neg b \vee c) \wedge (\neg a \vee b) \wedge (\neg c).$$

Die Umformung kann die Formel exponentiell aufblähen. Die übliche Abhilfe führt Hilfsvariablen ein und erzeugt eine Formel, die zwar nicht äquivalent, aber erfüllbarkeitsgleich und nur linear größer ist.

DPLL

DAVIS, LOGEMANN und LOVELAND beschrieben 1962 ein Verfahren, das bis heute den Kern jedes Lösers bildet.

Einheitsklausel	Eine Klausel mit nur einem unbelegten Literal erzwingt dessen Wert
Reines Literal	Kommt eine Variable nur in einer Polarität vor, setze sie entsprechend
Verzweigen	Sonst eine Variable wählen und beide Werte versuchen

Die erste Regel ist die wichtigste. Sie ist die Kantenkonsistenz aus Kapitel 6, hier in ihrer einfachsten Form, und in gut gebauten Formeln erzwingt sie Kettenreaktionen über tausende Variablen.

Was seit 1999 hinzukam.

Klauseln lernen	Aus einem Konflikt wird eine neue Klausel abgeleitet, die denselben Fehler nie wieder zulässt
Nichtchronologisch zurücksetzen	Man springt zur Ursache des Konflikts zurück, nicht zur letzten Entscheidung
Zwei beobachtete Literale	Eine Datenstruktur, die das Fortpflanzen fast umsonst macht
Aktivitätsheuristik	Variablen, die zuletzt an Konflikten beteiligt waren, werden bevorzugt
Neustarts	Die Suche wird verworfen, die gelernten Klauseln bleiben

Die erste Zeile ist der eigentliche Durchbruch. Ein Konflikt wird nicht nur behoben, sondern ausgewertet: Man bestimmt die Teilbelegung, die ihn verursacht hat, und fügt ihre Negation als Klausel hinzu. Damit lernt der Löser aus jedem Fehlschlag.

Verbesserung	Beschleunigung	
Klauseln lernen	≈ 100-fach	**das meiste**
Zwei Literale beobachten	≈ 10-fach	
Aktivitätsheuristik	≈ 10-fach	
Neustarts	≈ 3-fach	

Die letzte Zeile klingt widersinnig: Man wirft die halbe Arbeit weg und wird schneller. Der Grund ist, dass die gelernten Klauseln erhalten bleiben und die Suche danach mit besserem Wissen und einer besseren Variablenreihenfolge neu beginnt.

Wofür man SAT-Löser benutzt

Schaltungsprüfung	Sind zwei Schaltungen äquivalent? Als Formel, die genau dann erfüllbar ist, wenn sie sich unterscheiden
Modellprüfung	Kann ein System in k Schritten einen verbotenen Zustand erreichen?
Planen	Gibt es einen Plan der Länge k? (Kapitel 11)
Paketverwaltung	Welche Fassungen sind gemeinsam installierbar?
Stundenpläne	Als Bedingungen kodiert

Das Muster ist stets dasselbe: Man übersetzt die Aufgabe in eine Formel, ruft den Löser und übersetzt die Antwort zurück. Der Löser weiß nichts über den Gegenstand, und genau darin liegt seine Brauchbarkeit — dieselbe Maschine für sehr verschiedene Fragen.

Es ist derselbe Gedanke wie beim Übersetzerbau in Band 6: eine gemeinsame Zwischenform, auf die viele Vorderteile abbilden und für die es ein sehr gut optimiertes Hinterteil gibt.

Die Kodierung entscheidet

Dieselbe Aufgabe lässt sich sehr verschieden in Formeln fassen, und die Lösungszeiten unterscheiden sich um Größenordnungen.
Beispiel: höchstens eines von n Literalen ist wahr.

Kodierung	**Klauseln**	**Hilfsvar.**	**bei $n = 100$**
Paarweise	$\binom{n}{2}$	0	4950
Befehlskette	$3n$	n	**300**
Binär	$n \log n$	$\log n$	700

Die zweite Zeile ist bei großen n deutlich besser, und zwar nicht nur wegen der Größe: Sie pflanzt sich auch besser fort. Wird ein Literal wahr, so setzt die Kette alle anderen in linear vielen Schritten auf falsch, während die paarweise Kodierung dafür quadratisch viele Klauseln durchgeht.

Das ist dieselbe Beobachtung wie in Kapitel 1: Die Darstellung entscheidet mehr als das Verfahren. Nur ist sie hier besonders unangenehm, weil man erst nach dem Ausprobieren weiß, welche Kodierung die bessere war.

Aufgabe 7.1.

(a) Führen Sie DPLL an einer Formel vollständig aus.
(b) Leiten Sie eine gelernte Klausel aus einem Konflikt ab.
(c) Untersuchen Sie, wo die schweren Fälle liegen.

Lösung Schritt für Schritt.

(a) DPLL.
Die Formel.

$$C_1 = (a \vee b \vee c), \quad C_2 = (\neg a \vee b), \quad C_3 = (\neg b \vee c),$$
$$C_4 = (\neg c \vee \neg a), \quad C_5 = (a \vee \neg c).$$

Schritt 1 – Einheitsklauseln? Keine. **Reine Literale?** Keine.
Schritt 2 – Verzweigen auf a, zuerst wahr.

Klausel	nach $a = \top$	Zustand	
C_1	erfüllt	—	
C_2	(b)	**Einheit**	$b = \top$
C_4	$(\neg c)$	**Einheit**	$c = \bot$
C_5	erfüllt	—	

Schritt 3 – Fortpflanzen: $b = \top$, $c = \bot$.

$$C_3 = (\neg b \vee c) \;\to\; (\bot \vee \bot) = \bot. \qquad \textbf{Konflikt}$$

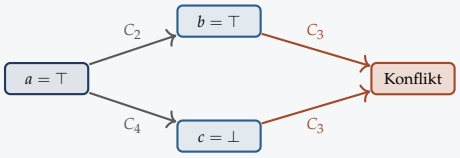
Schritt 4 – Zurück, a **falsch.**

C_2	erfüllt	—	
C_4	erfüllt	—	
C_5	$(\neg c)$	**Einheit**	$c = \bot$
C_1	$(b \vee c) \to (b)$	**Einheit**	$b = \top$
C_3	$(\neg b \vee c) \to \bot$	**Konflikt**	

Beide Zweige führen auf einen Konflikt. Die Formel ist ***unerfüllbar****.*

Bemerkenswert ist, dass in beiden Zweigen derselbe *Konflikt auftritt, nämlich an C_3 mit $b = \top$ und $c = \bot$. Ein Löser mit Klauselernen würde nach dem ersten Zweig eine Klausel ableiten, die den zweiten sofort erledigt.*

(b) Eine Klausel lernen.
Der Konflikt aus Schritt 3.



Die Ableitung durch Resolution.

Schritt	Klausel	Herkunft
1	$(\neg b \vee c)$	C_3, die verletzte
2	$(\neg a \vee c)$	mit $C_2 = (\neg a \vee b)$ über b
3	$(\neg a)$	mit $C_4 = (\neg c \vee \neg a)$ über c

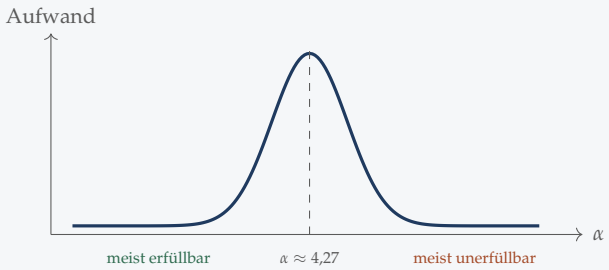
Gelernt wird die Klausel $(\neg a)$. Sie ist eine logische Folge der ursprünglichen Formel und schließt aus, dass a je wieder wahr gesetzt wird. Der zweite Zweig aus Aufgabe (a) wird damit nicht durchsucht, sondern erzwungen.

Warum die gelernte Klausel gültig ist.

Resolution	Aus $(X \vee P)$ und $(\neg X \vee Q)$ folgt $(P \vee Q)$
Folge	Jede abgeleitete Klausel ist eine logische Folge der Ausgangsformel
Also	Das Hinzufügen ändert die Erfüllbarkeit nicht und schneidet nur ab

Ein Löser kann also beliebig viele Klauseln lernen, ohne das Ergebnis zu gefährden. Das einzige Problem ist der Speicher, und deshalb werden selten benutzte gelernte Klauseln regelmäßig wieder verworfen.

(c) Wo die schweren Fälle liegen.
Der Übergang bei zufälligen Formeln.
Bei zufälligen Formeln mit drei Literalen je Klausel hängt die Schwierigkeit am Verhältnis $\alpha = m/n$ von Klauseln zu Variablen.



α	**erfüllbar**	**Aufwand**	
2,0	≈ 100 %	sehr gering	leicht
4,0	≈ 90 %	mittel	
4,27	≈ 50 %	**Maximum**	
5,0	≈ 5 %	mittel	
8,0	≈ 0 %	gering	leicht

Bei wenigen Klauseln findet man leicht eine Lösung; bei vielen sieht man den Widerspruch schnell. Schwer ist nur der schmale Bereich dazwischen, in dem beides gerade nicht mehr gilt. Das ist ein Phasenübergang wie in der Physik, mit einer scharfen Schwelle.

Warum das für die Praxis wenig bedeutet.

Zufällige Formeln	Haben keine Struktur; gelernte Klauseln nützen wenig
Formeln aus Anwendungen	Haben viel Struktur: Gemeinschaften, Wiederholungen, lokale Abhängigkeiten

Herkunft	**Variablen**	**Zeit**	
Zufällig, $\alpha = 4{,}27$	700	Stunden	schwer
Schaltungsprüfung	10^6	Minuten	leicht
Planung	10^5	Sekunden	leicht

Eine Formel mit einer Million Variablen aus einer realen Anwendung ist tausendmal leichter als eine zufällige mit siebenhundert. Das ist der eigentliche Grund für den Erfolg der modernen Löser: Sie nutzen Struktur aus, und reale Aufgaben haben welche.

Und es ist dieselbe Einsicht wie am Ende von Kapitel 5: Kein Verfahren ist allgemein gut. Ein SAT-Löser ist gut, weil die Formeln, die man ihm gibt, nicht zufällig sind.

Davis und Putnam[8] legten das Verfahren an; Marques-Silva und Sakallah[19] führten das konfliktgetriebene Lernen ein; das Handbuch von Biere und Mitautoren[2] deckt das ganze Gebiet ab.

8 Prädikatenlogik

Warum die Aussagenlogik nicht reicht

Aussagenlogik kennt nur ganze Behauptungen, keine Struktur darin.

Aussagenlogisch	`sokrates_ist_mensch`, `sokrates_ist_sterblich`
Das Problem	Der Zusammenhang zwischen beiden ist nicht ausdrückbar
Behelf	Für jeden Menschen eine eigene Regel; bei n Personen n Regeln

Die Prädikatenlogik führt Gegenstände, Beziehungen und Quantoren ein. Damit lässt sich „alle Menschen sind sterblich“ in einer Formel sagen statt in einer je Mensch.

$$\forall x \left(\text{Mensch}(x) \to \text{sterblich}(x)\right), \qquad \text{Mensch}(\text{sokrates}).$$

	Aussagenlogik	**Prädikatenlogik**
Ausdrucksstärke	gering	hoch
Erfüllbarkeit	entscheidbar	**unentscheidbar**
Allgemeingültigkeit	entscheidbar	**semi-entscheidbar**
Löser	sehr schnell	langsamer

Die zweite Zeile ist der Preis. Nach Band 2 lässt sich nicht entscheiden, ob eine prädikatenlogische Formel erfüllbar ist. Man kann nur systematisch nach einem Beweis suchen und findet ihn, wenn es einen gibt — sonst läuft die Suche ewig.

Diese Halbierung ist bemerkenswert: Man erhält eine positive Antwort in endlicher Zeit und eine negative unter Umständen nie. Für ein Beweisprogramm bedeutet das, dass „noch nicht fertig“ und „nicht beweisbar“ ununterscheidbar sind.

Unifikation

Die Prädikatenlogik verlangt einen Vorgang, den die Aussagenlogik nicht kennt: das Gleichmachen zweier Ausdrücke durch Einsetzen.

Definition 8.1. Ein **Unifikator** zweier Ausdrücke ist eine Ersetzung von Variablen durch Terme, die beide gleich macht. Der **allgemeinste Unifikator** ist derjenige, der am wenigsten festlegt.

Ausdruck 1	Ausdruck 2	Unifikator
$p(x, b)$	$p(a, y)$	$\{x/a,\ y/b\}$
$p(x, x)$	$p(a, b)$	keiner
$p(x, f(y))$	$p(g(z), f(a))$	$\{x/g(z),\ y/a\}$
$p(x)$	$p(f(x))$	Vorkommensprüfung

Die letzte Zeile ist berüchtigt. Eine Variable darf nicht durch einen Term ersetzt werden, der sie selbst enthält, sonst entsteht ein unendlicher Term. Die Prüfung darauf kostet Zeit, und viele Prolog-Umsetzungen lassen sie aus Geschwindigkeitsgründen weg.

Ohne diese Prüfung kann ein Programm in eine Endlosschleife laufen oder einen Speicherfehler erzeugen. Es ist eine bewusste Entscheidung gegen die Korrektheit zugunsten der Geschwindigkeit, und sie steht in jeder Prolog-Norm als Warnung.

Der Zusammenhang mit Band 6.

Typinferenz	Der Algorithmus von Milner löst Typgleichungen durch Unifikation
Mustervergleich	Funktionale Sprachen benutzen eine eingeschränkte Form
Beweisen	Resolution braucht sie, um Klauseln zusammenzuführen

Robinsons Verfahren von 1965 ist damit eines der am breitesten wiederverwendeten Ergebnisse der Informatik. Es steht in Beweisern, in Übersetzern und in Programmiersprachen, meist ohne dass jemand es beim Namen nennt.

Resolution

JOHN ALAN ROBINSON zeigte 1965, dass ein einziges Schlussprinzip für die gesamte Prädikatenlogik genügt.

Die Regel	Aus $(P \vee L)$ und $(Q \vee \neg L')$ folgt $(P \vee Q)\sigma$, wenn σ der allgemeinste Unifikator von L und L' ist
Vollständig	Ist eine Klauselmenge unerfüllbar, so lässt sich die leere Klausel ableiten
Widerlegend	Man beweist φ, indem man $\neg\varphi$ hinzunimmt und einen Widerspruch ableitet

Die dritte Zeile ist ein Kunstgriff, der das Verfahren erst brauchbar macht. Statt aus den Voraussetzungen vorwärts zu schließen, was in alle Richtungen führt, arbeitet man auf ein Ziel hin: den Widerspruch.

Der Weg zur Klauselform.

1. Implikationen und Äquivalenzen auflösen.
2. Negationen nach innen schieben.
3. Variablen umbenennen, damit keine zweimal vorkommt.
4. **Existenzquantoren durch Funktionen ersetzen** (Skolemisierung).
5. Allquantoren nach vorn ziehen und weglassen.
6. In konjunktive Normalform bringen.

Schritt 4 ist der interessanteste. „Jeder Mensch hat eine Mutter“ wird zu „jeder Mensch x hat die Mutter $\mathrm{mutter}(x)$“. Man erfindet also eine Funktion, die das behauptete Objekt liefert. Die neue Formel ist nicht äquivalent zur alten, aber erfüllbarkeitsgleich, und mehr braucht man für ein Widerlegungsverfahren nicht.

Aufgabe 8.1.

(a) Bringen Sie eine Formel in Klauselform.
(b) Führen Sie einen Resolutionsbeweis vollständig durch.
(c) Untersuchen Sie, warum die Suche entgleisen kann.

Lösung Schritt für Schritt.

(a) Von der Formel zur Klauselform.
Die Aussage. *Jeder, der alle Tiere liebt, wird von jemandem geliebt.*

$$\forall x \Big(\big(\forall y \, (\mathrm{Tier}(y) \to \mathrm{liebt}(x,y)) \big) \to \big(\exists z \, \mathrm{liebt}(z,x) \big) \Big).$$

Schritt 1 – Implikationen auflösen.

$$\forall x \Big(\neg \big(\forall y \, (\neg \mathrm{Tier}(y) \vee \mathrm{liebt}(x,y)) \big) \vee \exists z \, \mathrm{liebt}(z,x) \Big).$$

Schritt 2 – Negation nach innen.

$$\forall x \Big(\big(\exists y \, (\mathrm{Tier}(y) \wedge \neg \mathrm{liebt}(x,y)) \big) \vee \exists z \, \mathrm{liebt}(z,x) \Big).$$

Schritt 3 – Skolemisieren.
Beide Existenzquantoren stehen im Bereich von $\forall x$, also werden sie zu Funktionen von x:

$$\forall x \Big(\big(\mathrm{Tier}(f(x)) \wedge \neg \mathrm{liebt}(x, f(x)) \big) \vee \mathrm{liebt}(g(x), x) \Big).$$

Schritt 4 – Allquantor weglassen und ausmultiplizieren.

$$K_1 = \mathrm{Tier}(f(x)) \vee \mathrm{liebt}(g(x), x),$$
$$K_2 = \neg \mathrm{liebt}(x, f(x)) \vee \mathrm{liebt}(g(x), x).$$

Die Funktion f liefert zu jedem x ein Tier, das x nicht liebt — falls es eines gibt. Die Funktion g liefert jemanden, der x liebt. Beide sind erfunden und tragen keine Bedeutung; sie halten nur fest, dass ein solches Objekt existieren soll.

(b) Ein Resolutionsbeweis.
Die Voraussetzungen.

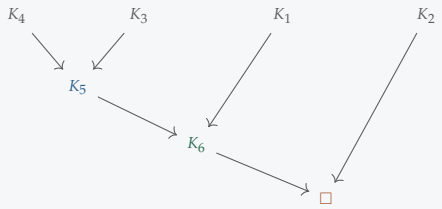
	Klausel	**Bedeutung**
K_1	$\neg \mathrm{Mensch}(x) \vee \mathrm{sterblich}(x)$	alle Menschen sterblich
K_2	$\mathrm{Mensch}(\mathrm{sokrates})$	
K_3	$\neg \mathrm{sterblich}(y) \vee \mathrm{endlich}(y)$	Sterbliche sind endlich

Die Behauptung. endlich(sokrates).
Die Negation als Klausel.

$$K_4 = \neg\text{endlich}(\text{sokrates}).$$

Die Ableitung.

Nr.	Klausel	aus	Unifikator
K_5	¬sterblich(sokrates)	K_3, K_4	$\{y/\text{sokrates}\}$
K_6	¬Mensch(sokrates)	K_1, K_5	$\{x/\text{sokrates}\}$
K_7	□	K_2, K_6	—



Die leere Klausel ist abgeleitet, also ist die Klauselmenge unerfüllbar, also folgt die Behauptung aus den Voraussetzungen. □

Bemerkenswert ist die Richtung. Der Beweis beginnt bei der negierten Behauptung und arbeitet sich zu den Voraussetzungen vor. Das ist keine Notwendigkeit der Regel, sondern eine Strategie: Sie hält die Suche auf das Ziel gerichtet.

(c) Warum die Suche entgleist.
Das Problem. Die Resolution ist vollständig und sagt nicht, in welcher Reihenfolge man resolvieren soll.

Klauseln	mögliche Paare	nach k Runden
10	45	55
55	1485	1540
1540	$1{,}2 \cdot 10^6$	$1{,}2 \cdot 10^6$

Ohne Steuerung explodiert die Klauselmenge nach wenigen Runden. Die allermeisten abgeleiteten Klauseln sind zwar korrekt und für den Beweis nutzlos.

Die Strategien.

Stützmenge	Jede Resolution muss mindestens eine Klausel benutzen, die von der negierten Behauptung abstammt
Einheitsvorzug	Kurze Klauseln zuerst; die leere ist das Ziel, also strebt man abwärts
Subsumtion	Eine Klausel, die von einer allgemeineren abgedeckt wird, wird verworfen
Linear	Jeder Schritt benutzt das Ergebnis des vorigen

Strategie	**Klauseln**	**vollständig?**
Ohne	10^6	ja
Stützmenge	10^4	ja
+ Subsumtion	10^3	ja
Nur Einheiten	10^2	nein

Die letzte Zeile zeigt die Abwägung. Manche Einschränkungen erhalten die Vollständigkeit, andere nicht. Ein Beweiser, der unvollständig ist, findet manche Beweise nie, und man erfährt es nicht, weil er einfach weiterläuft.

Die Unentscheidbarkeit ist keine Formalität

Beweis gefunden	Die Behauptung folgt; die Antwort ist sicher
Noch nicht gefunden	Entweder folgt sie nicht, oder der Beweis ist noch nicht dran

Ein Beweiser, der seit einer Stunde läuft, sagt nichts. Man kann nicht unterscheiden, ob die Behauptung falsch ist oder ob noch eine Minute fehlt. Das ist keine Schwäche des Verfahrens, sondern eine Folge von Band 2.

Ausweg 1	Sich auf entscheidbare Teilsprachen beschränken (Kapitel 10)
Ausweg 2	Eine Zeitschranke setzen und „unbekannt“ melden
Ausweg 3	Nach Gegenmodellen suchen, nicht nur nach Beweisen

Der erste Ausweg ist der folgenreichste. Die Beschreibungslogiken sind genau daraus entstanden: Man gibt Ausdrucksstärke auf, um Entscheidbarkeit und berechenbare Antwortzeiten zu gewinnen.

Robinson[27] führte Resolution und Unifikation ein; Russell und Norvig[28] behandeln die Umformung und die Strategien; McCarthy[20] stellte das Programm auf, Wissen logisch darzustellen.

9 Logisches Programmieren

Ein Programm als Menge von Tatsachen und Regeln

ROBERT KOWALSKI formulierte 1974 eine Gleichung, die dem Gebiet seinen Namen gab.

Algorithmus = Logik + Steuerung.

Logik	*Was* gilt: die Tatsachen und Regeln des Gegenstands
Steuerung	*Wie* gesucht wird: Reihenfolge, Rücknahme, Abschneiden
Die Hoffnung	Der Programmierer schreibt nur die Logik, das System liefert die Steuerung

Die Hoffnung hat sich nur zum Teil erfüllt, und Kapitel 14 kommt darauf zurück. In der Praxis muss man die Steuerung sehr wohl mitdenken, und ein Prolog-Programm, das logisch richtig und schlecht geordnet ist, läuft nicht.

Hornklauseln.
Prolog beschränkt sich auf Klauseln mit höchstens einem positiven Literal.

Art	Logisch	Prolog
Tatsache	A	`eltern(anna, bert).`
Regel	$A \leftarrow B_1 \wedge \cdots \wedge B_n$	`opa(X,Z) :- eltern(X,Y), eltern(Y,Z).`
Anfrage	$\leftarrow B_1 \wedge \cdots \wedge B_n$	`?- opa(anna, Z).`

Die Beschränkung auf Hornklauseln ist der Preis für die Effizienz. Man kann damit nicht ausdrücken, dass eines von zweien gilt, ohne zu sagen welches. Dafür wird die Resolution zu einem gerichteten, gut steuerbaren Verfahren.

Wie Prolog sucht

Die Abarbeitung ist eine Tiefensuche mit Rücknahme über die Regeln.

1. Nimm das linkeste Ziel der Anfrage.
2. Suche die Klauseln **von oben nach unten** durch.
3. Unifiziere den Kopf mit dem Ziel; gelingt es, ersetze das Ziel durch den Rumpf.
4. Weiter mit dem neuen linkesten Ziel.
5. Bei Fehlschlag: zurück zum letzten offenen Wahlpunkt.

Die Punkte 1 und 2 sind die Steuerung, und sie sind fest vorgegeben. Wer die Reihenfolge der Ziele oder der Klauseln ändert, ändert nichts an der Logik und alles am Verhalten.

Reihenfolge	**Logisch**	**Verhalten**
Basisfall zuerst	gleich	terminiert
Rekursion zuerst	gleich	oft Endlosschleife

Der Schnitt.
Der Operator `!` verwirft alle Wahlpunkte, die seit dem Aufruf des aktuellen Prädikats entstanden sind.

Grüner Schnitt	Entfernt nur Wahlpunkte, die ohnehin scheitern würden; die Bedeutung bleibt
Roter Schnitt	Ändert die Bedeutung; ohne ihn liefert das Programm andere Antworten

Der rote Schnitt ist der Punkt, an dem Kowalskis Gleichung bricht. Ein Programm mit rotem Schnitt lässt sich nicht mehr rein logisch lesen; man muss die Abarbeitung mitdenken. Er ist zugleich in der Praxis unentbehrlich.

Negation als Fehlschlag

Prolog kennt keine echte Negation. Es kennt `\+`, was bedeutet: Das Ziel ließ sich nicht beweisen.

Logische Negation	$\neg p$ gilt, wenn p falsch ist
Fehlschlagnegation	`\+ p` gilt, wenn p aus dem Programm nicht folgt

Das ist die Annahme der abgeschlossenen Welt: Was nicht in der Wissensbasis steht, gilt als falsch. Für eine Fahrplanauskunft ist das richtig — was nicht im Fahrplan steht, fährt nicht. Für ein medizinisches System ist es gefährlich.

	abgeschlossen	**offen**
Nicht abgeleitet heißt	**falsch**	**unbekannt**
Passend für	Datenbanken	**unvollständiges Wissen**
Beispiel	Fahrplan	Krankengeschichte

Diese Unterscheidung durchzieht die gesamte Wissensrepräsentation und kehrt in Kapitel 10 und 13 wieder. Sie ist der häufigste Grund dafür, dass ein System richtig rechnet und falsche Schlüsse zieht.

Die Reihenfolge bei der Negation

`\+` ist nur zuverlässig, wenn alle Variablen im Ziel bereits belegt sind.

```
?- \+ vogel(X),          scheitert immer
saeuger(X).

?- saeuger(X), \+        liefert die Säuger
vogel(X).
```

Im ersten Fall fragt Prolog: „Gibt es kein X, das ein Vogel ist?“ Da es Vögel gibt, scheitert das Ziel sofort. Im zweiten Fall ist X bereits belegt, und die Negation prüft eine konkrete Behauptung.

Beide Anfragen sind logisch gleichwertig, und nur eine funktioniert. Das ist die schärfste Illustration dafür, dass Logik und Steuerung sich in Prolog nicht trennen lassen.

Aufgabe 9.1.

(a) Verfolgen Sie eine Anfrage durch den Suchbaum.

(b) Untersuchen Sie, wie die Klauselreihenfolge über die Terminierung entscheidet.

(c) Zeigen Sie den Unterschied zwischen grünem und rotem Schnitt.

Lösung Schritt für Schritt.

(a) Eine Anfrage verfolgen.
Das Programm.

```
eltern(anna, bert).

eltern(anna, clara).

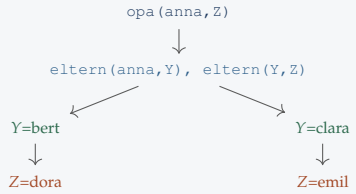
eltern(bert, dora).

eltern(clara, emil).

opa(X, Z) :- eltern(X, Y), eltern(Y,
Z).
```

Die Anfrage. `?- opa(anna, Z).`

Schr.	Ziele	Belegung	Aktion
1	`opa(anna,Z)`	—	Regel anwenden
2	`eltern(anna,Y), eltern(Y,Z)`	X/anna	
3	`eltern(Y,Z)`	Y/bert	**1. Tatsache**
4	—	Z/dora	**Lösung**
5	`eltern(Y,Z)`	Y/clara	**Rücknahme**
6	—	Z/emil	**Lösung**
7	—	—	keine weiteren



Zwei Lösungen: `Z = dora` und `Z = emil`. Der Baum wird von links nach rechts durchlaufen, und die Reihenfolge der Antworten folgt der Reihenfolge der Tatsachen im Programm.

(b) Klauselreihenfolge und Terminierung.
Fassung 1 – Basisfall zuerst.

```
vorfahr(X,Y) :- eltern(X,Y).
vorfahr(X,Y) :- eltern(X,Z),
vorfahr(Z,Y).
```

Fassung 2 – Rekursion zuerst und links.

```
vorfahr(X,Y) :- vorfahr(X,Z),
eltern(Z,Y).
vorfahr(X,Y) :- eltern(X,Y).
```

	Fassung 1	Fassung 2
Logisch	**gleichwertig**	
Verhalten	terminiert	**Endlosschleife**

Fassung 2 ruft sich als linkestes Ziel sofort selbst auf, ohne dass sich etwas verändert hätte. Die Rekursion erreicht nie einen Basisfall, und der Stapel läuft über. Es ist die Linksrekursion, die auch dem Zerleger in Band 6 zu schaffen macht, hier mit derselben Ursache.

Die Regel für die Praxis.

Klauseln	Basisfälle vor rekursive Fälle
Ziele	Erst das, was Variablen belegt, dann das, was sie prüft
Rekursion	Nicht ganz links; erst muss sich etwas verkleinert haben

(c) Grüner und roter Schnitt.
Ohne Schnitt.

```
max(X,Y,X) :- X >= Y.
max(X,Y,Y) :- X < Y.
```

Korrekt und leicht ineffizient: Bei `max(5,3,M)` wird nach der ersten Lösung noch die zweite Klausel versucht, obwohl $5 < 3$ *scheitern muss.*

Grüner Schnitt.

```
max(X,Y,X) :- X >= Y, !.
max(X,Y,Y) :- X < Y.
```

Anfrage	ohne Schnitt	mit grünem
`max(5,3,M)`	`M=5`	`M=5`
`max(3,5,M)`	`M=5`	`M=5`
`max(5,3,3)`	`nein`	`nein`

Die Antworten sind identisch; nur der Aufwand sinkt. Deshalb heißt der Schnitt hier grün: Er entfernt Wahlpunkte, die ohnehin gescheitert wären.

Roter Schnitt.

```
max(X,Y,X) :- X >= Y, !.
max(_,Y,Y).
```

Anfrage	Antwort	richtig?
`max(5,3,M)`	`M=5`	ja
`max(3,5,M)`	`M=5`	ja
`max(5,3,3)`	`ja`	**falsch**

Bei der dritten Anfrage ist das dritte Argument bereits mit 3 *belegt. Die erste Klausel scheitert an der Unifikation des Kopfes, der Schnitt wird nie erreicht, und die zweite Klausel behauptet, das Maximum von* 5 *und* 3 *sei* 3*.*

Der Fehler	Die zweite Klausel verlässt sich darauf, dass der Schnitt sie schützt
Die Bedingung	Sie gilt nur, wenn das Ergebnis unbelegt ankommt
Die Lehre	Ein rotes Programm ist nur für bestimmte Aufrufmuster richtig

Damit ist die Umkehrbarkeit dahin, die den Reiz der logischen Programmierung ausmacht. Ein reines Prolog-Prädikat lässt sich in jede Richtung benutzen: als Berechnung, als Prüfung, als Erzeugung. Ein rotes nicht.

Was von der Gleichung geblieben ist

Kowalskis Trennung von Logik und Steuerung war das Versprechen. Prolog hat es nur teilweise eingelöst.

Eingelöst	Für viele Aufgaben genügt die Logik; das System findet den Weg
Nicht eingelöst	Reihenfolge, Schnitt, Negation und Terminierung muss man mitdenken

Die Nachfolger haben daraus gelernt. Constraint-Programmierung über Logik (Kapitel 6) und Antwortmengenprogrammierung geben die feste Abarbeitungsreihenfolge auf und übergeben die Steuerung einem Löser. Damit ist die Trennung wieder hergestellt, und man zahlt mit einer eingeschränkten Sprache.

Colmerauer und Roussel[7] erzählen die Entstehung von Prolog; Kowalski[17] formulierte die Trennung von Logik und Steuerung; Bratko[3] ist die verbreitetste praktische Einführung.

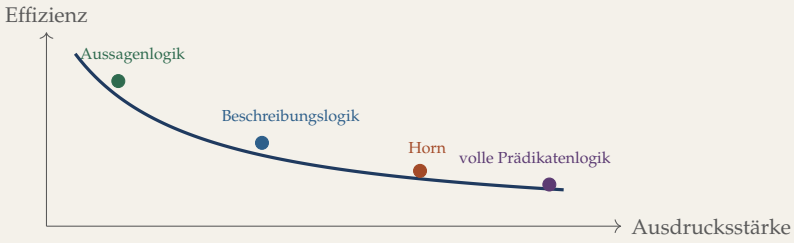
Part III

Wissen und Handeln

10 Wissen darstellen

Die Abwägung des Gebiets

Kapitel 8 endete mit einer Unentscheidbarkeit. Die gesamte Wissensrepräsentation lässt sich als Antwort darauf lesen.



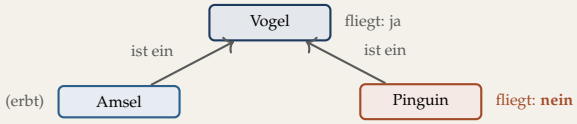
Sprache	Entscheidbar	Aufwand
Aussagenlogik	ja	NP-vollständig
Beschreibungslogik	ja	meist ExpTime
Hornklauseln	semi	unentscheidbar
Prädikatenlogik	nein	—

Es gibt keinen Punkt, an dem man beides bekommt. Jede Entscheidung für eine Darstellungssprache legt fest, was sich sagen und was sich berechnen lässt, und diese beiden Ziele stehen gegeneinander.

Semantische Netze und Rahmen

Vor der Logik gab es anschaulichere Vorschläge. MINSKY beschrieb 1974 die Rahmen.

Rahmen	Eine Sammlung benannter Felder für einen Begriff
Vererbung	Ein Unterbegriff übernimmt die Felder des Oberbegriffs
Vorgabewerte	Felder tragen typische Werte, die überschrieben werden dürfen
Auslöser	An ein Feld kann eine Berechnung geknüpft sein



Der Pinguin überschreibt den geerbten Wert. Das ist praktisch und logisch heikel: Aus „alle Vögel fliegen“ und „Pinguine sind Vögel“ folgt in der Prädikatenlogik, dass Pinguine fliegen. Ein Widerspruch lässt sich nicht durch Überschreiben beheben.

Genau daran hängt Kapitel 13. Vorgabewerte, die sich zurücknehmen lassen, verlangen eine Logik, in der neues Wissen alte Schlüsse aufheben kann.

Was an den frühen Netzen fehlte

Semantische Netze waren anschaulich und in ihrer Bedeutung ungeklärt.

Pfeil	Lesart 1	Lesart 2
Amsel → Vogel	jede Amsel ist ein Vogel	die Art Amsel gehört zur Gattung
Vogel → Flügel	jeder Vogel hat Flügel	Vögel haben typischerweise

Derselbe Pfeil trug sehr verschiedene Bedeutungen, und welche gemeint war, stand im Kopf des Erbauers. Zwei Systeme mit gleich aussehenden Netzen zogen verschiedene Schlüsse, und niemand konnte sagen, welches recht hatte.

Die Beschreibungslogiken sind aus dieser Kritik entstanden. Sie geben den Netzen eine genaue modelltheoretische Bedeutung und bezahlen mit einer eingeschränkten Ausdrucksstärke.

Beschreibungslogik

Eine Familie entscheidbarer Teilsprachen der Prädikatenlogik, zugeschnitten auf Begriffshierarchien.

Bestandteil	Schreibweise	Bedeutung
Konzept	Arzt	eine Menge von Dingen
Rolle	behandelt	eine Beziehung
Schnitt	$C \sqcap D$	beides zugleich
Einschränkung	$\forall R.C$	alle R-Nachfolger sind C
Existenz	$\exists R.C$	es gibt einen R-Nachfolger in C
Unterbegriff	$C \sqsubseteq D$	jedes C ist ein D

$$\text{Facharzt} \sqsubseteq \text{Arzt} \sqcap \exists \text{hatWeiterbildung.Fachgebiet}$$

Die Trennung in zwei Teile.

Teil	Enthält	Beispiel
Terminologie	Begriffsdefinitionen	$\text{Facharzt} \sqsubseteq \text{Arzt}$
Fakten	Einzelfälle	$\text{Arzt}(\text{maier})$

Die Standardaufgaben.

Erfüllbarkeit	Kann ein Begriff überhaupt Instanzen haben?
Einordnung	Welche Begriffe sind Unterbegriffe welcher?
Zugehörigkeit	Zu welchen Begriffen gehört ein Einzelfall?

Die erste Aufgabe ist praktisch die wichtigste. Ein Begriff, der keine Instanzen haben kann, ist meist ein Modellierungsfehler, und ein Einordnungsprogramm findet ihn automatisch. Bei medizinischen Terminologien mit hunderttausend Begriffen ist das unentbehrlich.

Terminologie	Begriffe	Anwendung
SNOMED CT	$\approx 350\,000$	klinische Befunde
Genontologie	$\approx 45\,000$	Molekularbiologie
ICD	$\approx 70\,000$	Diagnose-verschlüsselung

Diese Systeme sind das erfolgreichste Erbe der symbolischen KI. Sie heißen nicht so, sie werden täglich benutzt, und ihre Einordnungsprogramme sind Beschreibungslogik-Beweiser.

Aufgabe 10.1.

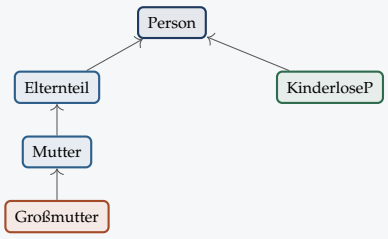
(a) Ordnen Sie eine kleine Terminologie ein.
(b) Untersuchen Sie, was die offene Weltannahme ändert.
(c) Rechnen Sie den Aufwand einer Wissensbasis nach.

Lösung Schritt für Schritt.

(a) Eine Terminologie einordnen.
Die Definitionen.

T_1	$\mathsf{Elternteil} \equiv \mathsf{Person} \sqcap \exists \mathsf{hatKind}.\mathsf{Person}$
T_2	$\mathsf{Mutter} \equiv \mathsf{Elternteil} \sqcap \mathsf{Weiblich}$
T_3	$\mathsf{Großmutter} \equiv \mathsf{Mutter} \sqcap \exists \mathsf{hatKind}.\mathsf{Elternteil}$
T_4	$\mathsf{KinderloseP} \equiv \mathsf{Person} \sqcap \forall \mathsf{hatKind}.\bot$

Die abgeleitete Hierarchie.



Der abgeleitete Schluss.

Behauptung	folgt?	Begründung
Großmutter $\sqsubseteq$ Mutter	ja	aus T_3
Großmutter $\sqsubseteq$ Elternteil	ja	über T_2
Elternteil $\sqcap$ KinderloseP	leer	**Widerspruch**

Die dritte Zeile ist der Nutzen der Einordnung. Niemand hat gesagt, dass die beiden Begriffe unvereinbar sind; das Programm leitet es aus $\exists$hatKind.Person und $\forall$hatKind.$\bot$ ab. Bei dreihunderttausend Begriffen findet man solche Konflikte nicht von Hand.

(b) Offene Weltannahme.
Die Wissensbasis.

Person(anna)	hatKind(anna, bert)
Person(bert)	

Frage	geschlossen	offen
Ist anna Elternteil?	ja	ja
Ist bert Elternteil?	nein	**unbekannt**
Hat anna genau ein Kind?	ja	**unbekannt**

Die zweite Zeile ist der entscheidende Unterschied. Unter der offenen Annahme heißt das Fehlen einer Angabe nicht, dass das Gegenteil gilt. Bert könnte Kinder haben, von denen die Wissensbasis nichts weiß.

Datenbank	Geschlossen; die Tabelle ist vollständig (Band 10)
Prolog	Geschlossen; Fehlschlagnegation (Kapitel 9)
Beschreibungslogik	**Offen**; Wissen ist grundsätzlich unvollständig

Für eine Krankengeschichte ist die offene Annahme die richtige: Dass eine Allergie nicht vermerkt ist, heißt nicht, dass keine besteht. Für einen Fahrplan wäre sie unbrauchbar. Die Wahl ist inhaltlich, nicht technisch.

Die praktische Folge.

Erwartung	„Zeige mir alle Personen ohne Kinder“
Ergebnis	leer, denn von niemandem ist bewiesen, dass er keine hat
Abhilfe	Ausdrücklich modellieren: $\mathsf{KinderloseP}(\mathrm{bert})$ als Tatsache eintragen

Das ist die häufigste Enttäuschung bei der Arbeit mit Ontologien. Wer sie wie eine Datenbank benutzt, bekommt leere Antworten, und die Ursache ist keine Fehlfunktion, sondern eine bewusste Entwurfsentscheidung.

(c) Der Aufwand einer Wissensbasis.
Die Erfahrungswerte.

Größe	**Wert**	
Regeln je Personentag	2–5	Expertensystem
Begriffe je Personentag	10–30	Terminologie
Aufwand für 10^4 Regeln	$\approx$ 3000 Tage	$\approx$ 13 Personenjahre

Das Beispiel Cyc.

Beginn	1984	
Geplant	$\approx$ 10 Jahre	Alltagswissen vollständig
Tatsächlich	über 35 Jahre	
Umfang	$> 10^6$ Aussagen	
Aufwand	$>$ 1000 Personenjahre	

Cyc ist das größte Vorhaben dieser Art und zugleich das strittigste. Es hat nie die erhoffte Wirkung entfaltet, und die Gründe dafür sind lehrreicher als die Zahlen. Kapitel 14 geht ihnen nach.

Die Kosten der Pflege.

Phase	Anteil am Aufwand	
Erstellen	$\approx$ 30 %	
Prüfen und berichtigen	$\approx$ 25 %	
Pflegen über die Jahre	$\approx$ 45 %	dauerhaft

Eine Wissensbasis ist kein Erzeugnis, sondern eine Verpflichtung. Wenn niemand sie pflegt, veraltet sie schneller, als sie Nutzen bringt, und genau daran sind die meisten Expertensysteme der 1980er Jahre gescheitert.

Der Engpass liegt nicht beim Schließen

Teil	Stand	
Schlussfolgern	gelöst	Löser sind schnell und korrekt
Wissen beschaffen	offen	**teuer, langsam, fehleranfällig**

> *Dieses Missverhältnis ist die zentrale Erfahrung des ganzen Gebiets. Man hat hervorragende Maschinen gebaut, die aus gegebenem Wissen zuverlässig Schlüsse ziehen, und keinen Weg gefunden, das Wissen in vertretbarem Aufwand hineinzubekommen.*

Minsky[22] schlug die Rahmen vor; das Handbuch von Baader und Mitautoren[1] deckt die Beschreibungslogiken ab; Lenat und Guha[18] beschreiben das Cyc-Vorhaben und seine Begründung.

11 Planen

Warum Planen nicht einfach Suche ist

Formal ist Planen eine Suche im Zustandsraum. Praktisch bricht die Suche aus Kapitel 2 sofort zusammen.

	Wegsuche	**Planen**
Zustand	ein Ort	**Belegung vieler Aussagen**
Zustände	10^5	2^n bei n Aussagen
Handlungen	wenige	**aus Schemata erzeugt**
Struktur	undurchsichtig	**sichtbar**

Die letzte Zeile ist die Rettung. Ein Planungszustand ist nicht undurchsichtig, sondern eine Menge geltender Aussagen, und eine Handlung sagt genau, welche davon sie hinzufügt und welche sie entfernt. Aus dieser Struktur lassen sich Heuristiken gewinnen, die keine Wegsuche hat.

Die Darstellung nach STRIPS.
FIKES und NILSSON führten 1971 eine Form ein, die bis heute die Grundlage bildet.

Zustand	Eine Menge von Aussagen, die gelten
Vorbedingung	Was gelten muss, damit die Handlung möglich ist
Hinzufügeliste	Was nach der Handlung zusätzlich gilt
Streichliste	Was nicht mehr gilt

```
Handlung:  fahre(v, von, nach)
Vor:  an(v, von), strasse(von, nach)
Add:  an(v, nach)
Del:  an(v, von)
```

Die stillschweigende Annahme lautet: Alles, was nicht in einer der beiden Listen steht, bleibt unverändert. Das klingt selbstverständlich und ist der Kern eines Problems, das die KI dreißig Jahre beschäftigt hat.

Das Rahmenproblem

McCarthy und Hayes benannten 1969 eine Schwierigkeit, die in der logischen Fassung des Planens auftritt.

Das Problem	In der Logik folgt aus „ich fahre das Auto“ nicht, dass die Farbe der Wand gleich bleibt
Der Behelf	Für jede Handlung und jede unbeteiligte Aussage eine Rahmenaxiomatik
Der Umfang	Bei a Handlungen und p Aussagen: $\mathcal{O}(a \cdot p)$ Axiome

Handlungen	Aussagen	Rahmenaxiome	
10	50	500	
100	1000	10^5	
1000	10^4	10^7	unhandhabbar

STRIPS löst das Problem, indem es die Annahme in das Verfahren einbaut statt in die Logik: Was nicht gestrichen wird, gilt weiter. Das ist keine logische Lösung, sondern eine Umgehung, und sie hat den Fortschritt beim Planen erst ermöglicht.

Rahmenproblem	Was ändert sich nicht?
Qualifikations-problem	Welche Bedingungen müssen alle erfüllt sein, damit eine Handlung gelingt?
Verzweigungs-problem	Welche Folgen hat eine Handlung indirekt?

Das zweite Problem ist unlösbar: Der Motor startet, wenn Benzin da ist, die Batterie geladen ist, kein Marder das Kabel durchgebissen hat, und so fort ohne Ende. Man kann die Liste nie abschließen, und Kapitel 13 zeigt den einzigen brauchbaren Umgang damit.

Drei Wege zum Plan

Ansatz	Sucht in	Stärke
Vorwärtssuche	Zuständen	gute Heuristiken
Rückwärtssuche	Teilzielen	wenige Zweige
Planraumsuche	unvollständigen Plänen	Reihenfolge offen lassen
Über SAT	**Formeln**	**fremder Löser**

Die entscheidende Heuristik.
Man lockert die Aufgabe, indem man alle Streichlisten weglässt.

Ohne Streichlisten	Nichts wird je ungültig; der Zustand wächst monoton
Folge	Die gelockerte Aufgabe ist in polynomialer Zeit lösbar
Die Schätzung	Die Länge dieses gelockerten Plans schätzt die echte

Das ist genau das Verfahren aus Kapitel 3: eine Bedingung streichen und die vereinfachte Aufgabe exakt lösen. Hier ist die gestrichene Bedingung, dass Handlungen etwas zerstören können, und genau das macht Planen schwer.

Verfahren	gelöste Aufgaben	Jahr
Planraumsuche	wenige	1990
Graphplan	mehr	1995
Vorwärts mit Lockerung	**viele**	**2000**
Mit Landmarken	mehr	2010

Der Sprung um 2000 war die Wiederentdeckung, dass eine einfache Vorwärtssuche mit einer guten Heuristik die ausgeklügelten Planraumverfahren schlägt. Es ist dieselbe Lehre wie bei SAT: Die Verfahrensstruktur zählt weniger als die Steuerung.

Aufgabe 11.1.

(a) Planen Sie in einer Klötzchenwelt von Hand.
(b) Berechnen Sie die Lockerungsheuristik.
(c) Übersetzen Sie eine Planungsaufgabe in eine Formel.

Lösung Schritt für Schritt.

(a) Die Klötzchenwelt.
Anfang und Ziel.



Der Anfangszustand.

$$\{\mathsf{auf}(C,A),\ \mathsf{auf}(A,B),\ \mathsf{aufTisch}(B),\ \mathsf{frei}(C),\ \mathsf{handLeer}\}$$

Das Ziel.

$$\{\mathsf{auf}(A,B),\ \mathsf{auf}(B,C)\}$$

Die Handlung.

```
Handlung:  nimm(x, y)
Vor:  auf(x,y), frei(x), handLeer
Add:  haelt(x), frei(y)
Del:  auf(x,y), frei(x), handLeer
```

Der Plan.

Schr.	Handlung	Zustand danach (Auszug)
1	`nimm(C,A)`	$\mathsf{haelt}(C), \mathsf{frei}(A)$
2	`setzeAbTisch(C)`	$\mathsf{aufTisch}(C), \mathsf{frei}(C)$
3	`nimm(A,B)`	$\mathsf{haelt}(A), \mathsf{frei}(B)$
4	`setzeAbTisch(A)`	$\mathsf{aufTisch}(A)$
5	`nimmVomTisch(B)`	$\mathsf{haelt}(B)$
6	`setzeAb(B,C)`	$\mathsf{auf}(B,C)$
7	`nimmVomTisch(A)`	$\mathsf{haelt}(A)$
8	`setzeAb(A,B)`	**Ziel erreicht**

Bemerkenswert sind die Schritte 3 und 4. Im Anfangszustand gilt bereits $\mathsf{auf}(A, B)$*, also ein Teilziel. Der Plan zerstört es und stellt es später wieder her. Ein Verfahren, das nie ein erreichtes Teilziel aufgibt, findet keinen Plan.*

Diese Aufgabe heißt die Sussman-Anomalie und war in den 1970er Jahren ein Gegenbeispiel gegen alle Planer, die Teilziele unabhängig behandelten. Sie ist der Grund, warum das Gebiet die Wechselwirkung zwischen Teilzielen ernst nehmen musste.

(b) Die Lockerungsheuristik.
Die Aufgabe. Zustand $\{p_1\}$, Ziel $\{p_4, p_5\}$.

Handl.	**Vor**	**Add**	**Del**
a_1	p_1	p_2	p_1
a_2	p_1	p_3	—
a_3	p_2, p_3	p_4	p_2
a_4	p_3	p_5	—

Der gelockerte Erreichbarkeitsgraph.

Ebene	**Aussagen**	**anwendbar**
0	p_1	a_1, a_2
1	p_1, p_2, p_3	a_1, a_2, a_3, a_4
2	$p_1, \ldots, p_5$	**Ziel enthalten**

Ohne Streichlisten wächst die Aussagenmenge nur. Nach zwei Ebenen sind alle Ziele enthalten, also ist die Schätzung der Planlänge zwei.

Die verfeinerte Schätzung: ein gelockerter Plan.
Man arbeitet vom Ziel rückwärts und sammelt die nötigen Handlungen ein:

Ziel	**erreicht durch**	**braucht**
p_4	a_3	p_2, p_3
p_5	a_4	p_3
p_2	a_1	p_1 ✓
p_3	a_2	p_1 ✓

$$h = |\{a_1, a_2, a_3, a_4\}| = \mathbf{4}.$$

Schätzung	Wert	Eigenschaft
Ebenen im Graphen	2	zulässig
Gelockerter Plan	4	nicht zulässig
Echte Planlänge	4	

Die zweite Zeile ist genauer und nicht mehr zulässig, denn den kürzesten gelockerten Plan zu finden ist selbst NP-schwer, und man begnügt sich mit einem beliebigen. Die meisten heutigen Planer verzichten deshalb auf Optimalität und suchen gierig.

(c) Planen über SAT.
Der Gedanke. Man fragt: Gibt es einen Plan der Länge genau k? Das lässt sich als aussagenlogische Formel schreiben.

Variablen	p^t für jede Aussage p und jeden Zeitpunkt $t \leq k$
	a^t für jede Handlung a und jeden Zeitpunkt $t < k$

Die Klauseln.

Anfang	p^0 für alle geltenden, $\neg p^0$ für alle übrigen
Ziel	p^k für jedes Teilziel
Vorbedingung	$a^t \to q^t$ für jede Vorbedingung q
Wirkung	$a^t \to p^{t+1}$ bzw. $a^t \to \neg p^{t+1}$
Rahmen	$\left(p^t \wedge \neg p^{t+1}\right) \to \bigvee\{a^t : p \in \mathrm{Del}(a)\}$
Ausschluss	höchstens eine Handlung je Zeitpunkt

Die Rahmenzeile ist die elegante Wendung. Statt zu sagen, was gleich bleibt, sagt man: Wenn sich etwas ändert, muss eine Handlung schuld sein. Das sind erheblich weniger Klauseln als die klassischen Rahmenaxiome, und das Rahmenproblem ist damit praktisch erledigt.

Der Ablauf.

k	**Variablen**	**Klauseln**	**Ergebnis**
1	120	840	unerfüllbar
2	240	1680	unerfüllbar
⋮			
8	960	6720	**erfüllbar**

Man erhöht k, bis der Löser eine Belegung findet. Die belegten Handlungsvariablen bilden den Plan, und weil man von unten kommt, ist er automatisch kürzestmöglich.

	eigener Planer	**über SAT**
Heuristiken	**aufgabenspezifisch**	keine nötig
Kürzeste Pläne	schwierig	von selbst
Nebenläufige Handlungen	umständlich	leicht
Lange Pläne	gut	Formel wächst mit k

Die letzte Zeile ist die Grenze. Bei Plänen mit hunderten Schritten wird die Formel zu groß, und ein eigener Planer mit guter Heuristik gewinnt. Bei kurzen Plänen mit vielen Wechselwirkungen ist der Umweg über SAT deutlich überlegen.

Das Muster ist dasselbe wie in Kapitel 7: Man übersetzt eine Aufgabe in eine Zwischenform, für die es eine sehr gut optimierte Maschine gibt. Wer das kann, erbt jeden künftigen Fortschritt an dieser Maschine, ohne selbst etwas zu tun.

Fikes und Nilsson[11] führten STRIPS ein; Ghallab, Nau und Traverso[12] sind die maßgebliche Darstellung des Gebiets; Russell und Norvig[28] behandeln die Heuristiken und die Übersetzung nach SAT.

12 Unsicherheit

Warum die Logik hier nicht genügt

Ein Arzt schließt nicht: „Wenn Fieber, dann Grippe.“ Er schließt: „Fieber macht Grippe wahrscheinlicher.“

Faulheit	Alle Vorbedingungen aufzuzählen ist unmöglich (Qualifikationsproblem)
Unwissen	Die Zusammenhänge sind nicht vollständig bekannt
Unvollständigkeit	Nicht alle nötigen Beobachtungen liegen vor

Die frühen Systeme behalfen sich mit eigenen Zahlensystemen für Gewissheit, die man addierte und multiplizierte. Sie funktionierten in engen Gebieten und ließen sich nicht begründen: Niemand konnte sagen, was ein Gewissheitsfaktor von 0,7 bedeutet.

Der Satz von Bayes.

$$P(H \mid E) = \frac{P(E \mid H)\, P(H)}{P(E)}.$$

$P(H)$	Was man vorher glaubte
$P(E \mid H)$	Wie gut die Beobachtung zur Erklärung passt
$P(H \mid E)$	Was man nachher glaubt

Der Satz ist eine Umformung der Definition der bedingten Wahrscheinlichkeit und mathematisch trivial. Seine Wirkung liegt darin, dass er die Richtung umdreht: Man kennt meist $P(\text{Befund} \mid \text{Krankheit})$ und braucht $P(\text{Krankheit} \mid \text{Befund})$.

Die Grundrate wird systematisch vergessen

Der Nenner enthält $P(H)$, und genau diesen Faktor unterschlagen Menschen regelmäßig.

Ein Beispiel. Ein Test hat eine Empfindlichkeit von 99 % und eine Spezifität von 99 %. Die Krankheit tritt bei einem von zehntausend auf.

bei 10^6	**krank**	**gesund**	**Summe**
Test positiv	99	**9999**	10 098
Test negativ	1	989 901	989 902
Summe	100	999 900	10^6

$$P(\text{krank} \mid \text{positiv}) = \frac{99}{10\,098} = \mathbf{0{,}98}\,\%.$$

Ein Test mit neunundneunzig Prozent Treffsicherheit liefert bei einem positiven Ergebnis eine Krankheitswahrscheinlichkeit von einem Prozent. Der Grund steht in der zweiten Spalte: Es gibt zehntausendmal mehr Gesunde, und ein Prozent davon sind hundertmal mehr Falschpositive als es Kranke gibt.

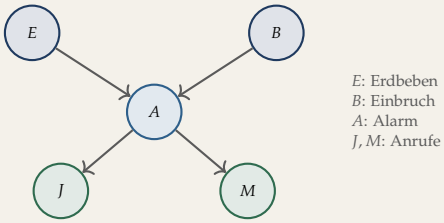
Dies ist derselbe Zusammenhang wie beim diagnostischen Beispiel in Band 7, dort informationstheoretisch, hier als Rechnung. Wer Systeme baut, die Befunde bewerten, muss die Grundrate ausdrücklich modellieren, weil sie sonst stillschweigend auf die Hälfte gesetzt wird.

Bayes-Netze

JUDEA PEARL zeigte 1988, wie sich Wahrscheinlichkeiten handhabbar darstellen lassen, indem man Unabhängigkeiten als Graph aufschreibt.

Darstellung	**Einträge**	**bei** 20 **Variablen**
Volle Verbundtabelle	$2^n - 1$	$\approx 10^6$
Bayes-Netz, $\leq$ 3 Eltern	$\leq n \cdot 2^3$	**160**

Faktor sechstausend, und zwar durch dieselbe Art von Einsicht wie in Kapitel 6: Man nutzt aus, dass die meisten Größen einander nicht unmittelbar beeinflussen. Der Graph ist die Buchführung darüber, was von was abhängt.



Die Kernaussage	Jede Variable ist von ihren Nichtnachfahren unabhängig, wenn ihre Eltern bekannt sind
Die Faktorisierung	$P(x_1, \ldots, x_n) = \prod_i P(x_i \mid \text{Eltern}(x_i))$

Sind J und M abhängig? Ja, denn beide hängen vom Alarm ab. Sind sie es noch, wenn man den Alarm kennt? Nein. Das ist die Bedingung der zweiten Zeile, und sie erlaubt es, das Netz als Produkt kleiner Tabellen zu schreiben.

Wann Pfade Information leiten

Ob zwei Knoten bei gegebener Beobachtung abhängig sind, lässt sich am Graphen ablesen.

Muster	Mitte unbekannt	Mitte bekannt
Kette $X \to Z \to Y$	leitet	blockiert
Gabel $X \leftarrow Z \to Y$	leitet	blockiert
Kollider $X \to Z \leftarrow Y$	blockiert	**leitet**

Die dritte Zeile ist umgekehrt zu den ersten beiden und der überraschendste Befund des Kapitels. Erdbeben und Einbruch sind unabhängig. Weiß man aber, dass der Alarm ging, so senkt ein gemeldetes Erdbeben die Wahrscheinlichkeit eines Einbruchs: Die Ursache ist erklärt.

Dieses Wegerklären ist der Grund, warum sich Unsicherheit nicht mit lokalen Gewissheitsfaktoren behandeln lässt. Eine Beobachtung kann den Einfluss einer anderen umkehren, und das lässt sich nicht durch Multiplizieren von Zahlen längs Regeln nachbilden.

Aufgabe 12.1.

(a) Rechnen Sie in einem Bayes-Netz eine Anfrage aus.
(b) Zeigen Sie das Wegerklären an Zahlen.
(c) Untersuchen Sie, wie teuer exakte Inferenz ist.

Lösung Schritt für Schritt.

(a) Eine Anfrage im Netz.
Die Tabellen.

$P(B) = 0{,}001$	$P(E) = 0{,}002$
$P(A \mid B, E)$:	
$B{=}\top, E{=}\top$	0,95
$B{=}\top, E{=}\bot$	0,94
$B{=}\bot, E{=}\top$	0,29
$B{=}\bot, E{=}\bot$	0,001
$P(J \mid A{=}\top) =$ 0,90	$P(J \mid A{=}\bot) =$ 0,05
$P(M \mid A{=}\top) = 0{,}70$	$P(M \mid A{=}\bot) = 0{,}01$

Die Frage. $P(B \mid J{=}\top, M{=}\top)$?
Schritt 1 – Die Summe über die verborgenen Variablen.

$$P(B, j, m) = \sum_e \sum_a P(B)\, P(e)\, P(a \mid B, e)\, P(j \mid a)\, P(m \mid a).$$

Schritt 2 – Für $B = \top$.
Zuerst die innere Summe über a, für jedes e:

$$\begin{aligned} e{=}\top: &\quad 0{,}95 \cdot 0{,}90 \cdot 0{,}70 + 0{,}05 \cdot 0{,}05 \cdot 0{,}01 = 0{,}5985 + 0{,}000025 = 0{,}598525, \\ e{=}\bot: &\quad 0{,}94 \cdot 0{,}90 \cdot 0{,}70 + 0{,}06 \cdot 0{,}05 \cdot 0{,}01 = 0{,}5922 + 0{,}00003 = 0{,}592230. \end{aligned}$$

Dann über e:

$$0{,}002 \cdot 0{,}598525 + 0{,}998 \cdot 0{,}592230 = 0{,}0011971 + 0{,}5910455 = 0{,}5922426.$$

$$P(B{=}\top, j, m) = 0{,}001 \cdot 0{,}5922426 = 5{,}922 \cdot 10^{-4}.$$

Schritt 3 – Für $B = \bot$.

$$\begin{aligned} e{=}\top: &\quad 0{,}29 \cdot 0{,}63 + 0{,}71 \cdot 0{,}0005 = 0{,}1827 + 0{,}000355 = 0{,}183055, \\ e{=}\bot: &\quad 0{,}001 \cdot 0{,}63 + 0{,}999 \cdot 0{,}0005 = 0{,}00063 + 0{,}0004995 = 0{,}0011295. \end{aligned}$$

$$0{,}002 \cdot 0{,}183055 + 0{,}998 \cdot 0{,}0011295 = 0{,}000366 + 0{,}001127 = 0{,}001493.$$

$$P(B{=}\bot, j, m) = 0{,}999 \cdot 0{,}001493 = 1{,}4915 \cdot 10^{-3}.$$

Schritt 4 – Normieren.

$$P(B{=}\top \mid j, m) = \frac{5{,}922 \cdot 10^{-4}}{5{,}922 \cdot 10^{-4} + 1{,}4915 \cdot 10^{-3}} = \frac{5{,}922}{20{,}837} = \mathbf{0{,}284}.$$

Vorher	0,001	
Nach zwei Anrufen	**0,284**	Faktor 284

Zwei unabhängige Anrufe erhöhen die Einbruchswahrscheinlichkeit um Faktor zweihundertvierundachtzig und lassen sie trotzdem unter dreißig Prozent. Die Grundrate von einem Promille wirkt weiter, und genau das ist der Punkt aus dem Warnkasten oben.

(b) Wegerklären in Zahlen.
Ausgangslage. Der Alarm ging: $A = \top$.

Beobachtung	$P(B{=}\top \mid \dots)$	
nichts	0,001	Grundrate
$A = \top$	**0,374**	stark erhöht
$A = \top, E = \top$	**0,003**	fast zurück

Die Rechnung für die dritte Zeile.

$$P(B{=}\top \mid A{=}\top, E{=}\top) = \frac{P(A \mid B{=}\top, E{=}\top)P(B{=}\top)}{\sum_b P(A \mid b, E{=}\top)P(b)}.$$

$$= \frac{0{,}95 \cdot 0{,}001}{0{,}95 \cdot 0{,}001 + 0{,}29 \cdot 0{,}999} = \frac{0{,}00095}{0{,}00095 + 0{,}28971} = \frac{0{,}00095}{0{,}29066} = \mathbf{0{,}00327}.$$

Die Nachricht vom Erdbeben senkt die Einbruchswahrscheinlichkeit von 37,4 auf 0,33 Prozent, ohne dass zwischen Erdbeben und Einbruch irgendein Zusammenhang bestünde. Der Alarm ist erklärt, und die zweite Erklärung wird überflüssig.

Ein regelbasiertes System mit Gewissheitsfaktoren kann das nicht abbilden. Es kennt nur Regeln, die Gewissheit erhöhen oder senken, und hier hängt die Wirkung einer Beobachtung davon ab, was sonst noch bekannt ist.

(c) Der Aufwand exakter Inferenz.
Das Ergebnis. Exakte Inferenz in Bayes-Netzen ist NP-schwer, im allgemeinen Fall sogar #P-schwer.

Netzform	Aufwand	
Kette	$\mathcal{O}(n)$	leicht
Baum	$\mathcal{O}(n)$	leicht
Polybaum	$\mathcal{O}(n)$	leicht
Mit Kreisen	$\mathcal{O}(n\,2^w)$	w: Baumweite
Dicht	exponentiell	aussichtslos

Die Baumweite tritt hier zum zweiten Mal auf; in Kapitel 6 bestimmte sie den Aufwand bei Nebenbedingungen. Es ist dieselbe Größe und derselbe Grund: Wie weit ist der Graph von einem Baum entfernt?

w	2^w	**bei** $n = 1000$	
5	32	$3{,}2 \cdot 10^4$	sofort
15	32 768	$3{,}3 \cdot 10^7$	Sekunden
25	$3{,}4 \cdot 10^7$	$3{,}4 \cdot 10^{10}$	Stunden
40	$1{,}1 \cdot 10^{12}$	$1{,}1 \cdot 10^{15}$	aussichtslos

Der Ausweg über Stichproben.

Vorwärts ziehen	Werte längs der Pfeile würfeln; Läufe verwerfen, die zur Beobachtung nicht passen
Gewichten	Beobachtungen festhalten und den Lauf mit ihrer Wahrscheinlichkeit gewichten
Markow-Kette	Eine Variable nach der anderen neu ziehen, die übrigen festhalten

Verfahren	Läufe	nutzbar	Fehler
Verwerfen	10^6	≈ 2000	$\pm 2\,\%$
Gewichten	10^6	10^6	$\pm 0{,}3\,\%$
Markow-Kette	10^6	10^6	$\pm 0{,}2\,\%$

Das Verwerfen ist bei seltenen Beobachtungen fast unbrauchbar: Von einer Million Läufen passen zweitausend. Es ist derselbe Effekt wie bei der Grundrate — seltene Ereignisse treten selten ein, auch in der Simulation.

Ein Netz ist kein Kausalmodell

Die Pfeile in einem Bayes-Netz drücken bedingte Unabhängigkeiten aus, nicht Ursachen.

Gleiche Verteilung	$X \to Y$ und $X \leftarrow Y$ beschreiben dieselbe gemeinsame Verteilung
Unterschied	Nur bei einem Eingriff: Wer an X dreht, ändert Y nur in einer der beiden Lesarten

Aus Beobachtungsdaten allein lässt sich die Richtung meist nicht bestimmen. Pearl selbst hat diesem Unterschied seine spätere Arbeit gewidmet und dafür eine eigene Notation entwickelt, die den Eingriff vom Beobachten trennt.

Für dieses Buch genügt die Warnung: Ein Netz, das gut vorhersagt, sagt nichts darüber, was geschieht, wenn man eingreift. Es ist dieselbe Grenze wie bei der Transinformation in Band 7 und bei der Verbindungsstruktur in Band 11.

Pearl[25] begründete die Bayes-Netze und die zugehörige Inferenz; Russell und Norvig[28] führen sie mit vielen Beispielen ein.

13 Nichtmonotones Schließen

Eine Eigenschaft, die stört

Die klassische Logik ist monoton: Was einmal folgt, folgt auch nach jeder Ergänzung.

Monotonie	Aus $\Gamma \vdash \varphi$ folgt $\Gamma \cup \{\psi\} \vdash \varphi$ für jedes ψ
Der Nutzen	Beweise bleiben gültig; man kann Wissen sammeln, ohne Altes zu prüfen
Das Problem	**Menschliches Schließen ist nicht so**

	Wissen	**Schluss**
1	Tweety ist ein Vogel	Tweety fliegt
2	+ Tweety ist ein Pinguin	Tweety fliegt nicht
3	+ Tweety sitzt im Flugzeug	Tweety fliegt

Jede Ergänzung kehrt den Schluss um. In der klassischen Logik ist das unmöglich: Sobald „Tweety fliegt" folgt, folgt es für immer. Man müsste stattdessen alle Ausnahmen vorab in die Regel schreiben, und deren Liste lässt sich nicht abschließen — das Qualifikationsproblem aus Kapitel 11.

Warum das keine Spitzfindigkeit ist.

Diagnose	Ein neuer Befund kann eine Verdachtsdiagnose aufheben
Recht	Eine Ausnahmeregel hebt die Grundregel auf
Handlungsplanung	Ein Hindernis macht einen fertigen Plan ungültig
Vorgabewerte	Ein Rahmen aus Kapitel 10 lebt davon

In allen vier Fällen zieht man einen Schluss aus dem, was man weiß, und ist bereit, ihn zurückzunehmen. Eine Logik, die das nicht kann, taugt für keinen dieser Bereiche.

Default-Logik

RAYMOND REITER schlug 1980 vor, die klassische Logik um Regeln zu erweitern, die nur unter Vorbehalt gelten.

Form	$\dfrac{\alpha \;:\; \beta}{\gamma}$
Lesart	Wenn α gilt und β mit dem Bekannten verträglich ist, dann schließe γ

$$\frac{\mathsf{Vogel}(x) \;:\; \mathsf{fliegt}(x)}{\mathsf{fliegt}(x)}$$

Der mittlere Teil ist der neue Bestandteil. Er prüft nicht, ob etwas folgt, sondern ob das Gegenteil nicht *folgt. Damit hängt der Schluss vom Gesamtzustand des Wissens ab und kann durch Hinzufügen ungültig werden.*

Erweiterungen.

Definition	Eine Erweiterung ist eine in sich stimmige Menge von Schlüssen, die alle anwendbaren Defaults ausnutzt
Eigenart	Es kann **mehrere** geben, und es kann **keine** geben

Das Nixon-Beispiel.

Quäker sind meist Pazifisten	$\dfrac{Q(x) : P(x)}{P(x)}$
Republikaner meist nicht	$\dfrac{R(x) : \neg P(x)}{\neg P(x)}$
Nixon war beides	$Q(\mathrm{n}), R(\mathrm{n})$

Erweiterung 1	$P(\mathrm{n})$	Pazifist
Erweiterung 2	$\neg P(\mathrm{n})$	kein Pazifist

Beide sind gleichermaßen zulässig, und die Logik gibt keinen Grund, eine vorzuziehen. Das ist kein Mangel des Formalismus, sondern eine ehrliche Abbildung der Lage: Die beiden Faustregeln widersprechen sich, und ohne weitere Angaben ist die Frage offen.

Vorsichtig	Nur glauben, was in **allen** Erweiterungen gilt
Mutig	Eine Erweiterung wählen und dabei bleiben
Mit Vorrang	Spezifischere Regeln schlagen allgemeinere

Die dritte Zeile löst den Pinguin, nicht den Nixon: „Pinguine fliegen nicht“ ist spezifischer als „Vögel fliegen“, aber Quäker und Republikaner stehen nebeneinander.

Andere Zugänge

Umschreibung	Man nimmt an, dass genau die Ausnahmen vorliegen, die bekannt sind, und keine weiteren
Abgeschlossene Welt	Was nicht ableitbar ist, gilt als falsch (Kapitel 9)
Wahrheitspflege	Ein System merkt sich, worauf jeder Schluss beruht, und nimmt ihn bei Wegfall zurück
Antwortmengen	Programme, deren Bedeutung über stabile Modelle erklärt wird

Die dritte Zeile ist die praktisch wirksamste. Ein Wahrheitspflegesystem führt Buch über die Begründungen: Wird eine Voraussetzung entfernt, so verschwinden genau die Schlüsse, die darauf beruhten, und keine anderen.

	ohne Buchführung	**mit**
Rücknahme	alles neu ableiten	nur Betroffenes
Aufwand	$\mathcal{O}(\text{alles})$	$\mathcal{O}(\text{Abhängige})$
Erklärbarkeit	keine	**Begründungskette**

Die letzte Zeile hat sich als wertvoller erwiesen als die Effizienz. Ein System, das sagen kann, worauf ein Schluss beruht, lässt sich prüfen und in Frage stellen. Das war bei den Expertensystemen der 1980er Jahre der meistgeschätzte Bestandteil.

Der Preis ist hoch

Frage	klassisch	mit Defaults
Folgt φ?	semi-entscheidbar	schwerer
Aussagenlogischer Fall	NP-vollständig	Σ_2^p-vollständig
Erweiterung existiert?	—	nicht garantiert

Die zweite Zeile bedeutet eine Stufe höher in der Hierarchie aus Band 3: Man braucht ein Orakel für NP, um in polynomialer Zeit zu entscheiden. Praktisch heißt das, dass jeder Default-Schluss eine eingebettete Erfüllbarkeitsprüfung verlangt.

Deshalb sind die reinen Formalismen dieses Kapitels selten im Einsatz. Was überlebt hat, sind die abgeschwächten Formen: Vorrangregeln in Ontologien, Fehlschlagnegation in Prolog, und die Antwortmengenprogrammierung, die die Schwierigkeit an einen spezialisierten Löser abgibt.

Aufgabe 13.1.

(a) Bestimmen Sie die Erweiterungen einer Default-Theorie.
(b) Untersuchen Sie, wie Spezifität Konflikte löst.
(c) Zeigen Sie die Wirkung einer Begründungsverwaltung.

Lösung Schritt für Schritt.

(a) Erweiterungen bestimmen.
Die Theorie.

Fakten	$\{\mathsf{Vogel}(t)\}$	
d_1	$\dfrac{\mathsf{Vogel}(x) : \mathsf{fliegt}(x)}{\mathsf{fliegt}(x)}$	Vögel fliegen meist
d_2	$\dfrac{\mathsf{Pinguin}(x) : \neg\mathsf{fliegt}(x)}{\neg\mathsf{fliegt}(x)}$	Pinguine nicht

Fall 1 – Nur $\mathsf{Vogel}(t)$.

Schr.	Prüfung	Ergebnis	
1	d_1: $\mathsf{Vogel}(t)$ gilt	ja	
2	$\mathsf{fliegt}(t)$ verträglich?	ja	anwenden
3	d_2: $\mathsf{Pinguin}(t)$?	nein	nicht anwendbar

$E = \{\mathsf{Vogel}(t),\ \mathsf{fliegt}(t)\}$ — genau eine Erweiterung.

Fall 2 – Zusätzlich $\mathsf{Pinguin}(t)$ und $\mathsf{Pinguin}(x) \to \mathsf{Vogel}(x)$.

Erweiterung	enthält	welcher Default
E_1	$\mathsf{fliegt}(t)$	d_1 zuerst
E_2	$\neg\mathsf{fliegt}(t)$	d_2 zuerst

Zwei Erweiterungen, und beide sind formal gleichberechtigt. Dass wir E_2 für richtig halten, ist zusätzliches Wissen, das in der Theorie nirgends steht: die Einsicht, dass die Pinguinregel spezifischer ist.

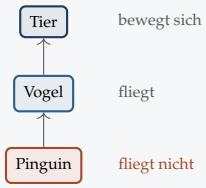
Fall 3 – Ein Default ohne Erweiterung.

$$d = \frac{\top \ :\ p}{\neg p}.$$

1	Wenden wir d nicht an, so ist p verträglich, also wäre d anwendbar.
2	Wenden wir d an, so gilt $\neg p$, also ist p nicht mehr verträglich, also war d nicht anwendbar.

Kein stabiler Zustand. Die Theorie hat gar keine Erweiterung. Das ist die Kehrseite der Selbstbezüglichkeit, die den Formalismus interessant macht, und dieselbe Struktur wie bei den Widersprüchen aus Band 2.

(b) Spezifität als Konfliktlösung.
Die Hierarchie.



Die Regel. Bei Konflikt gewinnt der Default, dessen Vorbedingung die andere logisch nach sich zieht.

Vergleich	Folgerung	Vorrang
Pinguin vs. Vogel	Pinguin $\rightarrow$ Vogel	Pinguin
Quäker vs. Republikaner	keine	unentschieden

Die Regel greift genau dann, wenn eine Klasse in der anderen enthalten ist. Bei sich überschneidenden Klassen versagt sie, und das ist kein Mangel: Es gibt in diesem Fall wirklich keinen sachlichen Grund für einen Vorrang.

Was in der Praxis daraus wurde.

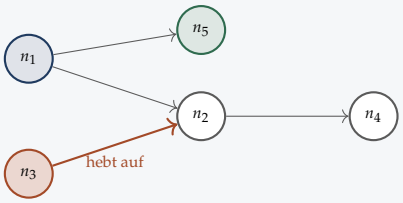
Objektorientierung	Die Methode der spezielleren Klasse überschreibt die allgemeinere
Regelsysteme	Regeln mit mehr Bedingungen werden zuerst geprüft
Ontologien	Ausnahmen werden ausdrücklich modelliert, nicht überschrieben

Die erste Zeile ist bemerkenswert: Die Vererbung in Programmiersprachen ist die praktische Umsetzung genau dieser Spezifitätsregel, und sie hat dieselben Grenzen — bei Mehrfachvererbung entsteht dasselbe Nixon-Problem, und die Sprachen lösen es durch eine willkürliche Reihenfolge.

(c) Begründungsverwaltung.
Die Ausgangslage.

Knoten	Aussage	Begründung
n_1	$\mathsf{Vogel}(t)$	Beobachtung
n_2	$\mathsf{fliegt}(t)$	n_1, kein n_3
n_3	$\mathsf{Pinguin}(t)$	— *(nicht belegt)*
n_4	$\mathsf{erreichtNest}(t)$	n_2
n_5	$\mathsf{hatFedern}(t)$	n_1

Der Eingriff. n_3 wird belegt: Tweety ist ein Pinguin.



Knoten	vorher	nachher	
n_1	gültig	gültig	
n_2	gültig	**ungültig**	unmittelbar
n_4	gültig	**ungültig**	**mittelbar**
n_5	gültig	gültig	unberührt

n_4 fällt weg, obwohl niemand etwas über das Nest gesagt hat: Es beruhte auf n_2. n_5 bleibt, weil es an n_1 hängt und nicht am aufgehobenen Schluss. Ohne Buchführung müsste man alles neu ableiten und wüsste nicht, welche Schlüsse betroffen sind.

Der Aufwand.

Schlüsse	neu ableiten	mit Buchführung	Faktor
10^3	10^3	≈ 8	125
10^5	10^5	≈ 20	5000
10^6	10^6	≈ 35	**29 000**

Die Zahl der betroffenen Schlüsse wächst nur logarithmisch mit der Größe der Wissensbasis, solange die Abhängigkeiten flach bleiben. Das macht die Buchführung so wirksam und erklärt, warum sie in fast jedes größere Regelsystem eingebaut wurde.

Der eigentliche Gewinn ist aber ein anderer. Ein System, das die Begründungskette kennt, kann sie ausgeben. Auf die Frage „warum?“ antwortet es mit den Voraussetzungen, und auf „warum nicht?“ mit der Stelle, an der die Kette bricht. Das ist die Erklärbarkeit, die den Expertensystemen ihre Glaubwürdigkeit gab.

Reiter[26] entwarf die Default-Logik; McCarthy und Hayes[21] benannten das Rahmenproblem; McCarthy[20] schlug die Umschreibung als Alternative vor.

Part IV

Bilanz

14 Was schiefging und was blieb

Zwei Zusammenbrüche

Das Gebiet hat zweimal erlebt, dass die Förderung versiegte und die Erwartungen sich als unhaltbar erwiesen.

	Erster Winter	Zweiter Winter
Zeitraum	1974–1980	1987–1993
Auslöser	Lighthill-Bericht 1973	Zusammenbruch des Marktes für LISP-Maschinen
Was versprochen war	maschinelle Übersetzung, allgemeines Problemlösen	Expertensysteme für alles
Was fehlte	Rechenleistung, Wissen	**Pflegbarkeit**

JAMES LIGHTHILL *untersuchte 1973 im Auftrag des britischen Forschungsrats den Stand des Gebiets. Sein Befund: Die Verfahren funktionieren an Spielzeugaufgaben und versagen, sobald die Aufgabe realistisch wird. Er nannte das die kombinatorische Explosion, und Kapitel 1 hat sie in Zahlen vorgeführt.*

Der zweite Winter im Detail.

Jahr	Marktvolumen	
1985	≈ 0,2 Mrd. $	
1988	≈ 1 Mrd. $	Höhepunkt
1993	nahe null	

Der Zusammenbruch hatte einen unspektakulären Grund: Gewöhnliche Arbeitsplatzrechner wurden schnell genug, um LISP auszuführen. Die Spezialrechner verloren ihre Berechtigung, und mit ihnen brach die ganze Branche weg, die sich um sie gebildet hatte.

Der Fall der Expertensysteme

Sie waren der größte Erfolg und das lehrreichste Scheitern des Gebiets.

System	Gebiet	Ergebnis
DENDRAL	Massenspektren	wissenschaftlich erfolgreich
MYCIN	Antibiotikawahl	besser als Assistenzärzte
XCON	Rechner-konfiguration	sparte Millionen
PROSPECTOR	Lagerstätten	Einzelerfolg

MYCIN erreichte in einer Bewertung durch Fachleute eine Trefferquote von etwa 65 Prozent bei der Wahl der Therapie und lag damit über den Assistenzärzten. Es wurde nie klinisch eingesetzt — teils wegen Haftungsfragen, teils weil die Eingabe der Befunde länger dauerte als eine ärztliche Entscheidung.

Warum sie trotzdem verschwanden.

Wissenserwerb	Ein Fachmann kann sein Können nicht in Regeln aussprechen
Sprödigkeit	Am Rand des Gebiets versagt das System ohne Vorwarnung
Pflege	XCON hatte am Ende über 10^4 Regeln, deren Wechselwirkungen niemand mehr überblickte
Kein Lernen	Jede Änderung musste von Hand eingetragen werden

Jahr	XCON-Regeln	Pflegeaufwand
1980	750	gering
1984	3300	mittel
1988	$\approx 10\,000$	> 50 Personen

Die letzte Zeile ist der Kern. Ein System, dessen Pflege fünfzig Personen bindet, muss sehr viel einbringen, um sich zu rechnen. Und der Aufwand wuchs überproportional, weil jede neue Regel mit allen bestehenden verträglich sein musste.

Die Sprödigkeit ist das grundsätzliche Problem

Ein regelbasiertes System hat keinen Begriff davon, wo sein Gebiet endet.

Fall	Mensch	Regelsystem
Im Gebiet	sicher	sicher
Am Rand	wird vorsichtig	gleich sicher
Außerhalb	sagt: weiß ich nicht	**antwortet trotzdem**

MYCIN hätte einem Patienten mit einem gebrochenen Bein eine Antibiotikatherapie vorgeschlagen, weil es nur Infektionen kannte und keine Vorstellung davon hatte, dass es etwas anderes geben könnte. Die Regeln passten, und dass die Frage falsch war, konnte das System nicht bemerken.

Das ist keine Nachlässigkeit der Erbauer, sondern eine Folge der abgeschlossenen Weltannahme aus Kapitel 9. Ein System, das nur weiß, was in seinen Regeln steht, kann nicht wissen, was nicht darin steht.

Die Einwände von außen

Zwei Kritiker haben das Gebiet über Jahrzehnte begleitet, und beide sind heute besser zu würdigen als damals.
Dreyfus.

Die These	Könnerschaft beruht auf Vertrautheit mit Situationen, nicht auf dem Befolgen von Regeln
Die Stufen	Der Anfänger folgt Regeln; der Meister sieht unmittelbar, was zu tun ist
Die Folgerung	Wer Regeln abfragt, erhält das Wissen des Anfängers

Diese Beobachtung erklärt den Wissenserwerbsengpass genauer als jede technische Analyse. Ein erfahrener Arzt kann nicht sagen, woran er die Diagnose erkennt, weil er sie nicht aus Regeln ableitet. Was er auf Nachfrage angibt, ist eine nachträgliche Begründung.

Brooks.

Die These	Intelligenz braucht keine innere Weltbeschreibung, sondern eine enge Kopplung an die Umwelt
Der Vorwurf	Die klassische KI baut Modelle der Welt und scheitert daran, sie aktuell zu halten
Der Vorschlag	Die Welt ist ihr eigenes bestes Modell; man schaue lieber nach, statt zu speichern

	klassisch	**Brooks**
Aufbau	Wahrnehmen, modellieren, planen, handeln	**gekoppelte Schichten**
Weltmodell	zentral	keines
Stärke	Planung, Schließen	Robustheit, Tempo
Schwäche	Modellpflege	kein Vorausdenken

Beide Seiten hatten recht und beide zu breit verallgemeinert. Heutige Robotik benutzt gekoppelte Reglerschichten für das Unmittelbare und symbolische Planung für das Langfristige, und niemand hält das noch für einen Widerspruch.

Aufgabe 14.1.

(a) Untersuchen Sie den Wissenserwerbsengpass quantitativ.
(b) Vergleichen Sie, was die Vorhersagen falsch machte.
(c) Bestimmen Sie, was von dem Gebiet geblieben ist.

Lösung Schritt für Schritt.

(a) Der Wissenserwerbsengpass.
Die Rechnung für ein mittleres System.

Größe	**Wert**	
Regeln	5000	
Regeln je Personentag	3	mit Prüfung
Erstellung	1667 Tage	$\approx$ 7 Personenjahre
Beteiligte Fachleute	2–3	neben ihrer Arbeit
Kalenderzeit	$\approx$ 4 Jahre	

Die Wechselwirkung als eigentliches Hindernis.

Regeln	**Paare**	**zu prüfen**	**Zeit**
100	4950	wenige	Tage
1000	$5 \cdot 10^5$	viele	Monate
10^4	$5 \cdot 10^7$	unmöglich	—

Die Zahl der möglichen Wechselwirkungen wächst quadratisch, die Zahl der Regeln linear. Ab einer gewissen Größe verbringt man mehr Zeit damit, Widersprüche zu suchen, als neue Regeln zu schreiben. Das ist der Punkt, an dem XCON angekommen war.

Was daraus folgt.

Modularität	Regeln in Gruppen ordnen, die einander nicht sehen
Prüfbarkeit	Ein Löser findet Widersprüche automatisch (Kapitel 10)
Enges Gebiet	Lieber zehn kleine Systeme als eines für alles

Die zweite Zeile ist der Grund, warum die Beschreibungslogiken überlebt haben und die freien Regelsysteme nicht. Wer seine Sprache so einschränkt, dass eine Maschine die Widersprüche findet, kann Wissensbasen bauen, die zehnmal größer sind.

(b) Warum die Vorhersagen falsch waren.

Angenommen	Tatsächlich	Faktor
Rechenleistung reicht bald	fehlte um Größenordnungen	10^6
Wissen ist bald eingetragen	Engpass bis heute	—
Schwer ist, was Menschen schwer fällt	**umgekehrt**	—

Die dritte Zeile im Detail.

Aufgabe	für Menschen	für Maschinen
Integralrechnung	schwer	seit 1960er lösbar
Schach	sehr schwer	seit 1997
Sätze beweisen	schwer	teilweise
Ein Glas greifen	trivial	jahrzehntelang offen
Ein Gesicht erkennen	trivial	lange offen
Alltagsschlüsse	trivial	**bis heute schwierig**

Diese Umkehrung ist als Moravecsches Paradox bekannt. Die Erklärung: Was Menschen bewusst und mühsam tun, ist evolutionär jung und explizit; was mühelos geht, beruht auf sehr alten und sehr aufwendigen Anpassungen, die niemand introspektiv erreicht.

Für die symbolische KI hieß das: Man wählte als erste Aufgaben genau die, die Menschen für Kennzeichen der Intelligenz halten, löste sie, und stellte fest, dass man dem Ziel keinen Schritt näher war.

Was man aus Cyc lernen kann.

Die Annahme	Alltagswissen ist eine große, aber endliche Menge von Aussagen
Der Befund	Die Menge ist größer als gedacht, und ihre Ränder sind unscharf
Das Grundsätzliche	Vieles Alltagswissen ist gar nicht in Aussagen fassbar

Die dritte Zeile ist Dreyfus' Einwand, dreißig Jahre später empirisch bestätigt. Es ist nicht so, dass man noch nicht genug eingetragen hätte; es ist so, dass ein erheblicher Teil des Wissens die Form von Aussagen nicht annimmt.

(c) Was geblieben ist.

Verfahren	Kapitel	heutiger Einsatz
A*	3	Navigation, Spiele, Robotik
Alpha-Beta	4	Spielprogramme
MCTS	4	Spiele, Planung
Constraint-Löser	6	Dienstpläne, Fertigung
SAT-Löser	7	**Schaltungsprüfung, Verifikation**
Unifikation	8	Übersetzer, Typprüfung
Beschreibungslogik	10	medizinische Terminologien
Planer	11	Logistik, Raumfahrt
Bayes-Netze	12	Diagnose, Risikoanalyse
Begründungsverwaltung	13	Tabellenkalkulation, Übersetzer

Zehn Verfahren, alle im täglichen Einsatz, und keines wird künstliche Intelligenz genannt. Die letzte Zeile ist besonders reizvoll: Eine Tabellenkalkulation, die nach einer Änderung genau die abhängigen Zellen neu berechnet, führt dieselbe Buchführung wie ein Wahrheitspflegesystem.

Was nicht geblieben ist.

Ansatz	Stand	Grund
Große Regelsysteme	aufgegeben	Pflege
Freie Rahmensysteme	aufgegeben	unklare Bedeutung
Gewissheitsfaktoren	aufgegeben	durch Bayes ersetzt
Reine Default-Logik	aufgegeben	zu teuer
Vollständiges Alltagswissen	offen	Ziel selbst fraglich

Das Muster hinter beiden Listen.

Überlebt hat	Was ein eng umrissenes Problem exakt löst und sich prüfen lässt
Gescheitert ist	Was allgemeine Intelligenz durch Ansammeln von Wissen erreichen wollte

Das ist eine ernüchternde und keine deprimierende Bilanz. Ein Fach, das in fünfzig Jahren zehn Verfahren hervorbringt, die in jedem Navigationsgerät, jedem Chip-Entwurf und jeder klinischen Terminologie stecken, hat nicht versagt. Es hat nur etwas anderes erreicht als angekündigt.

Und der KI-Effekt aus dem Vorwort sorgt dafür, dass niemand es bemerkt: Sobald ein Verfahren zuverlässig arbeitet, heißt es Optimierung, Verifikation oder Terminologieverwaltung, und die künstliche Intelligenz ist wieder das, was man noch nicht kann.

Dreyfus[10] formulierte den philosophischen Einwand; Brooks[4] den technischen; Buchanan und Shortliffe[6] dokumentieren MYCIN samt seiner Bewertung; Lenat und Guha[18] beschreiben die Annahmen hinter Cyc.

15 Der Stand im Jahr 2026

Was gelöst ist und was nicht

Frage	Stand	Bemerkung
Heuristische Suche	gelöst	seit 1968
Spiele mit vollständiger Information	gelöst	Suche und Lernen verbunden
Erfüllbarkeit in der Praxis	gelöst	**trotz NP-Vollständigkeit**
Beschreibungslogisches Schließen	gelöst	entscheidbar und schnell genug
Planen im Großen	in Arbeit	gute Heuristiken, harte Fälle bleiben
Symbolisches und Lernendes verbinden	in Arbeit	**seit etwa 2020**
Wissen automatisch gewinnen	in Arbeit	neue Möglichkeiten
Prüfbare Antworten	in Arbeit	Löser als Kontrolle
Alltagsschlüsse	offen	**seit 1956**
Qualifikationsproblem	offen	grundsätzlich
Bedeutung von Zeichen	offen	philosophisch

Die dritte Zeile des oberen Abschnitts verdient Beachtung. Ein NP-vollständiges Problem gilt in der Praxis als gelöst, weil die Fälle, die tatsächlich auftreten, Struktur haben. Das ist die wichtigste methodische Lehre dieses Bandes.

Die vorletzte Zeile ist die dauerhafteste.

Das Problem	Keine Bedingungsliste für eine Handlung lässt sich abschließen
Warum grundsätzlich	Die Ausnahmen sind nicht aufzählbar; jede neue Situation kann eine neue liefern
Der Umgang	Vorbehalte statt Vollständigkeit, und die Bereitschaft, Schlüsse zurückzunehmen

Kapitel 13 hat gezeigt, dass sich dieser Umgang formalisieren lässt und dabei teuer wird. Das ist der Stand: Man weiß genau, worin die Schwierigkeit besteht, und hat keine wohlfeile Antwort.

Fünf Entwicklungen

Erstens: Löser als Werkzeug statt als System.
Die klassische Bauweise war ein System, das aus Regeln schließt. Heute ist der Löser ein Bestandteil, den etwas anderes aufruft.

	früher	**heute**
Der Löser ist	**das System**	ein Werkzeug
Die Aufgabe stellt	ein Mensch	**ein anderes Programm**
Die Übersetzung macht	ein Mensch	zunehmend Software
Die Antwort prüft	niemand	der Löser selbst

Die letzte Zeile ist die wichtigste Verschiebung. Ein SAT- oder SMT-Löser liefert nicht nur eine Antwort, sondern bei Erfüllbarkeit eine Belegung, die sich in linearer Zeit nachprüfen lässt, und bei Unerfüllbarkeit auf Wunsch einen Beweis. Man muss dem Löser nicht vertrauen.

Vorgang	Aufwand	
Lösung finden	exponentiell	im schlechtesten Fall
Lösung prüfen	linear	**immer**

Dieses Gefälle ist die Definition von NP aus Band 3, und es ist hier praktisch nutzbar: Man darf ein unzuverlässiges Verfahren einsetzen, solange ein zuverlässiges die Antwort prüft. Für Systeme, die vorschlagen statt zu entscheiden, ist das die entscheidende Eigenschaft.

Zweitens: SMT statt SAT.
Reine Aussagenlogik ist umständlich, sobald Zahlen, Felder oder Funktionen vorkommen.

Der Gedanke	Ein SAT-Löser behandelt die logische Struktur, Fachlöser behandeln die Theorien
Theorien	Ganze Zahlen, reelle Zahlen, Bitvektoren, Felder, uninterpretierte Funktionen
Zusammenspiel	Der SAT-Teil schlägt eine Belegung vor, die Fachlöser prüfen sie und melden Konflikte zurück

Programm-verifikation	Kann diese Funktion je einen Bereichsfehler erzeugen?
Symbolische Ausführung	Welche Eingabe erreicht diesen Zweig?
Übersetzerbau	Ist diese Optimierung wirkungsgleich? (Band 6)

Die Rückmeldung der Fachlöser als neue Klausel ist genau das Klauselllernen aus Kapitel 7, nur dass der Konflikt nicht aus der Formel, sondern aus der Arithmetik stammt. Dasselbe Verfahren, ein anderer Lieferant für die Widersprüche.

Drittens: Symbolisches und Lernendes verbinden.

	Symbolisch	Lernend
Stärke	exakt, erklärbar	**Wahrnehmung, Muster**
Schwäche	sprödes Verhalten	keine Garantien
Wissen kommt	von Hand	aus Daten
Fehlerart	fehlende Regel	falsche Zuversicht

Die Schwächen sind komplementär, und das ist der Grund für die Verbindung. Ein lernendes Verfahren schlägt vor, ein Löser prüft; ein lernendes Verfahren übersetzt eine Frage in eine Formel, ein Löser beantwortet sie exakt.

AlphaGo-Muster	Gelernte Bewertung leitet die Monte-Carlo-Suche (Kapitel 4)
Übersetzungs-muster	Ein Modell erzeugt aus Umgangssprache eine formale Aufgabe; ein Löser bearbeitet sie
Prüfmuster	Ein Modell erzeugt einen Vorschlag; ein Beweiser bestätigt oder verwirft ihn

Beim dritten Muster liegt die Verantwortung eindeutig: Das Modell darf raten, der Beweiser entscheidet. Bei Aufgaben, für die es einen Prüfer gibt, ist das eine saubere Arbeitsteilung; für Aufgaben ohne Prüfer bleibt sie unerreichbar.

Viertens: Wissen automatisch gewinnen.
Der Engpass aus Kapitel 14 hat sich verschoben.

Aufgabe	früher	heute
Erster Entwurf	Wochen	Stunden
Prüfen	Wochen	**unverändert Wochen**
Pflegen	dauerhaft	unverändert dauerhaft

Nur die erste Zeile hat sich geändert, und sie war nie der Engpass. Ein Entwurf, den niemand geprüft hat, ist keine Wissensbasis, sondern eine Behauptungssammlung. Die eigentliche Arbeit bleibt.

Immerhin: Die Prüfung lässt sich teilweise automatisieren, sobald das Wissen in einer entscheidbaren Sprache steht. Ein Einordnungsprogramm findet Widersprüche, die ein Mensch nicht fände, und das verschiebt das Verhältnis von Schreiben zu Prüfen zugunsten des Schreibens.

Fünftens: Formale Verifikation wird gewöhnlich.

Gebiet	Stand	
Schaltungsentwurf	Routine	seit den 1990er Jahren
Betriebssystemkerne	möglich	seit etwa 2009
Übersetzer	möglich	
Verteilte Protokolle	in Arbeit	Band 9
Gewöhnliche Anwendungen	selten	Aufwand

Hinter jedem dieser Erfolge steht ein Löser aus diesem Band. Es ist der stillste und größte Beitrag der symbolischen KI zur Informatik, und er wird nicht als solcher wahrgenommen, weil er unter dem Namen Verifikation läuft.

Drei Einsichten

Erstens: NP-schwer heißt nicht unlösbar.

Problem	Theorie	Praxis
SAT	NP-vollständig	10^7 Variablen
Constraint-Probleme	NP-schwer	industriell im Einsatz
Planen	PSPACE-schwer	brauchbar

Die Komplexitätsklasse beschreibt den schlechtesten Fall über alle Eingaben. Reale Eingaben sind ein winziger und sehr strukturierter Teil davon. Wer aus der Klasse auf die Praxis schließt, macht denselben Fehler wie beim Verdichten in Band 7: Er behandelt die Daten, als wären sie zufällig.

Zweitens: Die Darstellung schlägt das Verfahren.

Beispiel	Kapitel	Gewinn
Acht Damen	1	878 000
SAT-Kodierung	7	Größenordnungen
Bayes-Netz statt Tabelle	12	6000
Baumstruktur	6	exponentiell zu polynomial

Viermal derselbe Befund. Kein Suchverfahren holt eine schlechte Darstellung wieder ein, und keine gute Darstellung braucht ein besonders raffiniertes Verfahren. Das ist die praktisch folgenreichste Einsicht des ganzen Bandes.

Drittens: Was zuverlässig funktioniert, verliert seinen Namen.

Verfahren	damals	heute
A*	KI	Wegplanung
SAT	KI	Verifikation
Constraint-Löser	KI	Optimierung
Beschreibungs-logik	KI	Terminologie-verwaltung

Der KI-Effekt ist keine bloße Sprachregelung. Er hat eine sachliche Grundlage: Sobald man genau versteht, wie ein Verfahren arbeitet und wo seine Grenzen liegen, erscheint es nicht mehr geheimnisvoll. Intelligenz ist der Name für das, was wir noch nicht durchschauen.

Band 13 wendet sich den lernenden Systemen zu. Vieles aus diesem Band kehrt dort wieder — Suche als Optimierung, Wahrscheinlichkeit als Grundlage, die Frage nach der Darstellung — und die Schwächen sind, wie Kapitel 15 gezeigt hat, gerade die umgekehrten.

Wir überschätzen, was in einem Jahr geschieht,
und unterschätzen, was in zehn geschieht.

— eine Beobachtung, die in verschiedenen Fassungen umläuft.
Für dieses Gebiet gilt beides zugleich: Die Vorhersagen von 1965 waren zu kühn, und niemand hätte 1990 geglaubt, dass ein NP-vollständiges Problem einmal industrielle Routine wird.

Part V

Anhang

A Formeln und Zahlen

Aufwand der Suchverfahren

Verfahren	vollst.	opt.	Zeit	Speicher
Breitensuche	ja	ja*	$\mathcal{O}(b^d)$	$\mathcal{O}(b^d)$
Tiefensuche	nein	nein	$\mathcal{O}(b^m)$	$\mathcal{O}(bm)$
Iterative Vertiefung	ja	ja*	$\mathcal{O}(b^d)$	$\mathcal{O}(bd)$
Kostensuche	ja	ja	$\mathcal{O}(b^{1+C/\varepsilon})$	hoch
Zweiseitig	ja	ja*	$\mathcal{O}(b^{d/2})$	$\mathcal{O}(b^{d/2})$
Gierig	nein	nein	$\mathcal{O}(b^m)$	$\mathcal{O}(b^m)$
A*	ja	ja**	exponentiell	$\mathcal{O}(b^d)$
IDA*	ja	ja**	exponentiell	$\mathcal{O}(d)$
Alpha-Beta	—	exakt	$\mathcal{O}(b^{d/2})$	$\mathcal{O}(bd)$

* *bei gleichen Schrittkosten.* ** *bei zulässiger bzw. konsistenter Heuristik.*

Begriffe und Formeln

A*-Bewertung	$f(n) = g(n) + h(n)$
Zulässig	$h(n) \leq h^*(n)$
Konsistent	$h(n) \leq c(n, n') + h(n')$
Effektiver Verzweigungsgrad	$N + 1 = \sum_{i=0}^{d} (b^*)^i$
Alpha-Beta, beste Ordnung	$b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$
UCT	$\frac{w_k}{n_k} + c\sqrt{\frac{\ln n}{n_k}}$
Abkühlen	$P(\text{annehmen}) = e^{\Delta/T}$
Neustarts	$\mathbb{E}[\text{Durchläufe}] = 1/p$
Bayes	$P(H \mid E) = \frac{P(E \mid H)P(H)}{P(E)}$
Netzfaktorisierung	$P(x_1..x_n) = \prod_i P(x_i \mid \text{Eltern}(x_i))$
Resolution	$\frac{(P \vee L),\ (Q \vee \neg L')}{(P \vee Q)\sigma}$

Größenordnungen

Aufgabe	**Zustände**	**Verzweigung**
Acht-Puzzle	$1{,}8 \cdot 10^5$	≈ 3
Fünfzehn-Puzzle	10^{13}	≈ 3
Rubik-Würfel	$4{,}3 \cdot 10^{19}$	≈ 13
Dame	$5 \cdot 10^{20}$	≈ 10
Schach	$\approx 10^{47}$	≈ 35
Go	$\approx 10^{170}$	≈ 250
Weltall, Atome	$\approx 10^{80}$	—

Löser, Stand 2026	**Größe**	**Zeit**
SAT, Anwendung	10^7 Variablen	Minuten
SAT, zufällig am Übergang	$\approx 10^3$	Stunden
Constraint-Probleme	10^5 Variablen	Minuten
Beschreibungslogik	$3{,}5 \cdot 10^5$ Begriffe	Minuten
Planen	10^3 Handlungen	Sekunden

B Landkarte des Bandes

Wo was steht

Kap.	Thema	Verweise
1	Zeichen, Zustandsräume	Band 2, Band 6 (LISP)
2	Uninformierte Suche	Band 1, Band 11
3	Heuristische Suche	Band 11 (kürzeste Wege)
4	Suche gegen Gegner	Band 3 (Schranken)
5	Lokale Suche	Band 7 (Nichtbeliebigkeit)
6	Nebenbedingungen	**Band 11 (Baumweite)**
7	SAT	**Band 3 (NP)**, Band 6
8	Prädikatenlogik	**Band 2 (Unentscheidbarkeit)**
9	Prolog	Band 6 (Linksrekursion)
10	Wissen darstellen	Band 10 (Ontologien)
11	Planen	Band 9 (Pläne, Strategien)
12	Unsicherheit	**Band 7 (Diagnose)**
13	Nichtmonotones	Band 2 (Selbstbezug)
14	Bilanz	Band 13
15	Stand 2026	Band 13, Band 15

Sechs Verbindungen zur Mathematikreihe

Zustandsraum	Graphen, Bäume, Halbordnungen
A*-Optimalität	Dreiecksungleichung, metrische Räume
Resolution	Formale Systeme, Vollständigkeitssätze
Unifikation	Termalgebren, allgemeinste Lösungen
Bayes-Netze	Bedingte Unabhängigkeit, Faktorisierung
Abkühlen	Markow-Ketten, Konvergenz

Die dritte Zeile ist die tiefste Verbindung. Robinsons Vollständigkeitssatz für die Resolution ist ein Gegenstück zu Gödels Vollständigkeitssatz: Was semantisch folgt, lässt sich syntaktisch ableiten. Und Gödels Un*vollständigkeitssatz aus Band 2 markiert, wo diese Übereinstimmung endet.*

C Glossar

Begriffe

Abgeschlossene Welt	Was nicht ableitbar ist, gilt als falsch
Alpha-Beta	Exaktes Abschneiden bei der Spielbaumsuche
Default	Regel, die unter Vorbehalt gilt
DPLL	Grundverfahren für die Erfüllbarkeit
Erweiterung	Stimmige Menge von Default-Schlüssen
Heuristik	Schätzung der Restkosten
Klausel	Disjunktion von Literalen
Klauselllernen	Aus einem Konflikt eine neue Klausel ableiten
Kollider	Knoten mit zwei eingehenden Pfeilen
Konsistent	Heuristik erfüllt die Dreiecksungleichung
Monotonie	Neues Wissen hebt alte Schlüsse nicht auf
Rahmenproblem	Was ändert sich bei einer Handlung nicht?
Resolution	Einziges Schlussprinzip für die Prädikatenlogik
Skolemisierung	Existenzquantoren durch Funktionen ersetzen
Sprödigkeit	Versagen ohne Vorwarnung am Rand des Gebiets
STRIPS	Handlungen über Vor-, Add- und Streichliste
Unifikation	Zwei Ausdrücke durch Einsetzen gleichmachen
Wegerklären	Eine Ursache macht die andere unwahrscheinlicher
Zulässig	Heuristik überschätzt nie

D Zeittafel

Von 1950 bis 2026

1950	Turing: Computing Machinery and Intelligence
1955	Newell, Simon: Logic Theorist
1956	**Dartmouth: der Name entsteht**
1958	McCarthy: LISP; Programs with Common Sense
1959	Samuel: lernendes Dameprogramm
1960	Davis, Putnam
1962	DPLL
1965	**Robinson: Resolution und Unifikation**
1965	Weizenbaum: ELIZA
1968	**Hart, Nilsson, Raphael: A***
1969	McCarthy, Hayes: Rahmenproblem
1971	**Fikes, Nilsson: STRIPS**
1972	Colmerauer, Roussel: Prolog
1972	Dreyfus: What Computers Can't Do
1973	Lighthill-Bericht
1974	Minsky: Rahmen
1974	Kowalski: Logik als Programmiersprache
1974–80	**Erster KI-Winter**
1975	Knuth, Moore: Analyse von Alpha-Beta
1976	Newell, Simon: Symbolsystemhypothese
1976	Shortliffe: MYCIN
1978	Wahrheitspflegesysteme

1980	**Reiter: Default-Logik**
1980	Searle: Das chinesische Zimmer
1980–87	**Blütezeit der Expertensysteme**
1983	Kirkpatrick u. a.: simuliertes Abkühlen
1984	Lenat: Cyc beginnt
1984	Buchanan, Shortliffe: MYCIN dokumentiert
1985	Korf: IDA*
1987–93	**Zweiter KI-Winter**
1988	**Pearl: Bayes-Netze**
1991	Brooks: Intelligence Without Representation
1995	Graphplan
1997	**Deep Blue schlägt Kasparow**
1999	Marques-Silva, Sakallah: Klauselllernen
2000	Vorwärtssuche mit Lockerungsheuristik
2001	Chaff: SAT wird praktisch brauchbar
2004	OWL: Beschreibungslogik im Netz
2006	Monte-Carlo-Baumsuche
2009	Verifizierter Betriebssystemkern
2016	**AlphaGo: Suche und Lernen verbunden**
2020er	Löser als Werkzeug lernender Systeme
2026	Prüfen statt Raten

E Kommentierte Quellen

[1] Baader, F., Calvanese, D., McGuinness, D., Nardi, D., Patel-Schneider, P.: *The Description Logic Handbook*. 2. Aufl., Cambridge University Press 2007.

Das Standardwerk zu den Beschreibungslogiken: Ausdrucksstärke, Entscheidbarkeit, Verfahren, Anwendungen. Zeigt sorgfältig, wo jede Erweiterung der Sprache die Berechenbarkeit kostet.

[2] Biere, A., Heule, M., van Maaren, H., Walsh, T. (Hrsg.): *Handbook of Satisfiability*. 2. Aufl., IOS Press 2021.

Über tausend Seiten zu SAT: Verfahren, Kodierungen, Anwendungen, Theorie. Die maßgebliche Quelle für alles, was mit Erfüllbarkeit zu tun hat.

[3] Bratko, I.: *Prolog Programming for Artificial Intelligence*. 4. Aufl., Pearson 2011.

Die verbreitetste praktische Einführung. Lehrt Prolog anhand klassischer KI-Aufgaben und ist gerade deshalb doppelt nützlich: Man lernt die Sprache und die Verfahren zugleich.

[4] Brooks, R.: *Intelligence Without Representation*. Artificial Intelligence 47, 1991.

Die schärfste Kritik am Weltmodell-Ansatz, geschrieben von jemandem, der stattdessen laufende Roboter baute. Der Satz, die Welt sei ihr eigenes bestes Modell, hat die Robotik nachhaltig verändert.

[5] Browne, C., Powley, E., Whitehouse, D., u. a.: *A Survey of Monte Carlo Tree Search Methods*. IEEE Trans. CIAIG 4(1), 2012.

Die Übersicht über MCTS, kurz bevor das Verfahren durch die Verbindung mit lernenden Bewertungen berühmt wurde. Ordnet die Varianten und erklärt die Abwägung zwischen Erkunden und Nutzen sauber.

[6] Buchanan, B., Shortliffe, E.: *Rule-Based Expert Systems: The MYCIN Experiments*. Addison-Wesley 1984.

Die ehrlichste Dokumentation eines Expertensystems. *Enthält nicht nur den Aufbau, sondern die Bewertung durch Fachleute und eine ausführliche Diskussion darüber, warum das System nie eingesetzt wurde. Lehrreicher als jeder Erfolgsbericht.*

[7] Colmerauer, A., Roussel, P.: *The Birth of Prolog*. HOPL-II, 1993.

Die Erfinder erzählen, wie aus einem Vorhaben zur Sprachverarbeitung eine Programmiersprache wurde. Zeigt gut, wie eng logisches Programmieren und Resolution zusammenhängen und wie viele Entscheidungen pragmatisch fielen.

[8] Davis, M., Putnam, H.: *A Computing Procedure for Quantification Theory*. Journal of the ACM 7(3), 1960.

Der Ausgangspunkt aller heutigen SAT-Löser. Zwei Jahre später ergänzten Davis, Logemann und Loveland das Verzweigen, und die Grundstruktur steht seither unverändert.

[9] Dechter, R.: *Constraint Processing*. Morgan Kaufmann 2003.

Die maßgebliche Darstellung der Constraint-Verfahren: Konsistenzstufen, Suche, Zerlegung nach Struktur. Besonders gut beim Zusammenhang zwischen Graphstruktur und Aufwand.

[10] Dreyfus, H.: *What Computers Still Can't Do*. MIT Press 1992.

Die überarbeitete Fassung der Kritik von 1972. Dreyfus wurde jahrzehntelang belächelt und hatte in den entscheidenden Punkten recht. Seine Analyse des Wissenserwerbsengpasses ist bis heute die beste, die es gibt.

[11] Fikes, R., Nilsson, N.: *STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving*. Artificial Intelligence 2, 1971.

Führte die Darstellung von Handlungen über Vorbedingungen und zwei Listen ein und umging damit das Rahmenproblem. Praktisch jeder heutige Planer benutzt eine Weiterentwicklung davon.

[12] Ghallab, M., Nau, D., Traverso, P.: *Automated Planning and Acting*. Cambridge University Press 2016.

Die gründliche Darstellung des Planens, von den klassischen Verfahren bis zu Planung unter Unsicherheit und in der Zeit. Der zweite Teil über das Handeln ist die Antwort auf Brooks' Einwand.

[13] Hart, P., Nilsson, N., Raphael, B.: *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*. IEEE Trans. SSC 4(2), 1968.

***A**. Enthält den Beweis der Optimalität und die Bedingungen an die Heuristik. Bemerkenswert klar geschrieben, und das Verfahren steckt heute in jedem Navigationsgerät.*

[14] Kirkpatrick, S., Gelatt, C., Vecchi, M.: *Optimization by Simulated Annealing*. Science 220, 1983.

Überträgt einen Gedanken der Metallurgie auf die Optimierung. Kurz, elegant und außerordentlich einflussreich. Der Konvergenzbeweis ist wahr und praktisch unbrauchbar, was der Aufsatz selbst einräumt.

[15] Knuth, D., Moore, R.: *An Analysis of Alpha-Beta Pruning*. Artificial Intelligence 6, 1975.

Die genaue Analyse, wie viele Blätter Alpha-Beta bei welcher Zugreihenfolge betrachtet, samt Beweis der unteren Schranke. Damit steht fest, dass alle weiteren Verbesserungen nur die Sortierung betreffen können.

[16] Korf, R.: *Depth-First Iterative-Deepening: An Optimal Admissible Tree Search*. Artificial Intelligence 27, 1985.

Zeigte, dass ein Verfahren, das als Notbehelf galt, asymptotisch optimal ist. Enthält auch IDA, die speicherarme Fassung von A*, ohne die viele Aufgaben unlösbar wären.*

[17] Kowalski, R.: *Predicate Logic as Programming Language*. IFIP 1974.

Die Trennung von Logik und Steuerung. Ein Programm ist eine logische Beschreibung, und die Abarbeitung ist eine Zutat. Kapitel 9 zeigt, wo diese Trennung in der Praxis bricht.

[18] Lenat, D., Guha, R.: *Building Large Knowledge-Based Systems*. Addison-Wesley 1989.

Die Begründung des Cyc-Vorhabens. Lesenswert als Zeitdokument und als Beleg für eine Annahme, die sich nicht bestätigt hat: dass Alltagswissen eine große, aber endliche Menge von Aussagen sei.

[19] Marques-Silva, J., Sakallah, K.: *GRASP: A Search Algorithm for Propositional Satisfiability*. IEEE Trans. Computers 48(5), 1999.

Führte das konfliktgetriebene Klausellernen ein und leitete damit den Leistungssprung ein, der SAT von einem theoretischen Problem zu einem industriellen Werkzeug machte.

[20] McCarthy, J.: *Programs with Common Sense*. Teddington Conference 1959.

Der Entwurf eines Systems, das Wissen logisch darstellt und daraus Schlüsse zieht. Enthält bereits die Forderung, dass ein System durch Hinzufügen von Aussagen lernen können soll, ohne umprogrammiert zu werden. Sechs Seiten, die das Programm der logikbasierten KI aufstellen.

[21] McCarthy, J., Hayes, P.: *Some Philosophical Problems from the Standpoint of Artificial Intelligence*. Machine Intelligence 4, 1969.

Benennt das Rahmenproblem und unterscheidet es von den verwandten Schwierigkeiten. Eine der wenigen Arbeiten, die ein Problem so klar formulieren, dass es ein ganzes Teilgebiet begründet.

[22] Minsky, M.: *A Framework for Representing Knowledge*. MIT AI Memo 306, 1974.

Die Rahmen als Wissensform: Felder, Vorgabewerte, Vererbung. Einflussreich weit über die KI hinaus — die objektorientierte Programmierung übernimmt dieselbe Struktur.

[23] Newell, A., Simon, H.: *Computer Science as Empirical Inquiry: Symbols and Search*. CACM 19(3), 1976.

Die Turing-Award-Vorlesung mit der Symbolsystemhypothese. *Kurz, klar und ausdrücklich als empirische Behauptung formuliert, nicht als Beweis. Die zweite Hypothese über die heuristische Suche ist der Titelgeber dieses Bandes.*

[24] Pearl, J.: *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley 1984.

Die maßgebliche theoretische Behandlung der Heuristiken: Woher sie kommen, wann sie zulässig sind, wie man ihre Güte misst. Das Verfahren der Regellockerung ist hier systematisch entwickelt.

[25] Pearl, J.: *Probabilistic Reasoning in Intelligent Systems*. Morgan Kaufmann 1988.

Das Buch, das die Wahrscheinlichkeitsrechnung in die KI zurückbrachte. *Bayes-Netze, ihre Semantik, die Inferenzverfahren und die Kritik an den Gewissheitsfaktoren. Pearl erhielt dafür 2011 den Turing-Award.*

[26] Reiter, R.: *A Logic for Default Reasoning*. Artificial Intelligence 13, 1980.

Der einflussreichste Formalismus für nichtmonotones Schließen. Die Idee, einen Vorbehalt in die Schlussregel selbst einzubauen, ist einfach und hat weitreichende Folgen, unter anderem die Existenz mehrerer oder gar keiner Erweiterung.

[27] Robinson, J. A.: *A Machine-Oriented Logic Based on the Resolution Principle*. Journal of the ACM 12(1), 1965.

Ein einziges Schlussprinzip für die gesamte Prädikatenlogik, dazu die Unifikation. *Eines der wirkungsvollsten Ergebnisse der Informatik überhaupt: Es steckt in Beweisern, in Prolog und in der Typprüfung funktionaler Sprachen.*

[28] Russell, S., Norvig, P.: *Artificial Intelligence: A Modern Approach*. 4. Aufl., Pearson 2020.

Das Standardlehrbuch. *Über tausend Seiten, die praktisch das gesamte Gebiet abdecken, mit Beweisen, Beispielen und einer ungewöhnlich guten historischen Einordnung am Ende jedes Kapitels. Wenn Sie ein Buch kaufen, dann dieses.*

Wenn Sie nur drei Bücher kaufen

[28]	**Russell/Norvig**	Zum Lernen und Nachschlagen. Deckt alles ab
[25]	**Pearl**	Zum Umgang mit Unsicherheit. Verändert die Sicht auf das ganze Gebiet
[6]	**Buchanan/Shortliffe**	Zum Verstehen, warum es nicht so kam. Ehrlicher als jeder Rückblick

Wer danach noch etwas möchte: [23] Newell und Simon sind neunzehn Seiten und formulieren die Grundthese des Gebiets. [10] Dreyfus liest sich als Gegengift, und beide zusammen sind lehrreicher als jedes einzeln.